Big Data

Big data is a term that refers to the large, complex, and diverse sets of data that are generated from various sources and applications. Big data can be structured, unstructured, or semi-structured, and can have different formats, types, and qualities. Big data can be analyzed for insights that can lead to better decisions and actions.

Some of the main characteristics of big data are:

- Volume: The amount of data that is generated and stored. Big data can range from terabytes to petabytes or even exabytes of data.
- Variety: The diversity of data sources, formats, and types. Big data can include text, images, audio, video, sensor data, web logs, social media data, etc.
- Velocity: The speed at which data is generated, collected, and processed. Big data can be produced and consumed in real-time or near-real-time, requiring fast and efficient processing and analysis.
- Veracity: The quality and reliability of data. Big data can be noisy, incomplete, inconsistent, or inaccurate, requiring data cleansing and validation techniques.
- Value: The potential benefits and outcomes that can be derived from data analysis. Big data can provide valuable insights, patterns, trends, predictions, and recommendations that can improve decision making and performance.

Some of the main challenges of big data are:

- Storage: The need for scalable and cost-effective storage solutions that can handle the large and diverse data sets.
- Processing: The need for distributed and parallel computing frameworks that can process and analyze the data efficiently and effectively.
- Analysis: The need for advanced and innovative data mining, machine learning, and artificial intelligence techniques that can extract meaningful and actionable insights from the data.
- Visualization: The need for intuitive and interactive data visualization tools that can present the data and insights in a clear and understandable way.
- Security: The need for data protection and privacy mechanisms that can ensure the confidentiality, integrity, and availability of the data and the insights.

Unit 1 - Introduction to Big Data

- Big data means extremely large data sets that may be analyzed computationally to reveal patterns, trends, and associations, especially relating to human behavior and interactions.

- Big data can be structured (often numeric, easily formatted and stored) or unstructured (more free-form, less quantifiable).
- Big data is characterized by the three Vs: volume, velocity, and variety .
  - Volume refers to the amount of data generated and stored.
  - Velocity refers to the speed at which data is created and collected.
  - Variety refers to the diversity and complexity of the data types and sources.
- Big data can be used for various purposes, such as business analytics, scientific research, social media, health care, education, and more .
- Big data poses many challenges and opportunities for data processing, storage, analysis, and visualization .
  - Challenges include scalability, security, privacy, quality, and integration of data.
  - Opportunities include discovering new insights, enhancing decision making, improving efficiency, and creating value.

Types of digital data in big data

Big data is a term that refers to the large and complex datasets that are generated from various sources and require advanced techniques and technologies to store, process, and analyze. Big data can be classified into different types based on the nature, structure, and format of the data. Some of the common types of digital data in big data are:

- **Structured data**: This is the data that has a predefined schema and can be easily organized and stored in relational databases or tables. Structured data is usually numeric or categorical and follows a fixed format. Examples of structured data are transaction records, customer information, sensor readings, etc.
- **Unstructured data**: This is the data that has no predefined schema and cannot be easily organized or stored in relational databases or tables. Unstructured data is usually text, image, audio, video, or other complex formats that require special processing and analysis techniques. Examples of unstructured data are social media posts, emails, documents, images, videos, etc.
- **Semi-structured data**: This is the data that has some elements of structure but also contains unstructured or irregular components. Semi-structured data can be partially organized and stored in relational databases or tables, but also requires additional processing and analysis techniques. Examples of semi-structured data are XML, JSON, HTML, log files, etc.
- **Multistructured data**: This is the data that is composed of different types of data, such as structured, unstructured, and semi-structured data. Multistructured data can be generated from multiple sources and formats and requires complex integration and analysis techniques. Examples of

multistructured data are web pages, social media feeds, online transactions, etc.

History of Big Data Innovation

- Big data is a term that refers to a large set of data that is almost impossible to manage and process using traditional business intelligence tools.
- The term was coined by Roger Mougalas from O'Reilly Media in 2005, only a year after they created the term Web 2.0.
- However, the concept of big data and its analysis dates back to much earlier times, when humans started to collect and store information for various purposes.
- Some of the milestones in the history of big data innovation are:
  - In 1880, the US Census Bureau faced a problem of handling and processing the data collected during the 1880 census, which they estimated would take eight years. They predicted the data from the 1890 census would take more than 10 years to process.
  - In 1890, Herman Hollerith invented a machine that could read data from punched cards and tabulate the results. This machine reduced the time required to process the 1890 census data to less than three years.
  - In 1965, the US Government planned the world's first data center to store 742 million tax returns and 175 million sets of fingerprints on magnetic tape.
  - In 1970, IBM mathematician Edgar F Codd presented his framework for a "relational database", which allowed data to be stored and accessed in a structured and efficient way.
  - In 1989, Tim Berners-Lee proposed the World Wide Web, which enabled the sharing and linking of information across the internet.
  - In 1995, Yahoo! was founded as one of the first web search engines, which indexed and ranked web pages based on keywords and popularity.
  - In 1999, Google introduced the PageRank algorithm, which improved the quality and relevance of web search results by using the link structure of the web as a measure of authority.
  - In 2004, Google published a paper on MapReduce, a programming model for processing large-scale data sets in parallel using clusters of commodity hardware.
  - In 2005, Amazon launched Amazon Web Services (AWS), which offered cloud computing and storage services to businesses and individuals.
  - In 2006, Hadoop, an open-source framework for distributed processing of big data, was created by Doug Cutting and Mike Cafarella, based on Google's MapReduce and File System papers.

- In 2009, Google released Google Fusion Tables, a web service that allowed users to upload, visualize, and share large data sets online.
- In 2010, Facebook launched the Open Graph protocol, which enabled the integration of social data and activities across different websites and applications.
- In 2011, IBM's Watson, a cognitive computing system, won the Jeopardy! quiz show against human champions, demonstrating the power of natural language processing and machine learning.
- In 2012, Kaggle, a platform for data science competitions, hosted the Heritage Health Prize, which awarded $3 million to the best predictive model for hospitalization risk.
- In 2013, the Obama administration announced the BRAIN Initiative, a research project that aimed to map the human brain and understand its functions using big data technologies.
- In 2014, Alibaba, a Chinese e-commerce giant, set a new record for the largest initial public offering (IPO) in history, raising $25 billion on the New York Stock Exchange, thanks to its massive data-driven business model.
- In 2015, Microsoft launched Cortana Analytics Suite, a cloud-based platform that offered a range of big data and artificial intelligence services, such as speech recognition, sentiment analysis, and predictive analytics.
- In 2016, Google's AlphaGo, a deep learning system, defeated Lee Sedol, a world champion of Go, a complex board game, in a historic match that showcased the advances in reinforcement learning and neural networks.
- In 2017, China announced its plan to become the world leader in artificial intelligence by 2030, investing heavily in big data research and development.
- In 2018, Cambridge Analytica, a political consulting firm, was exposed for harvesting and using the personal data of millions of Facebook users to influence the 2016 US presidential election and the Brexit referendum, sparking a

Introduction to Big Data Platform

- A big data platform is a system that enables the collection, storage, processing, analysis, and visualization of large and complex data sets.
- A big data platform typically consists of several components, such as:
  - Data sources: These are the origin of the data, such as sensors, web logs, social media, databases, etc.
  - Data ingestion: This is the process of acquiring, transforming, and loading the data from the sources to the platform.
  - Data storage: This is the component that stores the data in a scalable and reliable way, such as distributed file systems, cloud storage, data warehouses, etc.

  - Data processing: This is the component that performs various operations on the data, such as cleaning, filtering, aggregating, joining, etc.
  - Data analysis: This is the component that applies various techniques to extract insights from the data, such as statistics, machine learning, data mining, etc.
  - Data visualization: This is the component that presents the data and the analysis results in a graphical and interactive way, such as charts, dashboards, maps, etc.
- A big data platform can be deployed on-premise, in the cloud, or in a hybrid mode, depending on the requirements and the resources of the organization.
- A big data platform can provide various benefits, such as:
  - Improving decision making by providing data-driven insights and predictions.
  - Enhancing customer experience by personalizing products and services.
  - Increasing operational efficiency by optimizing processes and resources.
  - Reducing costs by leveraging economies of scale and automation.
  - Innovating new products and services by discovering new patterns and opportunities.

Drivers for Big Data

- Big data refers to the large and complex datasets that are generated from various sources and require advanced techniques and technologies to store, process, and analyze.
- Some of the drivers for big data are:
  - The increasing volume of data from various domains, such as social media, e-commerce, health care, education, science, etc.
  - The increasing variety of data types and formats, such as structured, unstructured, semi-structured, text, images, videos, audio, etc.
  - The increasing velocity of data generation and consumption, such as real-time, streaming, batch, etc.
  - The increasing veracity of data quality and reliability, such as accuracy, completeness, consistency, etc.
  - The increasing value of data for decision making, innovation, and competitive advantage, such as insights, patterns, trends, predictions, etc.
- These drivers pose various challenges and opportunities for big data management and analytics, such as scalability, performance, security, privacy, integration, visualization, etc.

Big Data Architecture

- Big data architecture is the logical and/or physical layout/structure of how big data is stored, accessed and managed within a big data or IT environment.
- Big data architecture primarily serves as the key design reference for big data infrastructures and solutions. It is created by big data designers/architects before physically implementing a big data solution.
- A big data architecture is designed to handle the ingestion, processing, and analysis of data that is too large or complex for traditional database systems.
- A data architecture describes how data is managed–from collection through to transformation, distribution, and consumption. It sets the blueprint for data and the way it flows through data storage systems.
- A typical big data architecture consists of the following components:
  - Data sources: These are the various sources of data, such as sensors, web logs, social media, etc. that generate or collect data in different formats and volumes.
  - Data ingestion: This is the process of acquiring, importing, and validating data from data sources into a data storage system, such as a data lake or a data warehouse.
  - Data storage: This is the system that stores the ingested data in a scalable and reliable manner, such as a distributed file system, a cloud storage service, or a relational or non-relational database.
  - Data processing: This is the process of transforming, enriching, and aggregating data using various tools and frameworks, such as MapReduce, Spark, or SQL.
  - Data analysis: This is the process of applying analytical techniques, such as machine learning, statistics, or visualization, to extract insights and value from data.
  - Data consumption: This is the process of delivering or presenting the results of data analysis to end users, such as business analysts, data scientists, or customers, using various applications, such as dashboards, reports, or APIs.

Big data characteristics

Big data is a term that refers to data sets that are too large, complex, or diverse to be processed by conventional methods. Big data can be characterized by five Vs: volume, variety, velocity, value, and veracity .

- **Volume** refers to the amount of data that is generated and stored. Big data can range from terabytes to petabytes or even exabytes of data. The volume of data can affect the performance, scalability, and cost of data processing and storage systems .
- **Variety** refers to the types and sources of data that are collected and analyzed. Big data can include structured, semi-structured, or unstructured

data, such as text, images, audio, video, sensor data, web logs, social media posts, etc. The variety of data can pose challenges for data integration, quality, and analysis .

- **Velocity** refers to the speed and frequency at which data is generated, collected, and processed. Big data can be produced in real-time or near real-time, such as streaming data from sensors, mobile devices, social media, or the Internet of Things (IoT). The velocity of data can require fast and efficient data processing and analysis techniques, such as stream processing, in-memory computing, or distributed computing .
- **Value** refers to the usefulness and potential of data for decision making, innovation, or competitive advantage. Big data can provide valuable insights and opportunities for businesses, organizations, or individuals, such as customer behavior, market trends, risk analysis, product development, etc. The value of data can depend on the quality, relevance, and timeliness of the data, as well as the analytical methods and tools used to extract value from the data .
- **Veracity** refers to the accuracy, reliability, and trustworthiness of data. Big data can be affected by noise, inconsistency, incompleteness, or bias, which can reduce the quality and validity of the data. The veracity of data can be ensured by applying data quality, security, and governance measures, such as data cleansing, validation, encryption, or authentication .

These are some of the main characteristics of big data that distinguish it from traditional data and require new and innovative approaches for data management and analysis.

5 Vs of Big Data

Big data is a term that refers to the large and complex datasets that are generated from various sources and require advanced techniques and technologies to process, analyze, and derive insights from. Big data can be characterized by five Vs: volume, velocity, variety, veracity, and value. These are explained below:

- **Volume**: This refers to the amount or size of data that is generated and stored. Big data typically involves terabytes, petabytes, or even exabytes of data. The volume of data can affect the storage, processing, and analysis of big data.
- **Velocity**: This refers to the speed or rate at which data is generated, collected, and analyzed. Big data often involves real-time or near-real-time data streams that need to be processed and analyzed quickly and efficiently. The velocity of data can affect the performance, scalability, and reliability of big data systems.
- **Variety**: This refers to the diversity or heterogeneity of data sources, formats, and types. Big data can include structured, semi-structured, or unstructured data from various sources such as sensors, social media, web logs, images, videos, audio, text, etc. The variety of data can affect the

integration, quality, and analysis of big data.

- **Veracity**: This refers to the accuracy, reliability, and trustworthiness of data. Big data can include noisy, incomplete, inconsistent, or erroneous data that can affect the validity and usefulness of the analysis results. The veracity of data can affect the cleaning, filtering, and verification of big data.
- **Value**: This refers to the potential or actual benefits or outcomes that can be derived from big data analysis. Big data can provide valuable insights, patterns, trends, predictions, or recommendations that can help in decision making, problem solving, innovation, or optimization. The value of data can affect the selection, prioritization, and evaluation of big data projects.

Big Data Technology Components

Big data technology refers to the technical innovation and super-large datasets that enable automated parallel computation, data management schemes, and data mining. Big data technology components are the essential elements that constitute a big data solution. Some of the common components are:

- **Data sources**: These are the origin of the data that is collected, processed, and analyzed by big data solutions. Data sources may be static files produced by applications (such as web server log files), application data stores (such as relational databases), or real-time data sources (such as IoT devices) .
- **Data pipeline**: This is the process of moving and transforming data from the data sources to the data analytics or data warehousing layer. Data pipeline may involve data ingestion, data integration, data quality, data transformation, and data delivery.
- **Data warehousing**: This is the storage and organization of data in a structured and queryable format. Data warehousing may involve data modeling, data schema, data partitioning, data indexing, and data compression.
- **Data analytics**: This is the analysis and visualization of data to derive insights and support decision making. Data analytics may involve data mining, machine learning, natural language processing, business intelligence, and cloud computing .
- **Data delivery**: This is the presentation and dissemination of data analysis results to the end users or applications. Data delivery may involve dashboards, reports, alerts, APIs, and data products .

These components may vary depending on the specific requirements and objectives of the big data solution. However, they are generally interdependent and play a crucial and unique role in ensuring the smooth functioning of the entire big data technology stack.

Big Data importance

Big data is the term used to describe the large and complex datasets that are generated from various sources and applications in the modern world. Big data has become increasingly important for various reasons, such as:

- Big data can provide valuable insights and patterns that can help businesses, governments, and organizations make better decisions, improve efficiency, and enhance performance.
- Big data can enable new innovations and discoveries that can solve complex problems, create new products and services, and advance scientific and social knowledge.
- Big data can empower individuals and communities by giving them access to information, education, and opportunities that can improve their quality of life and well-being.
- Big data can also pose some challenges and risks, such as:
- Big data can require specialized tools and techniques to store, process, analyze, and visualize, which can be costly and complex.
- Big data can raise ethical, legal, and social issues, such as privacy, security, ownership, and accountability, which can affect the rights and responsibilities of data producers, users, and consumers.
- Big data can also create biases, inequalities, and conflicts, such as discrimination, exclusion, and manipulation, which can harm the interests and values of individuals and groups.

Big Data applications

Big data applications are software systems that use big data technologies to process, analyze, and derive insights from large and complex data sets. Big data applications can help companies to make better business decisions, improve customer experience, optimize operations, and create new products or services. Some of the domains where big data applications are widely used are:

- **Healthcare**: Big data applications can help healthcare providers to improve patient care, reduce costs, and enhance research and innovation. For example, big data applications can enable personalized medicine, disease prediction and prevention, drug discovery, and health monitoring.

- **Finance**: Big data applications can help financial institutions to manage risks, detect fraud, enhance customer service, and optimize trading strategies. For example, big data applications can enable credit scoring, sentiment analysis, market surveillance, and algorithmic trading.

- **Retail**: Big data applications can help retailers to understand customer behavior, preferences, and needs, and offer personalized recommendations, promotions, and services. For example, big data applications can enable customer segmentation, loyalty programs, dynamic pricing, and inventory management.

- **Manufacturing**: Big data applications can help manufacturers to improve product quality, efficiency, and safety, and reduce waste and downtime. For example, big data applications can enable predictive maintenance, quality control, supply chain optimization, and smart manufacturing.

- **Education**: Big data applications can help educators and learners to enhance teaching and learning outcomes, and provide personalized and adaptive learning experiences. For example, big data applications can enable learning analytics, curriculum design, student assessment, and feedback.

Big Data features – security, compliance, auditing and protection

- Security: The process of ensuring the confidentiality, integrity and availability of data and resources in a big data environment. Security involves protecting data from unauthorized access, modification, deletion or disclosure, as well as ensuring the availability and performance of the system and its components.
- Compliance: The process of adhering to the laws, regulations, standards and policies that govern the collection, processing, storage and use of data in a big data environment. Compliance involves ensuring that data is collected and used in a lawful, ethical and responsible manner, respecting the rights and preferences of data subjects and stakeholders.
- Auditing: The process of monitoring, reviewing and verifying the activities and operations of a big data environment. Auditing involves collecting and analyzing evidence of data provenance, quality, accuracy, completeness, consistency and security, as well as detecting and reporting any anomalies, errors, breaches or violations.
- Protection: The process of safeguarding data and resources from accidental or intentional damage, loss or corruption in a big data environment. Protection involves implementing backup, recovery, replication, encryption, anonymization and other techniques to prevent or mitigate the impact of data incidents.

Security of Big Data

- Big data refers to the large and complex datasets that are generated from various sources, such as social media, sensors, mobile devices, etc.
- Big data security is the process of protecting the confidentiality, integrity, and availability of big data from unauthorized access, modification, or

disclosure.

- Big data security challenges include:
  - Data volume: The sheer size of big data makes it difficult to store, process, and analyze using traditional security tools and techniques.
  - Data variety: The diversity of data types, formats, and sources poses challenges for data classification, encryption, and access control.
  - Data velocity: The high speed and frequency of data generation and transmission requires real-time security monitoring and analysis.
  - Data veracity: The uncertainty and inconsistency of data quality and accuracy increases the risk of data corruption, manipulation, or falsification.
- Big data security solutions include:
  - Data encryption: The process of transforming data into an unreadable form using a secret key, so that only authorized parties can decrypt and access the data.
  - Data masking: The process of hiding or replacing sensitive data elements with realistic but fictitious values, so that the data can be used for testing, analysis, or sharing without revealing the original data.
  - Data anonymization: The process of removing or modifying personally identifiable information (PII) from data, so that the data cannot be linked to a specific individual or entity.
  - Data access control: The process of defining and enforcing policies and rules that specify who can access, modify, or delete data, and under what conditions.
  - Data auditing: The process of recording and reviewing the activities and events related to data access, modification, or deletion, to ensure compliance with security policies and regulations.
  - Data security analytics: The process of applying advanced techniques, such as machine learning, artificial intelligence, or statistical analysis, to detect and respond to security threats, anomalies, or incidents related to big data.

Compliance of Big Data

- Big data is the term used to describe large and complex datasets that are generated from various sources and require advanced techniques and technologies to store, process, and analyze.
- Compliance of big data refers to the adherence of big data practices and applications to the relevant laws, regulations, standards, and ethical principles that govern the collection, use, and sharing of data.
- Compliance of big data is important for several reasons, such as:
  - Protecting the privacy and security of data subjects and data owners, and respecting their rights and preferences regarding data access and usage.
  - Ensuring the quality, accuracy, and reliability of data and data analy-

sis, and avoiding biases, errors, and misinterpretations that may lead to wrong decisions or actions.
  - Maintaining the trust and reputation of data providers and data users, and avoiding legal, financial, or reputational risks or penalties that may arise from data breaches, misuse, or non-compliance.
- Compliance of big data involves several challenges, such as:
  - The diversity and complexity of data sources, formats, and types, and the lack of common standards or frameworks for data governance and management.
  - The rapid and dynamic nature of data generation, collection, and processing, and the difficulty of ensuring data provenance, traceability, and auditability.
  - The scalability and distributed nature of data storage, processing, and analysis, and the difficulty of ensuring data security, integrity, and availability across different platforms and locations.
  - The variety and variability of data users, purposes, and contexts, and the difficulty of ensuring data consent, transparency, and accountability for data access and usage.
- Compliance of big data requires a combination of technical, organizational, and legal measures, such as:
  - Implementing data protection and security mechanisms, such as encryption, anonymization, authentication, and authorization, to safeguard data confidentiality, integrity, and availability.
  - Adopting data quality and ethics principles, such as accuracy, completeness, relevance, fairness, and responsibility, to ensure data validity, reliability, and usefulness.
  - Applying data governance and management frameworks, such as policies, standards, guidelines, and best practices, to define data roles, responsibilities, and rules, and to monitor and evaluate data activities and outcomes.
  - Complying with data regulations and laws, such as the General Data Protection Regulation (GDPR), the California Consumer Privacy Act (CCPA), and the Health Insurance Portability and Accountability Act (HIPAA), to respect data rights and obligations, and to avoid data violations and sanctions.

Auditing of Big Data

- Big data refers to the large and complex datasets that are generated from various sources and require advanced techniques and technologies to process, analyze and derive value from them .
- Auditing of big data involves the application of audit principles and methods to assess the quality, reliability, security and governance of big data and the related processes, systems and outcomes .
- Auditing of big data can help auditors to:
  - Expand the scope and depth of their audit projects by using larger

and more diverse data sources and samples .
  - Enhance the efficiency and effectiveness of their audit procedures by using automation and artificial intelligence to perform data extraction, transformation, analysis and reporting .
  - Discover new insights and patterns that can improve the audit risk assessment, planning, execution and communication .
  - Provide more value-added and forward-looking recommendations to the auditees and other stakeholders by using predictive and prescriptive analytics .
- Auditing of big data also poses many challenges and risks for auditors, such as:
  - Identifying and accessing the relevant and reliable data sources and ensuring their completeness and accuracy .
  - Protecting the confidentiality, integrity and availability of the data and complying with the applicable laws and regulations .
  - Integrating and harmonizing data from different systems, formats and structures, including structured and unstructured data .
  - Developing and validating the appropriate data models, algorithms and tools to perform data analysis and interpretation .
  - Communicating the audit results and findings in a clear, concise and understandable manner and supporting them with sufficient and appropriate audit evidence .
- Auditing of big data requires auditors to have the necessary skills, knowledge and competencies to understand and apply the big data concepts, techniques and technologies, as well as the relevant audit standards, frameworks and best practices .
- Auditing of big data also requires auditors to collaborate and coordinate with the auditees, management, IT specialists, data scientists and other internal and external parties to ensure the success and quality of the audit projects .

Protection of Big Data

- Big data refers to the large and complex datasets that are generated from various sources, such as social media, sensors, web logs, etc. Big data has the characteristics of volume, velocity, variety, veracity, and value.
- Protection of big data is the process of ensuring the confidentiality, integrity, and availability of big data from unauthorized access, modification, or disclosure. Protection of big data involves the following aspects:
  - Data security: This refers to the measures taken to prevent or detect data breaches, such as encryption, authentication, authorization, auditing, etc. Data security also includes data backup and recovery, data disposal, and data anonymization.
  - Data privacy: This refers to the respect for the rights and preferences of data subjects, such as data minimization, consent, transparency, accountability, etc. Data privacy also includes data protection laws

and regulations, such as GDPR, CCPA, etc.
  - Data ethics: This refers to the moral principles and values that guide the collection, analysis, and use of big data, such as fairness, justice, beneficence, non-maleficence, etc. Data ethics also includes data governance and stewardship, data quality and provenance, and data literacy and education.

Big Data Privacy

- Big data privacy involves properly managing big data to minimize risk and protect sensitive data. Big data comprises large and complex data sets, many traditional privacy processes cannot handle the scale and velocity required.
- Big data privacy is also a matter of customer trust. The more data you collect about users, the easier it gets to "connect the dots": to understand their current behavior, draw inferences about their future behavior, and eventually develop deep and detailed profiles of their lives and preferences.
- Big data includes big privacy concerns in an era of multi-cloud computing, data owners must keep up with both the pace of data growth and the proliferation of regulations that govern it—especially regulations protecting the privacy of sensitive data and personally identifiable information (PII).
- The key to protecting the privacy of your big data while still optimizing its value is ongoing review of four critical data management activities: data collection, retention and archiving, data use, including use in testing, DevOps, and other data masking scenarios, and creating and updating disclosure policies.
- Big data privacy also requires new rules of the data economy that are derived from the basic principle that personal data is an asset held by the people who generate it. These rules include: giving people the right to access, correct, and delete their data; requiring data collectors and processors to obtain explicit consent from data subjects; imposing limits on data collection, retention, and use; and ensuring accountability and transparency of data practices.
- Big data privacy also involves risks such as: data breaches, identity theft, cyberattacks, discrimination, surveillance, and manipulation. These risks can harm individuals, organizations, and society at large.
- Big data privacy can be enhanced by using various tools and techniques such as: using a reputable VPN, creating strong passwords, taking control of your online accounts and settings, encrypting your data, avoiding public Wi-Fi, and being aware of phishing and malware.

Big Data Ethics

Big data ethics is the study of the moral principles and values that guide the collection, analysis, and use of large amounts of data, especially personal data. Big data ethics is important because it helps to ensure that data is used in a

responsible, fair, and beneficial way, and that the rights and interests of data subjects are respected and protected. Some of the main topics and challenges in big data ethics are:

- Ownership: Who owns the data and who has the right to access, modify, or delete it? How can data subjects exercise control over their own data and benefit from its use?
- Consent: How can data subjects give informed and voluntary consent to the collection and use of their data? How can consent be obtained, recorded, and revoked? How can consent be meaningful and specific, especially when data is used for multiple or unforeseen purposes?
- Transparency: How can data collectors and users disclose their data practices and policies to data subjects and other stakeholders? How can data subjects access and understand how their data is collected, stored, processed, and shared? How can data subjects verify and challenge the accuracy and quality of their data and the decisions based on it?
- Privacy: How can data collectors and users protect the confidentiality and security of data and prevent unauthorized or harmful access, use, or disclosure? How can data collectors and users minimize the collection and retention of data and respect the data minimization principle? How can data collectors and users respect the privacy preferences and expectations of data subjects and comply with the relevant laws and regulations?
- Fairness: How can data collectors and users ensure that data is collected and used in a way that does not discriminate, exclude, or harm data subjects or groups based on their personal characteristics or circumstances? How can data collectors and users avoid or mitigate the biases and errors that may affect the data and the algorithms that process it? How can data collectors and users promote diversity and inclusion in data and its applications?

Big Data Analytics

- Big data analytics is the process of extracting useful insights from large and complex data sets that are often difficult to handle by traditional data processing methods.
- Big data analytics can help organizations to improve their performance, enhance their decision making, and discover new opportunities or patterns in their data.
- Big data analytics involves various techniques and tools, such as data mining, machine learning, statistics, visualization, natural language processing, and cloud computing.
- Big data analytics can be applied to various domains and industries, such as health care, finance, retail, education, social media, government, and security.
- Big data analytics can also pose some challenges and risks, such as data quality, privacy, security, ethics, and scalability.

Hello, I am Sydney, your AI assistant. I can help you with your study material on Big Data. Here is some content on the topic of challenges of conventional systems compared to Big Data:

Challenges of conventional systems compared to Big Data

- Conventional systems are designed to handle structured and homogeneous data, such as relational databases, spreadsheets, and text files. Big Data, on the other hand, is characterized by the 3Vs: volume, variety, and velocity. This means that Big Data is large, diverse, and fast-changing, and requires different approaches and technologies to process and analyze it.
- Conventional systems are based on the assumption that data is clean, consistent, and complete. Big Data, however, is often noisy, incomplete, and inconsistent, and may contain errors, outliers, and missing values. This poses challenges for data quality, integration, and governance, and requires more robust and flexible methods for data cleaning, validation, and transformation.
- Conventional systems are optimized for batch processing, where data is collected, stored, and processed in batches at regular intervals. Big Data, however, is often generated and consumed in real-time or near-real-time, and requires streaming processing, where data is processed as it arrives, without waiting for it to be stored. This poses challenges for data latency, scalability, and fault-tolerance, and requires more distributed and parallel architectures and frameworks, such as Hadoop, Spark, and Kafka.
- Conventional systems are limited by the capabilities and resources of a single machine or a cluster of machines, such as CPU, memory, disk, and network. Big Data, however, can exceed the capacity and performance of a single machine or a cluster, and requires cloud computing, where data and computation are distributed across multiple machines and locations, and accessed through the internet. This poses challenges for data security, privacy, and sovereignty, and requires more sophisticated and reliable methods for data encryption, authentication, and authorization.

Intelligent Data Analysis in Big Data

- Intelligent data analysis (IDA) is the process of applying artificial intelligence techniques, such as machine learning, data mining, natural language processing, and computer vision, to extract useful information and insights from large and complex data sets.
- Big data is a term that refers to data sets that are too large, diverse, dynamic, or complex to be handled by traditional data processing and analysis methods. Big data can be generated from various sources, such as social media, sensors, web logs, transactions, images, videos, and text.
- IDA in big data is a challenging and important task, as it can help discover hidden patterns, trends, anomalies, and relationships in the data, and sup-

port decision making, prediction, optimization, and innovation in various domains, such as business, health, education, security, and science.

- Some of the main steps involved in IDA in big data are:
  - Data collection: This involves acquiring, filtering, and integrating data from multiple and heterogeneous sources, and ensuring its quality, validity, and reliability.
  - Data preprocessing: This involves transforming, cleaning, normalizing, and reducing the data to make it suitable for analysis, and dealing with issues such as missing values, outliers, noise, and inconsistency.
  - Data representation: This involves selecting, extracting, and constructing relevant features, attributes, and variables from the data, and encoding them in appropriate formats, such as vectors, matrices, graphs, or tensors.
  - Data analysis: This involves applying various IDA techniques, such as classification, clustering, regression, association, dimensionality reduction, visualization, or sentiment analysis, to the data, and generating models, rules, patterns, or summaries that capture its underlying structure, behavior, or meaning.
  - Data interpretation: This involves evaluating, validating, and explaining the results of the data analysis, and deriving actionable insights, knowledge, or recommendations from them, and communicating them to the stakeholders or end-users.
- Some of the main challenges and opportunities in IDA in big data are:
  - Scalability: This refers to the ability of the IDA techniques to handle large volumes, velocities, and varieties of data, and to provide fast and accurate results, without compromising the computational resources, such as memory, storage, or processing power.
  - Diversity: This refers to the ability of the IDA techniques to deal with different types, formats, and structures of data, such as structured, unstructured, or semi-structured, numerical, categorical, or textual, and to integrate and fuse them to provide a holistic and comprehensive analysis.
  - Dynamics: This refers to the ability of the IDA techniques to cope with the changes, updates, and streams of data, and to adapt and update the analysis models, rules, or patterns accordingly, and to provide real-time or near-real-time analysis.
  - Complexity: This refers to the ability of the IDA techniques to capture and model the high-dimensional, nonlinear, and interdependent relationships, interactions, and dependencies among the data, and to provide interpretable, explainable, and trustworthy results.
  - Privacy: This refers to the ability of the IDA techniques to protect the sensitive, personal, or confidential information in the data, and to comply with the ethical, legal, and social norms and regulations

regarding the data collection, processing, and analysis.

Nature of data in Big Data

- Big data refers to the large and complex datasets that are generated from various sources and in various formats, such as text, images, videos, audio, sensor data, etc.
- Big data has some distinctive characteristics that make it different from traditional data, such as volume, velocity, variety, veracity, and value.
- Volume: Big data is characterized by the massive amount of data that is produced and stored. The volume of big data can range from terabytes to petabytes and beyond, depending on the application domain and the data source.
- Velocity: Big data is also characterized by the high speed at which data is generated and processed. The velocity of big data can vary from real-time to near-real-time to batch, depending on the data source and the analytical requirement.
- Variety: Big data is also characterized by the diversity of data types and formats that are collected and analyzed. The variety of big data can include structured, semi-structured, and unstructured data, as well as different data formats, such as text, images, videos, audio, sensor data, etc.
- Veracity: Big data is also characterized by the uncertainty and inconsistency of data quality and reliability. The veracity of big data can be affected by factors such as noise, incompleteness, ambiguity, duplication, and fraud, among others.
- Value: Big data is also characterized by the potential value that can be derived from the data analysis and utilization. The value of big data can be measured by the insights, knowledge, and decisions that can be generated from the data, as well as the economic and social benefits that can be achieved from the data.

Analytic Processes and Tools for Big Data

- Big data refers to the large and complex datasets that are generated from various sources, such as social media, sensors, web logs, etc. Big data poses challenges for traditional data analysis methods, such as scalability, heterogeneity, velocity, and veracity.
- Analytic processes for big data are the steps and techniques that are used to extract meaningful insights from big data, such as descriptive, predictive, and prescriptive analytics. Analytic processes for big data involve data collection, data preprocessing, data analysis, data visualization, and data interpretation.
- Tools for big data are the software and hardware platforms that enable and support the analytic processes for big data, such as distributed computing

frameworks, parallel databases, cloud computing, data mining, machine learning, and artificial intelligence. Tools for big data provide functionalities such as data storage, data processing, data querying, data mining, data modeling, data visualization, and data analytics.

- Some examples of analytic processes and tools for big data are:
  - Hadoop: A distributed computing framework that allows for the processing of large and diverse datasets across multiple nodes using the MapReduce programming model.
  - Spark: A distributed computing framework that provides fast and general-purpose data processing using in-memory caching and advanced analytics libraries.
  - Hive: A data warehouse system that provides a SQL-like interface for querying and analyzing structured and semi-structured data stored in Hadoop.
  - Pig: A data flow language and execution framework that allows for the manipulation and analysis of large and complex data sets using a high-level scripting language.
  - MongoDB: A document-oriented database that stores data in JSON-like format and supports dynamic schemas, horizontal scaling, and high availability.
  - Cassandra: A distributed database that provides high performance, scalability, and fault tolerance for large and dynamic data sets using a wide-column model and a peer-to-peer architecture.
  - R: A programming language and environment that provides a comprehensive set of statistical and graphical tools for data analysis and visualization.
  - Python: A general-purpose programming language that supports multiple paradigms and has a rich set of libraries and frameworks for data science, such as NumPy, pandas, scikit-learn, TensorFlow, and PyTorch.
  - Tableau: A data visualization tool that allows for the creation and sharing of interactive and dynamic dashboards and reports using drag-and-drop features and natural language queries.
  - Power BI: A business intelligence tool that enables the integration, analysis, and visualization of data from various sources using cloud-based services and interactive dashboards.

Analysis vs Reporting in Big Data

- Big data refers to the large and complex datasets that are generated from various sources and require advanced techniques and tools to process, store, and analyze.
- Analysis and reporting are two common ways of extracting value and insights from big data, but they have different purposes and characteristics.

- Analysis is the process of exploring, interpreting, and discovering patterns, trends, and relationships in big data, using various methods such as statistics, machine learning, data mining, and visualization.
- Reporting is the process of presenting and communicating the results of analysis in a clear and concise manner, using various formats such as tables, charts, dashboards, and narratives.
- Analysis and reporting are complementary and interdependent activities, but they also have some differences and challenges, such as:
  - Analysis requires more creativity, flexibility, and experimentation, while reporting requires more standardization, consistency, and accuracy.
  - Analysis involves more interaction and iteration with the data, while reporting involves more interaction and feedback with the audience.
  - Analysis can be exploratory or confirmatory, while reporting can be descriptive or prescriptive.
  - Analysis can be more complex and time-consuming, while reporting can be more simple and fast.
  - Analysis can be more subjective and uncertain, while reporting can be more objective and reliable.

Modern data analytic tools for Big Data

- Big data is a term that refers to large and complex datasets that are beyond the capabilities of traditional data processing and analysis methods.
- Big data analytics is the process of applying advanced techniques and tools to extract valuable insights from big data sources, such as social media, sensors, web logs, etc.
- Modern data analytic tools for big data can be classified into four categories: data storage and management, data processing and analysis, data visualization and exploration, and data mining and machine learning.
- Data storage and management tools are designed to handle the volume, variety, velocity, and veracity of big data. They provide scalable, distributed, and fault-tolerant systems for storing and accessing big data. Some examples are:
  - Hadoop: An open-source framework that allows for distributed processing of large datasets across clusters of computers using simple programming models.
  - Spark: An open-source unified analytics engine that provides fast and general-purpose data processing for batch, streaming, and interactive analytics.
  - MongoDB: An open-source document-oriented database that stores data in JSON-like format and supports dynamic schemas, indexing,

and aggregation.
  - Cassandra: An open-source distributed database that offers high availability, scalability, and performance for large-scale data.

- Data processing and analysis tools are designed to perform various operations on big data, such as filtering, transforming, aggregating, and querying. They provide different paradigms and languages for expressing and executing data analysis tasks. Some examples are:
  - MapReduce: A programming model and an associated implementation for processing and generating large datasets in parallel and distributed manner.
  - SQL: A standard query language for relational databases that can be used to manipulate and analyze structured data.
  - Hive: An open-source data warehouse system that provides a SQL-like interface for querying and analyzing data stored in Hadoop.
  - Pig: An open-source platform that provides a high-level language for expressing data analysis programs that are executed on Hadoop.

- Data visualization and exploration tools are designed to present and interact with big data in a graphical and intuitive way. They provide various techniques and features for creating and customizing charts, graphs, maps, dashboards, and reports. Some examples are:
  - Tableau: A commercial software that allows users to create and share interactive data visualizations and dashboards.
  - D3.js: An open-source JavaScript library that uses web standards to create dynamic and interactive data visualizations in the browser.
  - Power BI: A cloud-based business intelligence service that provides data visualization and analysis capabilities for various data sources.
  - Matplotlib: An open-source Python library that provides a comprehensive set of tools for creating static and interactive plots and charts.

- Data mining and machine learning tools are designed to discover patterns, trends, and relationships in big data. They provide various algorithms and methods for performing tasks such as classification, clustering, regression, recommendation, and anomaly detection. Some examples are:
  - Scikit-learn: An open-source Python library that provides a collection of tools for data mining and machine learning, such as preprocessing, feature extraction, model selection, and evaluation.
  - TensorFlow: An open-source platform that provides a comprehensive and flexible framework for building and deploying machine learning models and applications.
  - Weka: An open-source software that provides a suite of machine learning algorithms for data mining tasks, such as data preprocessing, classification, regression, clustering, and association rules.
  - Apache Mahout: An open-source project that provides scalable and distributed implementations of machine learning algorithms for big

data analytics.

Unit 2 - Hadoop and Map Reduce

- Hadoop is an open-source framework that allows distributed processing of large-scale data sets across clusters of computers using simple programming models.
- Hadoop consists of two main components: Hadoop Distributed File System (HDFS) and Hadoop MapReduce.
- HDFS is a distributed file system that provides high-throughput access to data stored on multiple nodes in a cluster. HDFS replicates data blocks across different nodes to ensure fault tolerance and reliability.
- MapReduce is a programming model that enables parallel processing of large data sets by dividing them into smaller chunks called splits. Each split is processed by a mapper function that transforms the input data into key-value pairs. The output of the mappers is then shuffled and sorted by keys and sent to the reducers. The reducer function aggregates the values associated with each key and produces the final output.
- Hadoop MapReduce follows a master-slave architecture, where a single node called the JobTracker acts as the master and coordinates the execution of multiple tasks across different nodes called TaskTrackers. The JobTracker assigns tasks to the TaskTrackers based on their availability and data locality. The TaskTrackers report their progress and status to the JobTracker periodically.
- Hadoop MapReduce supports various types of input and output formats, such as text, binary, sequence, etc. It also provides a set of built-in mapper and reducer classes, such as IdentityMapper, LongSumReducer, etc. that can be used for common operations. Users can also write their own custom mapper and reducer classes using Java or other languages supported by Hadoop.
- Hadoop MapReduce also provides a higher-level abstraction called Apache Pig, which is a scripting language that allows users to write complex data analysis programs using a SQL-like syntax. Pig translates the scripts into MapReduce jobs and executes them on Hadoop. Pig also provides a set of built-in functions and operators, such as FILTER, GROUP, JOIN, etc. that can be used for data manipulation and transformation.

Hadoop

- Hadoop is an open-source software framework for storing data and running applications on clusters of commodity hardware.
- Hadoop provides massive storage for any kind of data, enormous processing power and the ability to handle virtually limitless concurrent tasks or jobs.
- Hadoop is designed to scale up from a single computer to thousands of clustered computers, with each machine offering local computation and stor-

age. In this way, Hadoop can efficiently store and process large datasets ranging in size from gigabytes to petabytes of data.

- Hadoop is a collection of open-source software utilities that facilitates using a network of many computers to solve problems involving massive amounts of data and computation.
- Hadoop provides a software framework for distributed storage and processing of big data using the MapReduce programming model. MapReduce is a programming paradigm that allows for parallel processing of large data sets across multiple nodes in a cluster.
- Hadoop consists of four main components: Hadoop Common, Hadoop Distributed File System (HDFS), Hadoop MapReduce and Hadoop YARN.
  - Hadoop Common is a set of common utilities and libraries that support other Hadoop modules.
  - HDFS is a distributed file system that provides high-throughput access to application data and can store large data sets across multiple machines.
  - Hadoop MapReduce is a software framework for writing applications that process large amounts of structured and unstructured data in parallel across a cluster of nodes.
  - Hadoop YARN is a resource management platform that manages computing resources in clusters and schedules applications to run on them.
- Hadoop is used for various big data applications running under clustered systems, such as data analysis, data mining, data warehousing, machine learning, natural language processing, image processing, etc.

History of Hadoop

- Hadoop is an open-source software framework for storing and processing big data in a distributed manner. It consists of several components, such as Hadoop Distributed File System (HDFS), Hadoop MapReduce, Hadoop YARN, and Hadoop Ozone.
- Hadoop was started by Doug Cutting and Mike Cafarella in 2002, when they began working on Apache Nutch, a project to build a search engine that can index 1 billion pages .
- Hadoop was inspired by two papers published by Google in 2003 and 2004, describing the Google File System (GFS) and the MapReduce programming model, respectively .
- Hadoop was named after a toy elephant that belonged to Doug Cutting's son. The logo of Hadoop is a yellow elephant with the word "Hadoop" written on it.
- Hadoop was officially released as an Apache project in 2006. In 2008, Hadoop defeated the supercomputers and became the fastest system on the planet for sorting terabytes of data.
- Hadoop has evolved over the years, adding new features and components, such as Hadoop YARN in 2012, which is a platform for managing com-

puting resources and scheduling user applications, and Hadoop Ozone in 2020, which is an object store for Hadoop.
- Hadoop has also spawned a rich ecosystem of tools and applications that run on top of it, such as Apache Hive, Apache Pig, Apache Spark, Apache HBase, Apache Kafka, Apache Flume, Apache Sqoop, and many more .
- Hadoop is widely used by many organizations and industries for various purposes, such as data analysis, data mining, data warehousing, machine learning, artificial intelligence, and more .

Apache Hadoop

- Apache Hadoop is a collection of open-source software utilities that facilitates using a network of many computers to solve problems involving massive amounts of data and computation .
- Apache Hadoop software library is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models .
- Apache Hadoop is designed to scale up from single servers to thousands of machines, each offering local computation and storage.
- Apache Hadoop consists of four main components: Hadoop Common, Hadoop Distributed File System (HDFS), Hadoop MapReduce, and Hadoop YARN.
- Hadoop Common contains the common utilities and libraries that support other Hadoop modules.
- HDFS is a distributed file system that provides high-throughput access to application data and can store large amounts of data across multiple machines.
- Hadoop MapReduce is a programming model and software framework for writing applications that process large amounts of data in parallel on clusters of nodes.
- Hadoop YARN is a resource management platform that manages the computing resources and schedules the execution of applications on the Hadoop cluster.

Hadoop Distributed File System

- Hadoop Distributed File System (HDFS) is a distributed file system that provides high-throughput access to large data sets across scalable clusters of nodes.
- HDFS is the primary data storage system used by Hadoop applications and is one of the core components of the Apache Hadoop ecosystem, along with MapReduce and YARN.
- HDFS employs a master-slave architecture, where one node acts as the NameNode and manages the file system namespace and metadata, while the other nodes act as DataNodes and store the actual data in blocks.
- HDFS splits files into large blocks (typically 64 MB or 128 MB) and dis-

tributes them across the DataNodes in the cluster. The NameNode maintains the mapping of blocks to DataNodes and also replicates the blocks for fault tolerance and load balancing.
- HDFS supports a write-once-read-many model, where files are written by a single client and then read by multiple clients. HDFS also supports appending data to existing files, but not random writes or updates.
- HDFS is designed to run on commodity hardware and handle hardware failures gracefully. It can scale to thousands of nodes and store petabytes of data. HDFS also provides interfaces for applications to access the data, such as Java API, WebHDFS, NFS and Fuse.

Components of Hadoop

Hadoop is a framework for distributed storage and processing of large-scale data sets. It consists of the following core components:

- **Hadoop Distributed File System (HDFS)**: This is the storage layer of Hadoop that stores data in a distributed manner across multiple nodes in a cluster. HDFS provides high availability, fault tolerance, scalability, and data locality. HDFS can store any kind of data without prior organization .
- **Hadoop MapReduce**: This is the processing layer of Hadoop that allows parallel execution of user-defined functions on the data stored in HDFS. MapReduce consists of two phases: map and reduce. The map phase applies a function to each input key-value pair and produces intermediate key-value pairs. The reduce phase aggregates the intermediate values associated with the same key and produces the final output .
- **Hadoop YARN**: This is the resource management layer of Hadoop that allocates and schedules resources (such as CPU, memory, disk, and network) for the applications running on the cluster. YARN also monitors the health and performance of the nodes and the applications. YARN enables multiple processing frameworks (such as Spark, Hive, Pig, etc.) to run on the same cluster and share resources .

Hadoop also has some other components that provide additional functionality, such as:

- **Hadoop Common**: This is a set of shared libraries and utilities that support the other Hadoop components. It includes configuration, logging, security, serialization, and networking modules.
- **Hadoop ZooKeeper**: This is a service that provides coordination and synchronization for distributed applications. It maintains configuration information, naming, group membership, and leader election for the applications.
- **Hadoop Oozie**: This is a workflow scheduler that manages and executes Hadoop jobs. It supports dependency management, concurrency control, retry policies, and notifications for the jobs.

- **Hadoop HBase**: This is a column-oriented database that provides random access and real-time updates for large-scale data. It is built on top of HDFS and supports MapReduce operations.
- **Hadoop Hive**: This is a data warehouse that provides a SQL-like interface for querying and analyzing data stored in HDFS. It supports various data formats, such as text, JSON, ORC, Parquet, etc. It also supports user-defined functions and custom data types.
- **Hadoop Pig**: This is a scripting language that allows users to write complex data transformations and analysis using a high-level syntax. It compiles the scripts into MapReduce jobs and executes them on the cluster.
- **Hadoop Spark**: This is a fast and general-purpose processing framework that supports batch, streaming, interactive, and machine learning applications. It uses in-memory caching and lazy evaluation to optimize performance. It also provides various libraries, such as Spark SQL, Spark Streaming, MLlib, and GraphX.

Data Format Co

- Data format co is a company that provides data conversion and formatting services for various types of documents and files.
- Data format co can handle different formats such as PDF, Word, Excel, PowerPoint, HTML, XML, JSON, CSV, etc.
- Data format co can also perform data extraction, data validation, data cleansing, data transformation, data analysis, and data visualization tasks.
- Data format co uses advanced tools and techniques such as optical character recognition (OCR), natural language processing (NLP), machine learning (ML), and artificial intelligence (AI) to ensure high quality and accuracy of the data.
- Data format co offers customized solutions for different industries and domains such as education, healthcare, finance, legal, media, etc.
- Data format co has a team of experienced and skilled professionals who can handle complex and large-scale data projects with efficiency and speed.
- Data format co has a global presence and a diverse clientele base that includes individuals, businesses, organizations, and governments.

Analyzing data with Hadoop

Hadoop is a framework for distributed processing of large-scale data sets across clusters of computers. It consists of two main components: Hadoop Distributed File System (HDFS) and MapReduce.

- HDFS is a distributed file system that stores data in blocks across multiple nodes in a cluster. It provides high availability, fault tolerance, and scalability.
- MapReduce is a programming model that allows parallel processing of data using two functions: map and reduce. Map function takes a set of

input data and transforms it into intermediate key-value pairs. Reduce function takes the intermediate key-value pairs and aggregates them to produce the final output.
- Hadoop also provides other components and tools for data analysis, such as Hive, Pig, Spark, HBase, etc.
- To analyze data with Hadoop, one needs to perform the following steps:
  - Load the data into HDFS using tools like Flume, Sqoop, or Hadoop commands.
  - Write a MapReduce program or use other frameworks like Hive or Pig to process the data.
  - Run the program on the Hadoop cluster using tools like YARN, Oozie, or Hadoop commands.
  - Retrieve the output from HDFS or store it in other systems like HBase or Hive.
  - Analyze the output using tools like Hue, Zeppelin, or Excel.

Scaling out with Hadoop

- Hadoop is a framework for distributed processing of large-scale data sets across clusters of computers.
- Hadoop enables scaling out by adding more nodes to the cluster, rather than scaling up by upgrading the hardware of a single node.
- Hadoop consists of two main components: the Hadoop Distributed File System (HDFS) and the MapReduce programming model.
- HDFS is a distributed file system that stores data in blocks across multiple nodes, providing fault tolerance, high availability, and scalability.
- MapReduce is a programming model that allows parallel processing of data on the cluster, using two types of functions: map and reduce.
- Map functions take input data and transform it into key-value pairs, which are then shuffled and sorted by the framework.
- Reduce functions take the key-value pairs and aggregate them to produce the final output.
- Hadoop also provides other components and tools, such as YARN, Hive, Pig, Spark, HBase, and ZooKeeper, to support different types of data processing and analysis.

Hadoop streaming

- Hadoop streaming is a utility that comes with the Hadoop distribution.
- The utility allows you to create and run MapReduce jobs with any executable or script as the mapper and/or the reducer.
- The mapper and reducer scripts can be written in any language that can read from standard input and write to standard output, such as Python, Ruby, Perl, etc.
- Hadoop streaming works by passing the input data to the mapper script as lines of text via standard input, and collecting the output data from

the mapper script as lines of text via standard output.
- The output data from the mapper script is then shuffled and sorted by Hadoop, and passed to the reducer script as lines of text via standard input, grouped by key.
- The output data from the reducer script is then collected by Hadoop as the final output of the MapReduce job.
- Hadoop streaming can be invoked by using the `hadoop jar` command with the `hadoop-streaming.jar` file as the argument, followed by various options to specify the input, output, mapper, reducer, and other parameters of the job.
- For example, the following command runs a MapReduce job with a Python script as the mapper and a shell command as the reducer:

```
hadoop jar hadoop-streaming.jar \
-input myInputDirs \
-output myOutputDir \
-mapper mapper.py \
-reducer /bin/wc
```

- Hadoop streaming is a powerful feature that enables users to leverage the scalability and fault-tolerance of Hadoop with any programming language of their choice.

Hadoop Pipes

- Hadoop Pipes is a C++ API for writing MapReduce applications that run on Hadoop clusters.
- Hadoop Pipes allows developers to use C++ instead of Java for implementing the map and reduce functions, which can improve the performance and efficiency of some applications.
- Hadoop Pipes works by launching a Java process that communicates with the Hadoop framework and a C++ process that executes the user-defined map and reduce functions.
- The communication between the Java and C++ processes is done through a binary protocol based on Google's Protocol Buffers.
- Hadoop Pipes requires the following components:
  - A C++ compiler and linker that support the C++11 standard.
  - The Hadoop native libraries and headers, which can be built from the Hadoop source code or downloaded from the Hadoop website.
  - The Hadoop Pipes library and headers, which are included in the Hadoop source code under the `hadoop-mapreduce-project/hadoop-mapreduce-client/hadoop-ma` directory.
  - The Protocol Buffers library and headers, which can be obtained from the Protocol Buffers website or installed through a package manager.
- To write a Hadoop Pipes application, the developer needs to:
  - Include the `hadoop/Pipes.hh` header file in the C++ source code.
  - Implement the `hadoop::Mapper` and `hadoop::Reducer` classes,

which define the map and reduce functions respectively.
  - Implement the `hadoop::Factory` class, which creates instances of the mapper and reducer classes.
  - Compile and link the C++ source code with the Hadoop Pipes library and the Protocol Buffers library, producing an executable file.
  - Run the Hadoop Pipes application using the `hadoop pipes` command, specifying the input and output paths, the executable file, and any other options or parameters.

Hadoop Ecosystem

- The Hadoop Ecosystem is a collection of tools, libraries, and frameworks that help you build applications on top of Apache Hadoop.
- Apache Hadoop is a software library that allows for the distributed processing of large data sets across clusters of computers using simple programming models.
- Hadoop is designed to scale up from a single computer to thousands of clustered computers, with each machine offering local computation and storage .
- Hadoop enables multiple types of analytic workloads to run on the same data, at the same time, at massive scale on industry-standard hardware.
- The Hadoop Ecosystem consists of the following components:
  - HDFS: Hadoop Distributed File System, a distributed and fault-tolerant file system that stores data across multiple nodes.
  - YARN: Yet Another Resource Negotiator, a resource management layer that allocates and schedules tasks on the cluster.
  - MapReduce: A programming model and framework for parallel data processing using key-value pairs.
  - Spark: An in-memory data processing engine that supports batch, streaming, SQL, machine learning, and graph analytics.
  - PIG, HIVE: Query based processing of data services that provide a high-level abstraction over MapReduce.
  - HBase: A NoSQL database that provides random access and consistent writes for large-scale structured and semi-structured data.
  - Sqoop, Flume, Kafka: Data ingestion tools that transfer data from various sources to HDFS or HBase.
  - Oozie, Zookeeper: Coordination and workflow management tools that orchestrate and monitor the execution of Hadoop jobs.
  - Mahout, MLlib: Machine learning libraries that provide scalable and distributed algorithms for data mining and analysis.
  - Ambari, Hue: Web-based user interfaces that provide administration and visualization of the Hadoop cluster and its components.

Map Reduce

Map Reduce is a programming paradigm that allows processing large volumes of data in parallel on a cluster of machines. It consists of two phases: map and reduce. In the map phase, the input data is split into key-value pairs and distributed to the mapper tasks. The mapper tasks apply a user-defined function to each key-value pair and produce intermediate key-value pairs. In the reduce phase, the intermediate key-value pairs are shuffled and sorted by key and sent to the reducer tasks. The reducer tasks apply another user-defined function to the values associated with each key and produce the final output.

Some of the features and benefits of Map Reduce are:

- It is scalable and can handle petabytes of data on thousands of nodes.
- It is fault-tolerant and can recover from failures of machines, tasks, or network.
- It is simple and abstracts the complexity of distributed computing from the user.
- It is flexible and can process structured, semi-structured, or unstructured data.
- It is compatible with various data sources and formats, such as HDFS, HBase, JSON, XML, etc.

Some of the steps involved in writing and running a Map Reduce program are:

- Define the mapper and reducer functions in a programming language of choice, such as Java, Python, or C++.
- Compile and package the code into a JAR file or an executable file.
- Specify the input and output paths, the number of mappers and reducers, and other configuration parameters in a driver class or a script.
- Submit the job to the Hadoop cluster using the command-line interface or a web interface.
- Monitor the progress and status of the job using the JobTracker or the Resource Manager.
- Retrieve the output from the output path or the HDFS.

Map Reduce framework and basics

- Map Reduce is a programming model and an associated implementation for processing and generating large data sets with a parallel, distributed algorithm on a cluster.
- The model is inspired by the map and reduce functions commonly used in functional programming, although their purpose in the Map Reduce framework is not the same as in their original forms.
- The key idea is to split the input data set into independent chunks that are processed by the map tasks in a completely parallel manner. The framework sorts the outputs of the maps, which are then input to the reduce tasks. Typically both the input and the output of the job are

stored in a file system. The framework takes care of scheduling tasks, monitoring them and re-executes the failed tasks.

- The Map Reduce framework consists of a single master JobTracker and one slave TaskTracker per cluster-node. The master is responsible for scheduling the jobs' component tasks on the slaves, monitoring them and re-executing the failed tasks. The slaves execute the tasks as directed by the master.
- The Map Reduce framework operates exclusively on <key, value> pairs, that is, the framework views the input to the job as a set of <key, value> pairs and produces a set of <key, value> pairs as the output of the job, conceivably of different types.
- The Map Reduce framework consists of two phases: map and reduce. The map phase takes an input pair and produces a set of intermediate key/value pairs. The Map Reduce framework groups together all intermediate values associated with the same intermediate key I and passes them to the reduce function. The reduce function accepts an intermediate key I and a set of values for that key. It merges together these values to form a possibly smaller set of values. Typically just zero or one output value is produced per reduce invocation. The intermediate values are supplied to the user's reduce function via an iterator. This allows us to handle lists of values that are too large to fit in memory.
- The number of map tasks and reduce tasks for each job is usually driven by the total size of the inputs, that is, the total number of blocks of the input files. The right level of parallelism for maps seems to be around 10-100 maps per-node, although it has been set up to 300 maps for very cpu-light map tasks. Task setup takes awhile, so it is best if the maps take at least a minute to execute. Thus, for cpu-heavy jobs, one may want fewer maps, in order to limit the cost of setting up the tasks. For many jobs, a good rule of thumb is to make the number of maps be around 10x the number of reduce tasks.
- The number of reduces for the job is a small multiple of the number of nodes in the cluster for most jobs. With 0.95 being occupied by map tasks, 0.05 goes to reduce tasks, so the number of reduces is about 5% of the number of nodes. Speculative execution: Since the reduce phase can only start once the map phase is completed, the job execution can be delayed because of a few slow nodes. To avoid this, the JobTracker can speculatively execute backup copies of the remaining map tasks on other nodes. This process is called speculative execution. The JobTracker does not schedule redundant copies of a map task as long as the task is making progress. However, it does not know whether a particular task is progressing slowly or is stuck. To handle this, the JobTracker maintains a sorted list of the rate of progress on map tasks. When the JobTracker notices that the rate of progress of a map task is below a certain threshold, it schedules backup executions of these tasks on other nodes. The output of the task that finishes first is used and the other task is killed. Similarly, a small number of

backup copies of reduce tasks are also executed. Speculative execution is enabled by default. The number of speculative tasks at any time is at most 10% of the number of nodes in the cluster. This bound is configurable via the mapred.speculative.execution.speculativeCap property. The speculative execution threshold for maps is controlled by the mapred.map.tasks.speculative.execution property and for reduces by the mapred.reduce.tasks.speculative.execution property. The default value is 0.05. This means that if the progress rate of a task is below the average progress rate of all the tasks in the same job by 5%, then the task is marked as under progress and is a candidate for speculative execution. The speculative execution of maps and reduces can be turned off by setting the mapred.map.tasks.speculative.execution and mapred.reduce.tasks.speculative.execution properties to false respectively.

How Map Reduce works

- Map Reduce is a programming model and an associated implementation for processing and generating large data sets with a parallel, distributed algorithm on a cluster.
- The model is inspired by the map and reduce functions commonly used in functional programming, although their purpose in the Map Reduce framework is not the same as their original forms.
- The key idea is to split the input data set into independent chunks that are processed by the map tasks in a completely parallel manner. The framework sorts the outputs of the maps, which are then input to the reduce tasks. Typically both the input and the output of the job are stored in a file system. The framework takes care of scheduling tasks, monitoring them and re-executing the failed tasks.
- The Map Reduce framework consists of a single master JobTracker and one slave TaskTracker per cluster-node. The master is responsible for scheduling the jobs' component tasks on the slaves, monitoring them and re-executing the failed tasks. The slaves execute the tasks as directed by the master.
- The Map Reduce framework operates exclusively on <key, value> pairs, that is, the framework views the input to the job as a set of <key, value> pairs and produces a set of <key, value> pairs as the output of the job, conceivably of different types.
- The Map Reduce framework consists of two phases: map and reduce. The map phase takes an input pair and produces a set of intermediate key/value pairs. The Map Reduce framework groups together all intermediate values associated with the same intermediate key I and passes them to the reduce function. The reduce function accepts an intermediate key I and a set of values for that key. It merges together these values to form a possibly smaller set of values. Typically just zero or one output value is produced per reduce invocation. The intermediate values are supplied to the user's reduce function via an iterator. This allows us to handle lists

of values that are too large to fit in memory.

- The number of map tasks and reduce tasks for each job is usually driven by the total size of the inputs, that is, the total number of blocks of the input files. The right level of parallelism for maps seems to be around 10-100 maps per-node, although it has been set up to 300 maps for very cpu-light map tasks. Task setup takes awhile, so it is best if the maps take at least a minute to execute. Thus, for cpu-intensive jobs, one may want fewer maps, in order to limit the cost of sorting the intermediate key/value pairs. On the other hand, a large number of maps increases load balancing, as well as the effectiveness of fault tolerance. The number of reduces seems to be much less sensitive to the job. A rule of thumb is that the number of reduces is 0.95 or 1.75 multiplied by (<no. of nodes> * <no. of maximum containers per node>). With 0.95 all of the reduces can launch immediately and start transferring map outputs as the maps finish. With 1.75 the faster nodes will finish their first round of reduces and launch a second wave of reduces doing a much better job of load balancing. Increasing the number of reduces increases the framework overhead, but increases load balancing and lowers the cost of failures. The scaling factors above are slightly less than whole numbers to reserve a few reduce slots in the framework for speculative-tasks and failed tasks.

Developing a Map Reduce application

- Map Reduce is a programming model for processing large-scale data sets in parallel and distributed environments.
- A Map Reduce application consists of two main functions: a map function and a reduce function.
- The map function takes an input key-value pair and produces a set of intermediate key-value pairs. The intermediate keys are grouped by a partitioner and sent to different reducers.
- The reduce function takes an intermediate key and a list of values associated with that key, and produces a set of output key-value pairs. The output keys are sorted by a comparator and written to the final output file.
- A Map Reduce application also requires a driver class that specifies the input and output formats, the mapper and reducer classes, and other configuration parameters.
- To develop a Map Reduce application, one needs to follow these steps:
  - Write the map and reduce functions in Java, using the org.apache.hadoop.mapreduce.Mapper and org.apache.hadoop.mapreduce.Reducer interfaces.
  - Write the driver class in Java, using the org.apache.hadoop.mapreduce.Job class to configure and run the Map Reduce job.

  - Compile the Java classes into a JAR file, using the Hadoop command-line tool or an IDE.
  - Upload the JAR file and the input data to the Hadoop cluster, using the Hadoop Distributed File System (HDFS) commands or a web interface.
  - Run the Map Reduce job on the Hadoop cluster, using the Hadoop command-line tool or a web interface.
  - Monitor the progress and status of the Map Reduce job, using the Hadoop command-line tool or a web interface.
  - Download the output data from the Hadoop cluster, using the HDFS commands or a web interface.
  - Analyze the output data, using the Hadoop command-line tool or other tools.

Unit tests with MRUnit

- Unit tests are a way of verifying the correctness and functionality of individual components of a software system, such as classes, methods, or modules.
- MRUnit is a Java library that helps developers write and run unit tests for Apache Hadoop MapReduce jobs.
- MRUnit provides a set of classes and methods that simulate the behavior and environment of a MapReduce cluster, without requiring an actual cluster or data.
- MRUnit allows developers to test the logic and output of their mappers, reducers, combiners, and partitioners, as well as the interactions between them.
- MRUnit also supports testing custom counters, configuration properties, and input/output formats.
- To use MRUnit, developers need to add the MRUnit dependency to their project's build file, such as Maven or Gradle, and import the relevant classes in their test classes.
- A typical MRUnit test case consists of the following steps:
  - Create an instance of the class under test, such as a mapper or a reducer.
  - Create an instance of the MRUnit driver, such as MapDriver or ReduceDriver, and pass the class under test to its constructor.
  - Set up the input and expected output values for the test case, using the methods of the driver, such as withInput and withOutput.
  - Optionally, set up any configuration properties, counters, or input/output formats, using the methods of the driver, such as withConfiguration and withCounter.
  - Run the test case, using the runTest or run methods of the driver, and assert the results, using the methods of the driver, such as assertOutput and assertCounter.
  - Repeat the above steps for each test case or scenario.

- MRUnit provides several advantages for testing MapReduce jobs, such as:
  - It simplifies and speeds up the development and debugging process, by allowing developers to test their code locally and quickly, without requiring a cluster or data.
  - It improves the quality and reliability of the code, by enabling developers to catch and fix errors and bugs early, before deploying the code to a cluster.
  - It increases the coverage and completeness of the tests, by allowing developers to test various scenarios and edge cases, as well as the interactions between different components of the job.
  - It facilitates the maintenance and refactoring of the code, by providing a clear and consistent way of verifying the behavior and output of the code, after making any changes or modifications.

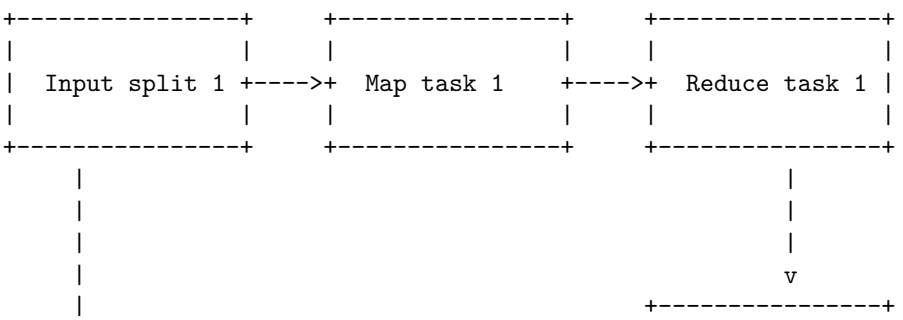
Test data and local tests in map reduce

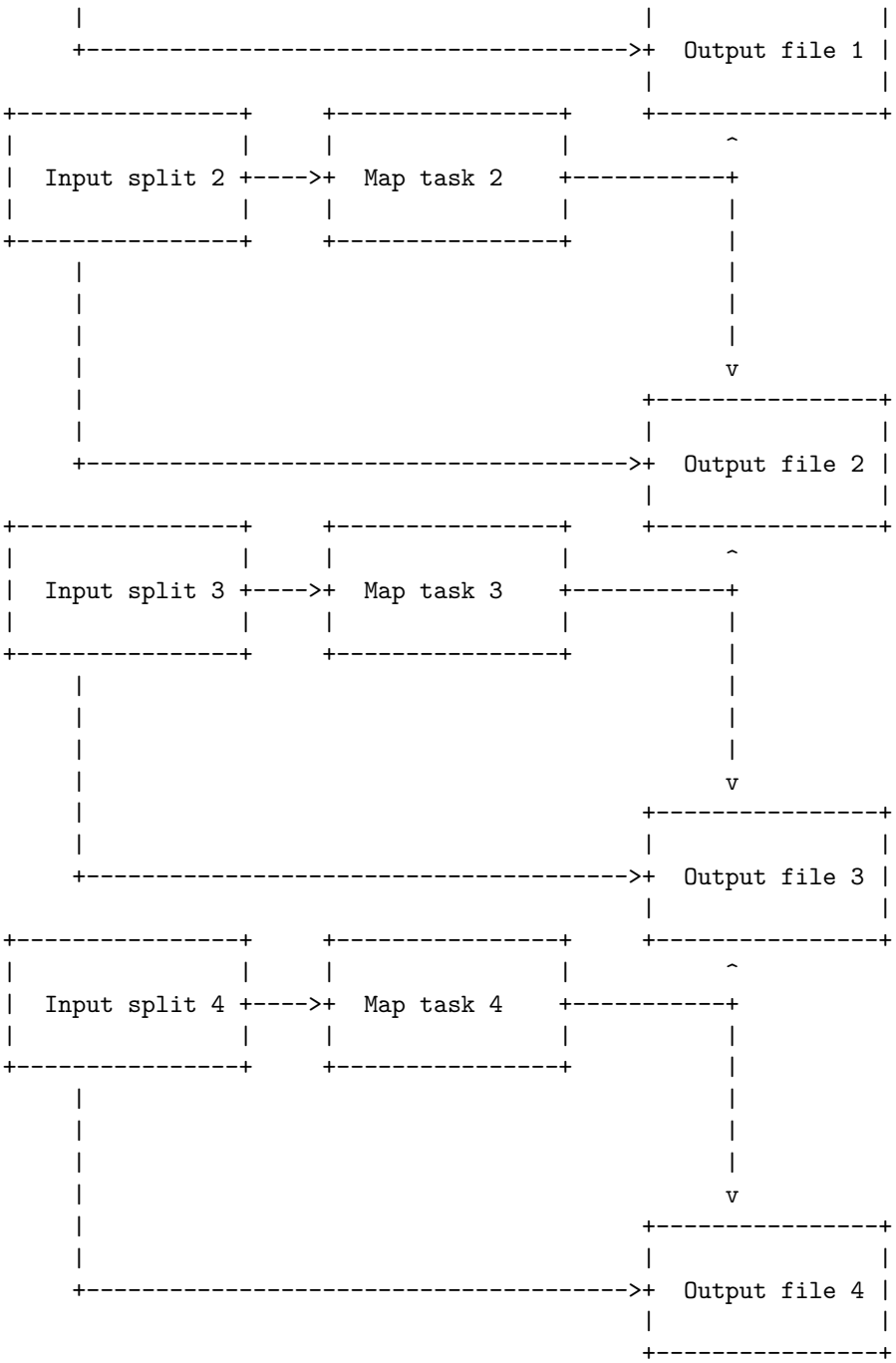
- Map reduce is a programming model for processing large-scale data sets in parallel and distributed manner.
- Testing map reduce applications can be challenging due to the complexity and scale of the data and the distributed environment.
- One way to simplify testing is to use test data and local tests that can run on a single machine without requiring a hadoop cluster.
- Test data can be generated or sampled from real data sets, depending on the requirements and objectives of the test cases.
- Local tests can use tools and frameworks such as hadoop streaming, MRUnit, and JUnit to test the functionality and performance of the map and reduce functions in isolation or in combination.
- Some examples of local tests are:
  - Using hadoop streaming to run the map and reduce scripts on a local file system and pipe the output to standard output or a file.
  - Using MRUnit to create mock input and output objects and assert the expected results of the map and reduce functions.
  - Using JUnit to write unit tests for the map and reduce classes and methods, and use mockito or powermock to mock the context and configuration objects.
- Local tests can help to catch many bugs and errors in the map reduce code before deploying it to a hadoop cluster, and can also speed up the development and debugging process.
- However, local tests cannot fully simulate the distributed and parallel environment of a hadoop cluster, and may not cover all the possible scenarios and edge cases that may occur in production.

- Therefore, local tests should be complemented by integration and system tests that run on a hadoop cluster with real or simulated data sets.

Anatomy of a Map Reduce job run

- A Map Reduce job is a unit of work that consists of a map function and a reduce function, applied to a set of input data.
- The map function takes a key-value pair as input and produces a list of intermediate key-value pairs as output. The intermediate keys and values can be of different types than the input keys and values.
- The reduce function takes an intermediate key and a list of values associated with that key as input and produces a list of final key-value pairs as output. The final keys and values can be of different types than the intermediate keys and values.
- A Map Reduce job run consists of the following phases:
  - Input: The input data is split into fixed-size chunks called input splits. Each input split is assigned to a map task, which runs the map function on the key-value pairs in the split.
  - Map: The map tasks process the input splits in parallel and emit intermediate key-value pairs to a distributed file system called HDFS (Hadoop Distributed File System). The intermediate key-value pairs are partitioned by a partitioner function based on the intermediate keys and sent to different reduce tasks.
  - Shuffle: The reduce tasks fetch the intermediate key-value pairs from HDFS and sort them by the intermediate keys. This phase is called shuffle because it involves transferring data across the network.
  - Reduce: The reduce tasks run the reduce function on the sorted intermediate key-value pairs and produce the final output key-value pairs. The output key-value pairs are written to HDFS or another output destination.
  - Output: The output data is the result of the Map Reduce job run. It can be consumed by other applications or stored for further analysis.
- The following diagram illustrates the anatomy of a Map Reduce job run:





Failures in MapReduce

- MapReduce is a programming model and framework for processing large-scale data sets in parallel and distributed environments.
- MapReduce consists of two phases: map and reduce, where the map phase applies a user-defined function to each input record and emits intermediate key-value pairs, and the reduce phase aggregates the intermediate values associated with the same key and produces the final output.
- MapReduce relies on a master-slave architecture, where a master node coordinates the execution of multiple worker nodes that perform the actual map and reduce tasks.
- Failures are inevitable in MapReduce, due to the large number of nodes, the long-running jobs, and the heterogeneous and unreliable hardware and network conditions.
- MapReduce handles failures by using fault tolerance mechanisms, such as replication, checkpointing, and re-execution.
- Replication: MapReduce replicates the input data across multiple nodes to ensure data availability and locality. The master node also maintains backup workers that can take over the tasks of failed workers.
- Checkpointing: MapReduce periodically checkpoints the intermediate data and the state of the workers to a distributed file system, such as HDFS. This allows the system to recover from failures without losing the progress of the job.
- Re-execution: MapReduce detects failures by using heartbeats and timeouts. If a worker fails to send a heartbeat to the master within a certain interval, the master assumes that the worker has failed and re-assigns its tasks to another worker. If a task takes longer than a specified threshold, the master may launch a speculative copy of the task on another worker to speed up the job completion.
- MapReduce can tolerate different types of failures, such as node failures, task failures, and network failures, by using the above mechanisms. However, some failures may still cause the job to fail, such as master node failure, data corruption, or malicious attacks. Therefore, MapReduce also requires backup and recovery strategies, such as storing the output data in a reliable storage system, verifying the integrity of the data, and securing the communication channels.

Job scheduling in MapReduce

- Job scheduling is the process of assigning tasks to workers in a distributed system, such as MapReduce, to optimize the performance and efficiency of the system.
- MapReduce is a programming model and framework for processing large-scale data sets in parallel using multiple machines, or nodes, in a cluster.
- A MapReduce job consists of two phases: map and reduce. The map phase applies a user-defined function to each input key-value pair and produces a set of intermediate key-value pairs. The reduce phase applies another

user-defined function to all the intermediate values associated with the same intermediate key and produces a set of output key-value pairs.

- Job scheduling in MapReduce involves deciding how to partition the input data, how to assign map and reduce tasks to nodes, how to balance the workload among nodes, and how to handle failures and stragglers.
- Some of the challenges and goals of job scheduling in MapReduce are:
  - Minimizing the total execution time of a job, or the makespan, by exploiting the parallelism and locality of the data and tasks.
  - Maximizing the cluster utilization and throughput by efficiently allocating the available resources, such as CPU, memory, disk, and network bandwidth, to different jobs and tasks.
  - Achieving fairness and quality of service for multiple concurrent jobs and users by ensuring that each job receives a fair share of the resources and meets its deadlines and priorities.
  - Adapting to the dynamic and heterogeneous nature of the cluster and the workload by adjusting the scheduling decisions based on the current status and performance of the nodes and tasks.
  - Handling failures and stragglers gracefully by detecting and recovering from node or task failures and reassigning or replicating the tasks that are slow or stuck.
- Some of the common techniques and algorithms for job scheduling in MapReduce are:
  - FIFO: A simple and intuitive scheduling policy that assigns tasks to nodes in the order of their arrival. It is easy to implement and guarantees fairness, but it may not utilize the cluster resources efficiently or exploit the data locality.
  - Fair Scheduler: A scheduling policy that aims to allocate resources fairly among multiple concurrent jobs and users. It divides the cluster resources into pools, each with a certain weight and minimum share, and assigns tasks to nodes based on the demand and availability of the resources in each pool. It also supports preemption, delay scheduling, and locality waits to improve the data locality and performance of the tasks.
  - Capacity Scheduler: A scheduling policy that aims to maximize the cluster utilization and throughput by allocating resources to different queues, each with a certain capacity and priority. It assigns tasks to nodes based on the demand and availability of the resources in each queue. It also supports preemption, delay scheduling, and locality waits to improve the data locality and performance of the tasks.
  - LATE: A scheduling algorithm that aims to handle stragglers effectively by estimating the progress and remaining time of each task and reassigning or replicating the tasks that are likely to finish late. It uses a speculative execution technique that runs multiple copies of

the same task on different nodes and picks the fastest one.
  - Quincy: A scheduling algorithm that aims to balance the trade-off between data locality and fairness by modeling the scheduling problem as a min-cost flow problem and solving it using a distributed auction algorithm. It assigns tasks to nodes based on the bids and prices of the resources and the data transfers. It also supports preemption and replication to improve the performance and reliability of the tasks.

Shuffle and sort in map reduce

- Map reduce is a programming model for processing large-scale data sets in parallel and distributed environments.
- Map reduce consists of two phases: map and reduce. In the map phase, each input record is transformed into a key-value pair by a user-defined function. In the reduce phase, all the values associated with the same key are combined by another user-defined function.
- Shuffle and sort is an intermediate step between the map and reduce phases. It is responsible for transferring the output of the map tasks to the reduce tasks, and sorting them by key.
- Shuffle and sort has the following steps:
  - Partitioning: The output of each map task is partitioned into R regions, where R is the number of reduce tasks. The partitioning function is usually a hash function of the key, but it can be customized by the user.
  - Sorting: Within each partition, the key-value pairs are sorted by key. This can be done either in memory or on disk, depending on the size of the data and the available resources.
  - Merging: The sorted partitions from different map tasks are merged into a single sorted stream for each reduce task. This can be done either by pulling the data from the map tasks (pull-based shuffle) or by pushing the data to the reduce tasks (push-based shuffle).
  - Grouping: The merged stream of key-value pairs is grouped by key, so that all the values with the same key are passed to the reduce function as an iterator.

Task execution in MapReduce

- MapReduce is a programming model designed to process large amounts of data in parallel by dividing the job into several independent local tasks.
- The execution of tasks is controlled by the MapReduce Execution Service, which plays the role of the worker process in the Google MapReduce implementation.
- The service manages the execution of map and reduce tasks and performs other operations, such as sorting and merging intermediate files.
- The complete execution process is also supervised by two types of entities

called a JobTracker and multiple TaskTrackers.
- The JobTracker acts like a master, responsible for scheduling, monitoring and re-executing the failed tasks .
- The TaskTrackers act like slaves, each of them performing the map or reduce tasks assigned by the JobTracker on their local data .
- The map tasks read the input data from the file system and apply a user-defined function to each record, producing a set of intermediate key-value pairs.
- The reduce tasks receive the intermediate key-value pairs from the map tasks, group them by key, and apply another user-defined function to each group, producing the final output.
- The framework sorts the outputs of the maps, which are then input to the reduce tasks.
- The framework also handles the failures of tasks, by re-executing them on different nodes if necessary .
- The framework also optimizes the performance of the tasks, by using techniques such as locality-aware scheduling, speculative execution, and combiners.

Map Reduce types in map reduce

MapReduce is a programming model for processing large-scale data sets in parallel and distributed manner. It consists of two main functions: map and reduce. The map function takes an input key-value pair and produces a list of intermediate key-value pairs. The reduce function takes an intermediate key and a list of values associated with that key, and produces a list of output key-value pairs.

The types of the input and output key-value pairs for the map and reduce functions can be different, depending on the application logic and the data format. The MapReduce framework provides a set of predefined types and formats for common use cases, as well as the ability to define custom types and formats.

Some of the predefined types and formats are:

- **Text**: A type that represents a sequence of characters. It can be used for both keys and values. The format for text is a plain text file, where each line is a record. The key and value are separated by a tab character.
- **LongWritable**: A type that represents a 64-bit integer. It can be used for both keys and values. The format for long is a binary file, where each record is a fixed-length byte array.
- **IntWritable**: A type that represents a 32-bit integer. It can be used for both keys and values. The format for int is a binary file, where each record is a fixed-length byte array.
- **FloatWritable**: A type that represents a 32-bit floating-point number. It can be used for both keys and values. The format for float is a binary file, where each record is a fixed-length byte array.

- **DoubleWritable**: A type that represents a 64-bit floating-point number. It can be used for both keys and values. The format for double is a binary file, where each record is a fixed-length byte array.
- **BooleanWritable**: A type that represents a boolean value. It can be used for both keys and values. The format for boolean is a binary file, where each record is a single byte.
- **BytesWritable**: A type that represents a variable-length byte array. It can be used for both keys and values. The format for bytes is a binary file, where each record is prefixed by a 4-byte integer that indicates the length of the byte array.
- **ArrayWritable**: A type that represents an array of writable objects. It can be used for both keys and values. The format for array is a binary file, where each record is prefixed by a 4-byte integer that indicates the number of elements in the array, followed by the serialized elements.
- **MapWritable**: A type that represents a map of writable objects. It can be used for both keys and values. The format for map is a binary file, where each record is prefixed by a 4-byte integer that indicates the number of entries in the map, followed by the serialized key-value pairs.
- **SequenceFile**: A format that stores a sequence of key-value pairs in a compressed binary file. It can be used for both input and output. The key and value types can be any writable types. The compression can be either record-level or block-level. SequenceFile is suitable for storing large and complex data sets that need to be processed efficiently.
- **TextInputFormat**: A format that reads plain text files as input. The key type is LongWritable, which represents the byte offset of the line in the file. The value type is Text, which represents the content of the line.
- **KeyValueTextInputFormat**: A format that reads plain text files as input. The key type and value type are both Text, which represent the key and value separated by a tab character in each line.
- **NLineInputFormat**: A format that reads plain text files as input. The key type is LongWritable, which represents the byte offset of the first line in the split. The value type is Text, which represents the content of N lines in the split. The number of lines per split can be specified by the user.
- **FileInputFormat**: An abstract class that defines the common functionality for input formats that read files. It handles the splitting of files into logical input splits, and the creation of record readers for each split. Subclasses of FileInputFormat need to implement the methods getRecordReader and isSplitable.
- **FileOutputFormat**: An abstract class that defines the common functionality for output formats that write files. It handles the creation of output files and directories, and the setting of compression options. Subclasses of FileOutputFormat need to implement the method getRecordWriter.

Input Formats in Map Reduce

- Input formats are classes that define how the input data is split into input splits and how the input records are read from the input splits.
- Input splits are logical chunks of the input data that are assigned to different map tasks for parallel processing.
- Input records are key-value pairs that are passed to the map function as input.
- The default input format in Map Reduce is TextInputFormat, which splits the input data by line and assigns the byte offset of the line as the key and the line content as the value.
- Other common input formats are KeyValueInputFormat, which splits the input data by line and assigns the first token as the key and the rest of the line as the value, and SequenceFileInputFormat, which reads binary key-value pairs from sequence files.
- Custom input formats can be implemented by extending the abstract class InputFormat and overriding the methods getSplits and createRecordReader.

Output Formats in Map Reduce

Output formats in Map Reduce are the specifications for how the output files of a Map Reduce job are written and stored in a file system. The output format also provides the implementation of the RecordWriter interface, which is responsible for writing the individual records of the output to the output files. The output format can be specified by the user using the `setOutputFormatClass()` method of the `Job` class.

There are different types of output formats available in Map Reduce, each with its own advantages and disadvantages. Some of the common output formats are:

- **TextOutputFormat**: This is the default output format in Map Reduce. It writes the output as plain text files, where each line is a record. The key and the value of each record are separated by a tab character. This output format is easy to read and process, but it may not be efficient for large or complex data types.
- **SequenceFileOutputFormat**: This output format writes the output as sequence files, which are binary files that store key-value pairs. Sequence files are more compact and efficient than text files, and they support compression and splitting. However, they are not human-readable and may require additional tools to process.
- **SequenceFileAsBinaryOutputFormat**: This output format is a variant of SequenceFileOutputFormat, where the key and the value of each record are written as binary data, without any serialization or deserialization. This output format is faster and more compact than SequenceFileOutputFormat, but it requires the user to provide custom classes for reading and writing the binary data.
- **MapFileOutputFormat**: This output format is another form of FileOut-

putFormat, where the output is written as map files, which are indexed sequence files that allow random access to the records. Map files are useful for applications that need to perform lookups or queries on the output data. However, they are more complex and require more disk space than sequence files.

- **MultipleOutputs**: This output format allows the user to write the output to multiple files or directories, based on the key or the value of each record. This output format is useful for partitioning or categorizing the output data. However, it requires the user to specify the output file names or paths for each record in the code.
- **LazyOutputFormat**: This output format is a wrapper around another output format, that prevents the creation of empty output files. This output format is useful for reducing the number of output files and saving disk space. However, it may incur some overhead in checking the existence of output files.
- **DBOutputFormat**: This output format writes the output to a relational database, using JDBC. This output format is useful for integrating the output data with other data sources or applications. However, it requires the user to provide the database connection details and the SQL statements for inserting the records.

Map Reduce features

MapReduce is a programming model and a framework for processing large-scale data sets in parallel and distributed manner. It consists of two main functions: map and reduce. The map function takes an input key-value pair and produces a set of intermediate key-value pairs. The reduce function takes all the intermediate values associated with the same intermediate key and combines them into a smaller set of values. Some of the salient features of MapReduce are:

- Scalability: MapReduce can handle huge amounts of data by distributing them across multiple nodes in a cluster. It can also scale up or down by adding or removing nodes as needed.
- Flexibility: MapReduce can process various types of data, such as structured, unstructured, or semi-structured. It can also support different kinds of operations, such as filtering, aggregation, sorting, joining, etc.
- Security and Authentication: MapReduce can provide security and authentication mechanisms to protect the data and the nodes from unauthorized access or malicious attacks. It can also encrypt the data during transmission and storage.
- Cost-effectiveness: MapReduce can run on commodity hardware, which reduces the cost of infrastructure and maintenance. It can also utilize the available resources efficiently and avoid wasting CPU cycles or disk space.
- Speed: MapReduce can perform parallel and distributed processing, which reduces the execution time and improves the performance. It can also leverage the locality of data and minimize the network traffic and latency.

- Simplicity: MapReduce provides a simple and intuitive programming model, which abstracts the complexity of parallel and distributed computing. The programmers only need to write the map and reduce functions, and the framework takes care of the rest.
- Parallelism: MapReduce can exploit the inherent parallelism in the data and the tasks, and execute them concurrently on multiple nodes. It can also handle the load balancing, fault tolerance, and synchronization issues automatically.
- Availability and Resilience: MapReduce can ensure the availability and resilience of the data and the nodes, by replicating them across the cluster. It can also recover from failures and resume the processing without losing any data or state.

Real-world Map Reduce

- MapReduce is a framework that was developed to process massive amounts of data efficiently by dividing the work into smaller tasks and distributing them among multiple machines or nodes.
- MapReduce consists of two phases: Map and Reduce. In the Map phase, the input data is split into chunks and each chunk is assigned to a mapper node that applies a user-defined function to transform the data into key-value pairs. In the Reduce phase, the key-value pairs from all the mappers are shuffled and sorted by key and then sent to a reducer node that applies another user-defined function to aggregate the values for each key and produce the final output.
- MapReduce can be used for various applications such as word count, inverted index, web log analysis, recommendation systems, machine learning, etc.
- One example of a real-world MapReduce application is Twitter, which receives around 500 million tweets per day, which is nearly 3000 tweets per second . Twitter uses MapReduce to analyze the tweets and extract useful information such as trending topics, sentiment analysis, user behavior, etc. The following steps illustrate how Twitter manages its tweets with the help of MapReduce:
  - Tokenize: Tokenizes the tweets into maps of tokens and writes them as key-value pairs, where the key is the token and the value is 1.
  - Filter: Filters unwanted words from the maps of tokens, such as stop words, punctuation, etc. and writes the filtered maps as key-value pairs.
  - Count: Generates a token counter per word by combining the values for each key and writes the key-value pairs, where the key is the word and the value is the count.
  - Aggregate Counters: Prepares an aggregate of similar counter values into small manageable units and writes the key-value pairs, where the

key is the count and the value is the list of words with that count.
  - Sort: Sorts the key-value pairs by key in descending order and writes the key-value pairs, where the key is the count and the value is the list of words with that count.
  - Output: Outputs the top N words with the highest counts as the final result.

Unit 4 - HDFS (Hadoop Distributed File System) and Hadoop Environment

- HDFS is a distributed file system that handles large data sets running on commodity hardware.
- HDFS is one of the major components of Apache Hadoop, the others being MapReduce and YARN.
- HDFS employs a NameNode and DataNode architecture to implement a distributed file system that provides high-performance access to data across highly scalable Hadoop clusters .
- HDFS has many similarities with existing distributed file systems, but also has some unique features, such as:
  - Fault tolerance: HDFS can tolerate hardware failures by replicating data blocks across multiple DataNodes.
  - High throughput: HDFS can support high data transfer rates by splitting large files into fixed-size blocks and distributing them across the cluster.
  - Scalability: HDFS can scale up to thousands of nodes and store petabytes of data.
  - Data locality: HDFS can optimize data processing by moving computation to the nodes where the data is stored, rather than moving data to the computation.
- HDFS consists of two types of nodes: NameNode and DataNode .
  - NameNode: The NameNode is the master node that manages the namespace and the metadata of the file system. It also coordinates the access to the files by the clients. There is only one active NameNode in a cluster, and it is a single point of failure .
  - DataNode: The DataNodes are the worker nodes that store the actual data blocks of the files. They also perform read and write operations on the blocks as instructed by the NameNode. There can be multiple DataNodes in a cluster, and they can be added or removed dynamically .
- HDFS follows a write-once-read-many model, which means that a file can be written only once and then read multiple times. This simplifies the data consistency and concurrency issues.
- HDFS also supports some advanced features, such as:
  - Snapshots: HDFS can create point-in-time copies of the file system, which can be used for backup or disaster recovery.
  - Federation: HDFS can support multiple namespaces, each managed

by a separate NameNode, to improve the scalability and isolation of the file system.
  - High Availability: HDFS can enable automatic failover of the NameNode by using a standby NameNode and a shared storage system.
  - Erasure Coding: HDFS can use a more efficient way of data replication, which reduces the storage overhead and improves the data durability.
- HDFS is designed to run on a Hadoop environment, which consists of a set of software tools and frameworks that enable distributed processing of large and complex data sets. Some of the key components of a Hadoop environment are:
  - MapReduce: A programming model and an execution framework that allows parallel processing of large data sets using a map and reduce functions.
  - YARN: A resource management and scheduling system that allocates and monitors the resources (CPU, memory, disk, network) for the applications running on the cluster.
  - Hadoop Common: A collection of libraries and utilities that provide the basic functionality and support for the other Hadoop components.
  - Hadoop Ecosystem: A set of projects and tools that extend the functionality and usability of Hadoop, such as Hive, Pig, Spark, HBase, Sqoop, Flume, etc.

HDFS

- HDFS stands for Hadoop Distributed File System. It is a distributed file system that handles large data sets running on commodity hardware .
- HDFS is one of the major components of Apache Hadoop, the others being MapReduce and YARN. Hadoop is an open source framework for processing and storing big data.
- HDFS operates by rapidly transferring data between nodes in a cluster. It is fault-tolerant and designed to be deployed on low-cost, commodity hardware .
- HDFS has a master-slave architecture, where one node acts as the NameNode (master) and the others act as DataNodes (slaves). The NameNode manages the file system namespace and the metadata, while the DataNodes store the actual data in blocks.
- HDFS provides a command-line interface and a web interface for users to interact with the file system. It also supports a Java API for application development.
- HDFS is suitable for applications that have large data sets, sequential access patterns, and high throughput requirements. It is not suitable for applications that need low latency, random access, or multiple writers.

Design of HDFS

- HDFS stands for Hadoop Distributed File System. It is a distributed file system that runs on a cluster of commodity hardware and provides high availability, scalability, fault tolerance, and reliability.
- HDFS follows a master-slave architecture, where there is one NameNode (master) and multiple DataNodes (slaves). The NameNode manages the namespace and the metadata of the file system, while the DataNodes store the actual data blocks of the files.
- HDFS splits large files into fixed-size blocks (typically 128 MB or 256 MB) and distributes them across the DataNodes. Each block is replicated on multiple DataNodes (usually three) for fault tolerance. The NameNode maintains the mapping of files to blocks and blocks to DataNodes.
- HDFS supports a write-once-read-many model, where files are written by a single client and then read by multiple clients. HDFS does not support random writes or updates to files, only appends. HDFS also supports atomic rename and delete operations on files and directories.
- HDFS provides a Java API for clients to interact with the file system. HDFS also supports a web-based interface and a command-line interface. HDFS supports integration with other frameworks such as MapReduce, Spark, Hive, and HBase.

HDFS concepts

- HDFS stands for Hadoop Distributed File System. It is a distributed file system that stores large-scale data across multiple nodes in a cluster.
- HDFS follows a master-slave architecture, where one node acts as the NameNode (master) and the rest of the nodes act as DataNodes (slaves).
- The NameNode is responsible for managing the namespace, metadata, and access control of the files and directories in HDFS. It also coordinates the replication and placement of data blocks among the DataNodes.
- The DataNodes are responsible for storing, reading, and writing the data blocks of the files in HDFS. They also send periodic heartbeats and block reports to the NameNode to indicate their status and availability.
- HDFS splits a file into fixed-size blocks (typically 128 MB) and distributes them across the DataNodes for parallel processing. Each block is replicated a number of times (default is 3) for fault tolerance and reliability.
- HDFS provides a client interface that allows users and applications to interact with the file system. The client communicates with the NameNode to obtain the location of the data blocks and then directly transfers data to and from the DataNodes.
- HDFS supports a write-once-read-many model, where a file can be written only once and then read multiple times. HDFS does not support random writes or updates to a file, but it supports appending new data to an existing file.
- HDFS is designed to handle large files and high throughput of data. It is optimized for streaming access rather than random access. It also supports compression and decompression of data to reduce the network and storage

overhead.

Benefits of HDFS

HDFS stands for Hadoop Distributed File System, which is a subproject of Apache Hadoop. It is a file system that stores large amounts of data across multiple nodes in a cluster, and provides high performance, fault tolerance, scalability, and compatibility.

Some of the benefits of HDFS are:

- **Fault tolerance**: HDFS can detect and recover from hardware failures automatically, ensuring data availability and reliability. It replicates data blocks across different nodes, and can re-replicate them if a node fails or is removed.
- **Speed**: HDFS can deliver high throughput of data by using a cluster architecture, where data is stored and processed locally on each node. It can support more than 2 GB of data per second per node.
- **Access to more types of data**: HDFS can store and process various kinds of data, such as structured, unstructured, and streaming data. It can handle data that is too large or complex for traditional file systems.
- **Compatibility and portability**: HDFS is compatible with multiple hardware platforms and operating systems, and can run on inexpensive off-the-shelf hardware. It is also compatible with various data processing frameworks, such as MapReduce, Spark, Hive, and Pig.
- **Scalable**: HDFS can scale up to thousands of nodes and petabytes of data, by adding or removing nodes as needed. It can handle the increasing volume, variety, and velocity of data in the big data era.
- **Data locality**: HDFS follows the principle of data locality, which means that data is stored and processed on the same or nearby nodes, reducing the network overhead and latency. This improves the performance and efficiency of data processing.
- **Cost effective**: HDFS is an open source project, which means that there is no licensing fee to use it. It also reduces the storage cost by using commodity hardware and compressing data blocks. It can store more data with less resources than traditional file systems.

Challenges of HDFS

HDFS is a distributed file system that is designed to store and process large amounts of data across multiple nodes in a cluster. HDFS has many advantages, such as high scalability, fault tolerance, and parallel processing. However, it also faces some challenges, such as:

- **Issues with small files**: HDFS is not suitable for storing and accessing small files, as each file occupies a block of fixed size (typically 64 MB or 128 MB) and requires a metadata entry in the NameNode, which is the master node that controls the file system. Storing many small files

can lead to inefficient use of disk space and memory, as well as increased network traffic and load on the NameNode .

- **Slow processing speed**: HDFS relies on MapReduce, a programming model that divides a large task into smaller subtasks that are executed in parallel by different nodes. MapReduce has some limitations, such as high latency, lack of support for iterative and interactive processing, and dependency on disk-based intermediate data .
- **Support for batch processing only**: HDFS is designed for batch processing, which means that it can only process data that is already stored in the file system. It is not suitable for streaming data, which is continuously generated and needs to be processed in real time or near real time .
- **No real-time processing**: HDFS does not support real-time processing, which means that it cannot provide immediate feedback or response to the user or the application. HDFS is mainly used for offline analysis and reporting, rather than online transactions or queries .
- **Iterative processing**: HDFS does not support iterative processing, which means that it cannot perform multiple passes over the same data set for complex computations, such as machine learning and graph algorithms. HDFS requires each iteration to read and write data from and to the disk, which adds to the overhead and latency .
- **Latency**: HDFS has a high latency, which means that it takes a long time to access and process data. This is because HDFS has to perform multiple operations, such as locating the data blocks, transferring the data across the network, and executing the MapReduce tasks, before returning the results .
- **No ease of use**: HDFS is not easy to use, as it requires the user to write complex MapReduce programs in Java or other languages, and to deal with low-level details, such as data partitioning, data serialization, and fault handling. HDFS also lacks a standard query language, such as SQL, that can simplify the data manipulation and analysis .
- **Security issue**: HDFS has a weak security mechanism, as it relies on the underlying operating system for authentication and authorization. HDFS does not encrypt the data at rest or in transit, and does not provide fine-grained access control or auditing features. HDFS also does not support data provenance, which is the ability to track the origin and history of the data .

File sizes in HDFS

- HDFS stands for Hadoop Distributed File System, which is a distributed storage system for large-scale data processing.
- HDFS is designed to support files that are gigabytes to terabytes in size . It provides high throughput and scalability by splitting files into fixed-size blocks and distributing them across multiple nodes in a cluster.
- The default size of an HDFS block is 128 MB, but it can be configured

to a different value depending on the application needs. Larger blocks reduce the overhead of managing metadata and network transfers, but also increase the latency of accessing small files or portions of files.
- To get the size of a file or a directory in HDFS, one can use the `hadoop fs -du` command . This command shows the base size of the file or directory before replication, which means the actual space used by the file or directory may be larger depending on the replication factor. The command also supports wildcards and summary options to filter and aggregate the results. For example, `hadoop fs -du -s /user/hduser/input/sou*` will show the total size of all files that start with `sou` in the `/user/hduser/input` directory.

Block sizes in HDFS

- HDFS is a distributed file system that stores large files across multiple nodes in a cluster.
- HDFS splits files into fixed-size blocks and distributes them across the nodes for storage.
- Blocks are the smallest unit of data that HDFS can read or write.
- The default block size in HDFS is 128 MB, which is much larger than the typical block size of 4 KB in a traditional file system.
- The advantages of having a large block size in HDFS are:
  - It reduces the overhead of managing metadata, such as the location and size of each block, by storing fewer blocks per file.
  - It improves the network bandwidth utilization, by transferring large chunks of data in a single operation, rather than many small ones.
  - It enhances the fault tolerance, by replicating each block across multiple nodes, and allowing the system to recover from node failures by switching to another replica.
- The disadvantages of having a large block size in HDFS are:
  - It increases the disk seek time, by requiring the disk head to move more to access a specific block.
  - It wastes disk space, by padding the last block of a file with zeros if the file size is not a multiple of the block size.
  - It limits the parallelism, by restricting the number of tasks that can process a file concurrently, as each task can only work on one block at a time.
- The block size in HDFS can be configured by setting the parameter `dfs.blocksize` in the `hdfs-site.xml` file.
- The block size can also be specified for individual files at the time of creation, by using the `-D` option with the `hadoop fs -put` or `hadoop fs -copyFromLocal` commands.
- The block size can be checked for existing files by using the `hadoop fs -stat` command.

Block abstraction in HDFS

- HDFS is a distributed file system that stores large files across multiple machines.
- HDFS breaks down each file into fixed-size blocks, typically 128 MB or 256 MB, and stores them on different nodes in the cluster.
- Each block is replicated across multiple nodes, usually three, for fault tolerance and availability.
- HDFS provides a block abstraction that hides the details of where and how the blocks are stored from the user and the application.
- The user and the application interact with HDFS through a logical view of the file, which consists of a file name, a file length, and a list of blocks.
- HDFS maintains a metadata service called the NameNode, which keeps track of the file names, file lengths, block locations, and block replicas for each file in the file system.
- The NameNode also handles operations such as creating, deleting, renaming, and appending files, as well as changing the replication factor of blocks.
- The actual data blocks are stored and served by the DataNodes, which are the worker nodes in the cluster.
- The DataNodes periodically report to the NameNode about the blocks they store and the available disk space.
- The NameNode uses this information to balance the load and the disk space across the cluster, and to recover from node failures by replicating the missing blocks to other nodes.

Data replication in HDFS

- HDFS is a distributed file system that stores large amounts of data across multiple nodes in a cluster.
- Data replication is a technique that creates multiple copies of the same data block and stores them on different nodes for fault tolerance and high availability.
- HDFS follows a master-slave architecture, where a single NameNode manages the metadata of the file system, and multiple DataNodes store the actual data blocks.
- When a file is written to HDFS, it is split into fixed-size blocks (default 128 MB) and each block is replicated to a number of DataNodes (default 3) based on the replication factor.
- The NameNode decides which DataNodes to store the replicas of each block, and maintains a mapping of file names, block IDs, and DataNode locations.
- The NameNode also periodically receives heartbeat and block report messages from the DataNodes, which indicate the health and status of each DataNode and the blocks they store.
- The NameNode uses a replication policy to balance the load and ensure the reliability of the data across the cluster. The policy considers factors such as rack awareness, node capacity, node availability, and block placement.

- The NameNode can initiate replication or deletion of blocks to maintain the desired replication factor and free up space on the DataNodes.
- The DataNodes are responsible for serving read and write requests from the clients, and performing block operations such as creation, deletion, and replication as instructed by the NameNode.
- The DataNodes also communicate with each other to transfer blocks for replication or recovery purposes.
- The clients interact with the NameNode to obtain the metadata of the files and the locations of the blocks, and then directly communicate with the DataNodes to read or write the data blocks.

How does HDFS store

- HDFS stands for Hadoop Distributed File System, which is a scalable and fault-tolerant storage system for big data processing.
- HDFS stores data in a distributed manner, by dividing the files into fixed-size blocks (default 128 MB) and storing them on different DataNodes in the cluster.
- DataNodes are the slave nodes that store the actual data blocks and serve read and write requests from the clients.
- NameNode is the master node that maintains the file system namespace and the metadata of all the files and blocks in the cluster. It also manages the replication and placement of blocks across DataNodes.
- HDFS follows a write-once-read-many model, which means that once a file is written, it cannot be modified. However, it can be appended or deleted.
- HDFS provides high availability and reliability by replicating each block on multiple DataNodes (default 3) according to the replication factor. This ensures that the data is not lost even if some DataNodes fail or become inaccessible.
- HDFS also supports rack awareness, which means that it tries to place the replicas of a block on different racks (groups of nodes) to reduce the network bandwidth and increase the fault tolerance.
- HDFS allows users to access the files stored in the cluster through a variety of interfaces, such as the Hadoop command-line, the HDFS Java API, the HDFS REST API, or the HDFS web UI.

Read operations in HDFS

- HDFS is a distributed file system that stores large files across multiple nodes in a cluster.
- HDFS follows a master-slave architecture, where a single NameNode manages the metadata of the file system, and multiple DataNodes store the actual data blocks of the files.
- To read a file from HDFS, a client application first contacts the NameNode and obtains the list of DataNodes that store the replicas of the blocks of the file.

- The client then contacts one of the DataNodes directly and requests the transfer of the desired block. The DataNode sends the block to the client as a stream of bytes over a TCP connection.
- The client can read the block from the stream and verify its checksum. If the checksum does not match, the client can request the same block from another DataNode that has a replica of that block.
- The client repeats this process until it has read all the blocks of the file. The client can also read the blocks in parallel from multiple DataNodes to improve the read performance.
- HDFS supports both sequential and random access to files. Sequential access is more efficient as it can take advantage of the locality of the blocks. Random access requires the client to seek to the desired position in the file, which may involve contacting the NameNode multiple times to locate the blocks.

Write operations in HDFS

- HDFS is a distributed file system that stores large files across multiple nodes in a cluster.
- HDFS supports write-once-read-many model, which means that a file can be written only once and then read by multiple clients.
- HDFS write operations involve three main steps: client request, data pipeline, and replication.
- Client request: The client contacts the NameNode to request permission to write a file. The NameNode checks if the file already exists, if the client has the write permission, and if there is enough space in the cluster. If all the conditions are met, the NameNode allocates a new file in the namespace and returns a list of DataNodes that will store the file blocks.
- Data pipeline: The client splits the file into fixed-size blocks (typically 128 MB) and sends them to the DataNodes in a pipeline fashion. The first DataNode in the pipeline stores the block and forwards it to the next DataNode, and so on. The client receives an acknowledgment from each DataNode after the block is stored successfully.
- Replication: The DataNodes replicate the blocks to other DataNodes based on the replication factor (typically 3) specified by the client or the default configuration. The replication process ensures fault tolerance and data availability in case of node failures. The NameNode periodically receives block reports from the DataNodes and maintains the metadata of the file system.

Java interfaces to HDFS

- HDFS stands for Hadoop Distributed File System, which is a scalable and fault-tolerant storage system for large-scale data processing applications.
- HDFS provides several interfaces for accessing and manipulating data stored in its filesystem, such as command-line interface, web interface,

and Java interface.

- The Java interface is the most commonly used interface for Hadoop filesystem interactions, as it exposes the Hadoop FileSystem class, which is the abstract base class for all Hadoop filesystem implementations.
- The FileSystem class provides methods for creating, reading, writing, deleting, renaming, and listing files and directories in HDFS, as well as obtaining metadata and statistics about the filesystem.
- To use the Java interface, one needs to create a FileSystem object by passing a Configuration object that contains the HDFS configuration parameters, such as the namenode address, the replication factor, and the block size.
- The FileSystem object can then be used to obtain a Path object, which represents a file or a directory in HDFS, and perform various operations on it using the FileSystem methods.
- For example, to create a file in HDFS, one can use the create() method of the FileSystem class, which returns a FSDataOutputStream object that can be used to write data to the file.
- Similarly, to read a file in HDFS, one can use the open() method of the FileSystem class, which returns a FSDataInputStream object that can be used to read data from the file.
- The FileSystem class also provides methods for appending, copying, moving, and deleting files and directories, as well as checking the existence, permission, and ownership of a Path object.
- Additionally, the FileSystem class provides methods for querying the filesystem, such as getting the status, capacity, usage, and block locations of a Path object, as well as listing the files and directories under a given Path object.
- The Java interface for HDFS is flexible and powerful, as it allows users to interact with different Hadoop filesystem implementations, such as HDFS, S3, and Azure Blob Storage, using the same FileSystem API.

Command Line Interface to HDFS

- HDFS stands for Hadoop Distributed File System, which is a scalable and fault-tolerant storage system for big data processing.
- Command Line Interface (CLI) is one of the simplest ways to interact with HDFS. CLI has support for filesystem operations like reading the file, creating directories, moving files, deleting data, and listing directories .
- To use CLI, we need to run the `hdfs` command with the appropriate subcommand and options. For example, `hdfs dfs -ls /` will list the contents of the root directory in HDFS.
- The `hdfs` command has several subcommands, such as `dfs`, `dfsadmin`, `fsck`, `balancer`, etc. Each subcommand has its own syntax and options.

We can run `hdfs <subcommand> -help` to get detailed help on every subcommand.

- Some of the common subcommands and their usage are:
  - `dfs` : Performs basic file system operations on HDFS, such as copy, move, delete, etc. For example, `hdfs dfs -put localfile.txt /user/hadoop/hdfsfile.txt` will copy a local file to HDFS.
  - `dfsadmin` : Performs administrative operations on HDFS, such as report, safemode, refresh, etc. For example, `hdfs dfsadmin -report` will display the summary of the cluster status, such as the number of live and dead nodes, the total and used capacity, etc.
  - `fsck` : Checks the health of the HDFS file system, such as the number of missing or corrupted blocks, the replication factor, etc. For example, `hdfs fsck /user/hadoop` will check the integrity of the files and directories under the given path.
  - `balancer` : Balances the disk space usage across the data nodes in the cluster, by moving blocks from over-utilized nodes to under-utilized nodes. For example, `hdfs balancer -threshold 10` will start the balancer process with a threshold of 10%, which means the balancer will try to make the disk space usage of each node within 10% of the cluster average.
- To use CLI with Data Lake Storage Gen2, which is a cloud-based storage service that supports HDFS, we need to establish remote access to an HDInsight Hadoop cluster on Linux, and then execute the basic HDFS commands as usual. For example, `ssh sshuser@clustername-ssh.azurehdinsight.net` will connect to the cluster via SSH, and then `hdfs dfs -ls /` will list the contents of the root directory in Data Lake Storage Gen2.

Hadoop file system interfaces

- Hadoop file system interfaces are the Java classes and methods that allow applications to interact with various file systems in Hadoop.
- The abstract class `org.apache.hadoop.fs.FileSystem` represents the client interface to a file system in Hadoop, and there are several concrete implementations for different file systems, such as HDFS, S3, FTP, etc.
- Hadoop file system interfaces provide methods for creating, deleting, renaming, reading, writing, and listing files and directories, as well as querying file system properties, such as capacity, used space, replication factor, block size, etc.
- Hadoop file system interfaces use the URI scheme to select the appropriate file system instance to communicate with, based on the scheme and authority of the URI. For example, `hdfs://namenode:port/path` refers to the HDFS file system, while `s3://bucket/path` refers to the S3 file system.

- Hadoop file system interfaces also support streaming access to file system data, which allows applications to read and write data in a sequential manner without seeking or random access. This is useful for processing large files or data streams efficiently.
- Hadoop file system interfaces can be accessed through the command-line interface, the Java API, or the web interface. The command-line interface provides basic commands for file system operations, such as `hadoop fs -ls`, `hadoop fs -put`, `hadoop fs -get`, etc. The Java API provides more flexibility and functionality for file system operations, such as `FileSystem.get()`, `FileSystem.create()`, `FileSystem.open()`, etc. The web interface provides a graphical view of the file system status and contents, such as the namenode and datanode information, the file system tree, the file properties, etc.

Data flow in HDFS

- HDFS is a distributed file system that stores large files across multiple nodes in a cluster.
- HDFS follows a master-slave architecture, where one node acts as the NameNode (master) and the rest of the nodes are DataNodes (slaves).
- The NameNode is responsible for managing the namespace, the metadata, and the access control of the files and directories in HDFS.
- The DataNodes are responsible for storing the actual data blocks of the files in HDFS.
- A file in HDFS is split into fixed-size blocks (typically 128 MB or 256 MB) and each block is replicated across multiple DataNodes for fault tolerance (typically 3 replicas).
- The NameNode maintains a mapping of which file corresponds to which blocks and which blocks are located on which DataNodes.
- When a client wants to read a file from HDFS, it first contacts the NameNode to get the list of blocks and their locations for the file.
- Then, the client contacts one of the DataNodes that has a replica of the first block and starts reading the data from it.
- When the end of the block is reached, the client contacts the NameNode again to get the location of the next block and repeats the process until the end of the file is reached.
- When a client wants to write a file to HDFS, it first contacts the NameNode to request a new file name and the list of DataNodes to store the replicas of the first block.
- Then, the client contacts the first DataNode in the list and starts sending the data to it.
- The first DataNode then forwards the data to the second DataNode in the list, which in turn forwards it to the third DataNode, forming a pipeline.
- When the first block is filled, the client contacts the NameNode again to get the list of DataNodes for the next block and repeats the process until the end of the file is reached.

- The NameNode periodically receives heartbeat and block report messages from the DataNodes to monitor their status and the block locations.
- If a DataNode fails or becomes unreachable, the NameNode detects it and marks the blocks on that DataNode as missing.
- The NameNode then initiates the replication of the missing blocks from the existing DataNodes to the new DataNodes to restore the replication factor.
- If the NameNode fails or becomes unreachable, the entire HDFS cluster becomes inaccessible, as there is no other node that can serve the namespace and the metadata.
- To prevent the loss of the NameNode state, HDFS supports the following mechanisms:
  - The NameNode stores its metadata on the local disk as well as on a remote NFS mount (called the secondary NameNode) as a backup.
  - The NameNode also writes its metadata to a write-ahead log (called the edit log) on the local disk and the remote NFS mount, which records every change made to the file system.
  - The NameNode can be configured to run in a high-availability mode, where there are two NameNodes (called the active and the standby) that share a common edit log and a common storage for the metadata.
  - The active NameNode is the one that serves the client requests and updates the edit log and the metadata, while the standby NameNode is the one that monitors the active NameNode and takes over its role in case of a failure.

Data Ingest with Flume and Sqoop in HDFS

- Data ingestion is the process of collecting, transferring, and loading data from various sources into a data store, such as HDFS, for analysis and processing.
- Data ingestion can be done in different ways, depending on the type, source, and volume of data, as well as the desired destination and frequency of ingestion.
- Flume and Sqoop are two popular tools in the Hadoop ecosystem that can be used for data ingestion into HDFS from different sources.
- Flume is a distributed, reliable, and scalable service that can collect, aggregate, and move large amounts of streaming data, such as log files, events, or messages, from various sources to HDFS or other data stores.
- Sqoop is a tool that can efficiently transfer bulk data between Hadoop and structured data sources, such as relational databases, using a map-reduce job.
- Flume and Sqoop have different use cases and advantages, depending on the nature and source of the data to be ingested.

Some of the key points to compare Flume and Sqoop are:

- Flume can handle streaming data, such as logs or events, that are gen-

erated continuously and need to be ingested in near real-time, while Sqoop can handle batch data, such as tables or records, that are stored in databases and need to be ingested periodically or on demand.
- Flume can ingest data from multiple and diverse sources, such as web servers, application servers, social media, sensors, or message queues, using various sources, channels, and sinks, while Sqoop can ingest data from relational databases or any JDBC compatible data source, using connectors and commands.
- Flume can perform data filtering, transformation, and enrichment on the ingested data, using interceptors and serializers, while Sqoop can perform data validation, compression, and partitioning on the ingested data, using parameters and options.
- Flume can ingest data into various destinations, such as HDFS, HBase, Hive, Kafka, or Solr, using different sinks, while Sqoop can ingest data into HDFS, Hive, or HBase, using different modes and formats.
- Flume can ingest data in parallel, fault-tolerant, and load-balanced manner, using multiple agents, channels, and sinks, while Sqoop can ingest data in parallel, fault-tolerant, and incremental manner, using multiple mappers, splits, and checkpoints.

Hadoop archives in HDFS

- Hadoop archives (HAR) are a way of compressing large numbers of small files in HDFS to reduce the metadata overhead and improve the performance of the file system.
- HAR files are similar to ZIP or TAR files, but they are not compressed. They are created by using the `hadoop archive` command, which takes a list of files or directories as input and produces a single HAR file as output.
- HAR files are stored in HDFS as regular files, but they have a special format that allows them to be accessed as directories. The HAR file system is a layer on top of the HDFS file system that transparently handles the mapping between HAR files and their contents.
- HAR files can be accessed using the `har://` scheme in the HDFS URI. For example, `har:///user/hadoop/myarchive.har/file1.txt` refers to the file `file1.txt` inside the HAR file `myarchive.har` in the HDFS directory `/user/hadoop`.
- HAR files can be used to improve the performance of MapReduce jobs that need to process a large number of small files. By archiving the small files into a single HAR file, the number of mappers can be reduced and the input split size can be increased. This reduces the overhead of launching and managing many tasks and improves the data locality.

Hadoop I/O

- Hadoop I/O is the data input and output system of Hadoop, which is

designed to handle large and distributed datasets efficiently.

- Hadoop I/O supports various data formats, such as text, binary, sequence, map files, and others, that can be used for different purposes and applications.
- Hadoop I/O also provides several compression codecs, such as gzip, bzip2, snappy, and lz4, that can reduce the storage space and network bandwidth required for data processing.
- Hadoop I/O uses the Hadoop Distributed File System (HDFS) as the underlying storage layer, which provides high availability, fault tolerance, and scalability for data blocks across multiple nodes.
- Hadoop I/O also leverages the MapReduce framework, which is a programming model for parallel and distributed processing of large datasets using key-value pairs.
- Hadoop I/O enables users to write custom input and output formats, as well as custom serializers and deserializers, to handle complex and heterogeneous data types and sources.

Compression in Hadoop IO

- Compression is a technique to reduce the size of data by encoding it in a different format that uses fewer bits.
- Compression can improve the performance of Hadoop applications by reducing the disk space, network bandwidth, and CPU usage required to store and process data.
- Hadoop supports various compression codecs, such as Gzip, Bzip2, Snappy, LZO, LZ4, and Zstandard, that have different trade-offs between compression ratio, speed, and splittability.
- Splittability refers to the ability to split a compressed file into smaller chunks that can be processed independently by different mappers.
- Some compression codecs, such as Gzip and Bzip2, are splittable only at file boundaries, while others, such as Snappy and LZO, are splittable within files using an index file or a container format.
- Hadoop allows users to configure compression at different levels, such as input, output, intermediate, and shuffle, by setting the appropriate properties in the configuration files or the code.
- Hadoop also provides a compression API that allows users to create and use custom compression codecs.

Serialization in Hadoop IO

- Serialization is the process of converting structured data into a byte stream for transmission over the network or storage on disk .
- Deserialization is the reverse process of converting the byte stream back into the original structured data .
- Serialization is used in Hadoop for interprocess communication between nodes using remote procedure calls (RPCs) .

- Serialization is also used in Hadoop for writing and reading data to and from Hadoop Distributed File System (HDFS).
- Hadoop provides its own serialization framework called Writable, which is optimized for performance and compactness .
- Writable is an interface that defines two methods: write(DataOutput out) and readFields(DataInput in), which are used to serialize and deserialize the data respectively .
- Hadoop also supports other serialization frameworks, such as Avro, Thrift, and Protocol Buffers, which offer more features and flexibility than Writable.
- Hadoop provides a mechanism for using different serialization frameworks in Hadoop through the org.apache.hadoop.io.serializer package .
- This package defines two interfaces: Serialization and Serializer, which are used to register and obtain serializers for different data types.
- Hadoop also provides a generic wrapper class called org.apache.hadoop.io.serializer.WritableSerialization, which can be used to serialize any class that implements Writable.

Avro and file based data structures in Hadoop io

- Avro is a language-neutral data serialization system that can be used for Hadoop and other big data processing frameworks.
- Avro creates binary structured files that are both compressible and splittable, which makes them efficient for Hadoop MapReduce jobs .
- Avro files store data along with the schema in JSON format in the metadata section, which makes them self-describing and easy to read and interpret by any program .
- Avro supports a rich data structure that includes primitive types, complex types, and logical types.
- Avro files can be imported and exported to and from HDFS using Sqoop, a tool for transferring data between relational databases and Hadoop.
- Avro files are similar to sequential files, which are the oldest binary file format in Hadoop, but offer better performance and features.
- Avro files can be used with Kafka, a distributed streaming platform, for faster data processing.

Hadoop Environment

- Hadoop is an open source software framework that is used for storing and processing large amounts of data in a distributed computing environment .
- Hadoop is based on the MapReduce programming model, which allows for the parallel processing of large datasets across clusters of commodity computers . Commodity computers are cheap and widely available.
- Hadoop has two main components: Hadoop Distributed File System (HDFS) and Hadoop MapReduce .

  - HDFS is a distributed file system that provides high-throughput access to data and can store data across multiple machines .
  - Hadoop MapReduce is a software framework that supports the execution of MapReduce programs on large clusters of nodes .
- Hadoop requires the Java Runtime Environment (JRE) 1.6 or higher and Secure Shell (SSH) to be set up between nodes in the cluster.
- Hadoop is one of the technologies by which data can be managed over very large amounts of data and is at the center of what is known as "big data".

Setting up a Hadoop cluster in Hadoop Environment

Hadoop is a framework for distributed processing of large-scale data using a cluster of machines. A Hadoop cluster consists of a master node and one or more worker nodes. The master node runs the NameNode and the ResourceManager, which are responsible for managing the file system and the resources of the cluster. The worker nodes run the DataNode and the NodeManager, which store and process the data.

To set up a Hadoop cluster in a Hadoop environment, you need to follow these steps:

- Configure the environment of the Hadoop daemons. This includes setting the JAVA_HOME and HADOOP_HOME variables, creating a Hadoop user and group, and enabling passwordless SSH access between the nodes .
- Configure the Hadoop parameters. This includes editing the core-site.xml, hdfs-site.xml, mapred-site.xml, and yarn-site.xml files in the $HADOOP_HOME/etc/hadoop directory to specify the cluster name, the NameNode address, the replication factor, the memory and CPU allocation, and other options .
- Format the HDFS file system. This is done by running the command `hdfs namenode -format` on the master node as the Hadoop user.
- Start the Hadoop daemons. This is done by running the commands `start-dfs.sh` and `start-yarn.sh` on the master node as the Hadoop user. This will start the NameNode, the DataNode, the ResourceManager, and the NodeManager on the respective nodes.
- Verify the cluster status. This can be done by using the commands `hdfs dfsadmin -report` and `yarn node -list` to check the health and availability of the nodes, or by accessing the web interfaces of the NameNode and the ResourceManager on the master node.

Alternatively, you can use a cloud service such as Azure HDInsight to create a Hadoop cluster using the Azure portal. This will allow you to choose the cluster type, size, location, storage, and security options, and will automatically configure and deploy the cluster for you .

Cluster Specification in Hadoop Environment

- A Hadoop cluster is a special type of computational cluster designed specifically for storing and analyzing huge amounts of unstructured data in a distributed computing environment .
- A Hadoop cluster consists of a collection of computers, known as nodes, that are networked together to perform parallel computations on big data sets.
- A Hadoop cluster is often referred to as a shared-nothing system because the only thing that is shared between the nodes is the network itself.
- A Hadoop cluster typically has two types of nodes: master nodes and worker nodes.
- Master nodes are responsible for coordinating the activities of the cluster, such as scheduling jobs, managing resources, and monitoring the health of the cluster.
- Worker nodes are responsible for executing the tasks assigned by the master nodes, such as storing and processing data.
- A Hadoop cluster can have one or more master nodes and any number of worker nodes, depending on the size and complexity of the data and the computational requirements.
- A Hadoop cluster runs Hadoop's open source distributed processing software, which consists of four main components: Hadoop Distributed File System (HDFS), MapReduce, YARN, and Hadoop Common .
- HDFS is a distributed file system that provides high-throughput access to large data sets across the cluster.
- MapReduce is a programming model and framework that allows parallel processing of data using key-value pairs.
- YARN is a resource management platform that allocates and manages resources for the cluster.
- Hadoop Common is a set of utilities and libraries that support the other components of Hadoop.
- To configure a Hadoop cluster, you will need to configure the environment in which the Hadoop daemons execute as well as the configuration parameters for the Hadoop daemons .
- The Hadoop daemons are NameNode, DataNode, ResourceManager, NodeManager, and ApplicationMaster.
- NameNode and DataNode are the master and worker daemons for HDFS, respectively.
- ResourceManager and NodeManager are the master and worker daemons for YARN, respectively.
- ApplicationMaster is the daemon that runs on a worker node and manages the execution of a single application on the cluster.
- The environment configuration includes setting up the Java environment, creating a dedicated Hadoop user, setting up SSH access, and creating directories for Hadoop.
- The configuration parameters include setting up the core-site.xml, hdfs-

site.xml, mapred-site.xml, and yarn-site.xml files, which specify the properties of the Hadoop components.

Cluster setup and installation in Hadoop Environment

- A Hadoop cluster is a collection of machines that run the Hadoop software and store the data in a distributed manner.
- A Hadoop cluster can be classified into two types: single-node cluster and multi-node cluster.
- A single-node cluster is a cluster that consists of only one machine, which acts as both the master and the worker node. It is useful for testing and development purposes, but not for production use.
- A multi-node cluster is a cluster that consists of more than one machine, which are divided into master nodes and worker nodes. The master nodes are responsible for managing the cluster resources and coordinating the tasks, while the worker nodes are responsible for executing the tasks and storing the data.
- To set up and install a Hadoop cluster, the following steps are required:
    1. Install Java on all the machines, as Hadoop is written in Java and requires Java Runtime Environment (JRE) to run.
    2. Download and extract the Hadoop software from the official website or a mirror site on all the machines.
    3. Configure the Hadoop software by editing the configuration files in the etc/hadoop directory. The main configuration files are core-site.xml, hdfs-site.xml, mapred-site.xml, and yarn-site.xml. These files specify the parameters for the Hadoop components, such as the location of the NameNode and the DataNodes, the replication factor, the memory and CPU allocation, and the scheduler.
    4. Set up the SSH connection between the machines, so that the master node can communicate with the worker nodes without password authentication. This can be done by generating and exchanging the SSH keys using the ssh-keygen and ssh-copy-id commands.
    5. Format the Hadoop Distributed File System (HDFS) by running the hdfs namenode -format command on the master node. This will initialize the NameNode and create the metadata for the file system.
    6. Start the Hadoop cluster by running the start-all.sh script on the master node. This will start the NameNode, the DataNode, the ResourceManager, the NodeManager, and the JobHistoryServer on the respective nodes.
    7. Verify the status of the Hadoop cluster by using the web interface or the command-line tools. The web interface can be accessed by using the URLs http://master-node:50070 for the NameNode, http://master-node:8088 for the ResourceManager, and

http://master-node:19888 for the JobHistoryServer. The command-line tools include the hdfs dfsadmin -report, the yarn node -list, and the mapred job -list commands.

Hadoop configuration in Hadoop Environment

- Hadoop configuration is the process of setting up the parameters and properties of the Hadoop components, such as HDFS, MapReduce, YARN, and HBase, to optimize their performance and functionality in a given environment.
- Hadoop configuration files are XML files that contain the key-value pairs of the configuration properties and their values. These files are stored in the etc/hadoop directory of the Hadoop installation.
- The main configuration files are:
  - core-site.xml: This file contains the core settings of Hadoop, such as the default file system URI, the I/O settings, and the security options.
  - hdfs-site.xml: This file contains the settings of HDFS, such as the replication factor, the block size, the name node and data node directories, and the checkpoint options.
  - mapred-site.xml: This file contains the settings of MapReduce, such as the framework name, the job tracker and task tracker addresses, the memory and CPU limits, and the compression options.
  - yarn-site.xml: This file contains the settings of YARN, such as the resource manager and node manager addresses, the scheduler type, the resource allocation and utilization, and the application master options.
  - hbase-site.xml: This file contains the settings of HBase, such as the zookeeper quorum, the region server and master addresses, the compaction and flush policies, and the WAL options.
- Hadoop configuration can be done in three ways:
  - Editing the XML files directly: This is the simplest and most common way of configuring Hadoop. However, it requires restarting the Hadoop services for the changes to take effect, and it can be prone to human errors and inconsistencies.
  - Using the Hadoop command-line interface: This is a more dynamic and flexible way of configuring Hadoop. It allows changing the configuration properties on the fly, without restarting the services, and it can be applied to specific jobs or applications. However, it can be tedious and complex to use, and it can override the default settings in the XML files.
  - Using the Hadoop web interface: This is a more user-friendly and graphical way of configuring Hadoop. It provides a web-based dashboard that shows the status and metrics of the Hadoop cluster, and allows modifying the configuration properties through a web form. However, it can be less secure and reliable than the other methods,

and it can have limited functionality and compatibility.

Security in Hadoop in Hadoop Environment

- Security in Hadoop is the process of protecting the data and services in a Hadoop cluster from unauthorized access, modification, or disclosure.
- Security in Hadoop can be divided into four pillars: authentication, authorization, encryption, and audit.
- Authentication is the process of verifying the identity of a user or a service before allowing access to the Hadoop cluster. Authentication can be done using Kerberos, which is a network protocol that uses tickets and encryption to authenticate users and services .
- Authorization is the process of granting or denying access to the data and services in the Hadoop cluster based on the authenticated identity of the user or service. Authorization can be done using Access Control Lists (ACLs), which are lists of users and groups that have permissions to access specific files or directories in HDFS, or using Apache Ranger, which is a framework that provides centralized and fine-grained access control for Hadoop components .
- Encryption is the process of transforming the data in the Hadoop cluster into an unreadable form that can only be decrypted by authorized parties. Encryption can be done using Transparent Data Encryption (TDE), which is a feature that encrypts the data at rest in HDFS, or using Apache Knox, which is a gateway that encrypts the data in transit between the Hadoop cluster and the external clients .
- Audit is the process of recording and monitoring the activities and events in the Hadoop cluster for security and compliance purposes. Audit can be done using Apache Audit, which is a component that collects and stores audit logs from various Hadoop components, or using Apache Atlas, which is a metadata management and governance framework that tracks the lineage and provenance of the data in the Hadoop cluster .
- Security in Hadoop is important for ensuring the confidentiality, integrity, and availability of the data and services in the Hadoop cluster, as well as complying with the regulatory and legal requirements of the organization that uses Hadoop.
- Security in Hadoop is challenging because Hadoop is a distributed system that consists of multiple components and services that interact with each other and with external clients, and because Hadoop was originally designed for web data in a trusted environment, not for enterprise data in a secure environment .
- Security in Hadoop can be improved by using the secure mode of Hadoop, which requires authentication for every user and service, by applying the best practices and recommendations for configuring and managing the security features of Hadoop, and by using the latest versions of Hadoop and its components that have the most updated security patches and enhancements .

Administering Hadoop in Hadoop Environment

- Hadoop is a framework for distributed processing of large-scale data sets across clusters of computers.
- Hadoop consists of two main components: Hadoop Distributed File System (HDFS) and MapReduce.
- HDFS is a distributed file system that stores data in blocks on multiple nodes in a cluster, providing high availability, fault tolerance, and scalability.
- MapReduce is a programming model that allows parallel processing of data on HDFS using two types of functions: map and reduce.
- Hadoop also provides other components and tools, such as Hadoop YARN, Hadoop Common, Hadoop Streaming, Hadoop Pig, Hadoop Hive, Hadoop HBase, Hadoop ZooKeeper, Hadoop Oozie, Hadoop Sqoop, and Hadoop Flume.
- Administering Hadoop in Hadoop environment involves the following tasks:
  - Installing and configuring Hadoop on a cluster of nodes.
  - Managing and monitoring Hadoop services, such as NameNode, DataNode, ResourceManager, NodeManager, JobTracker, TaskTracker, etc.
  - Managing and securing Hadoop users, groups, and permissions.
  - Managing and optimizing Hadoop performance, such as tuning parameters, balancing load, and troubleshooting issues.
  - Managing and backing up Hadoop data, such as creating and deleting files and directories, replicating and recovering blocks, and archiving and restoring data.
  - Managing and upgrading Hadoop versions, such as applying patches, rolling upgrades, and migrating data.
  - Managing and integrating Hadoop with other systems, such as databases, data warehouses, and analytics tools.

HDFS monitoring & maintenance in Hadoop Environment

- HDFS is the distributed file system that stores large amounts of data across multiple nodes in a Hadoop cluster.
- HDFS monitoring & maintenance involves checking the health and performance of the HDFS components, such as NameNode, DataNodes, Secondary NameNode, and clients.
- Some of the common tasks for HDFS monitoring & maintenance are:
  - Monitoring the disk space usage and capacity of the HDFS cluster, and adding or removing nodes as needed.
  - Monitoring the replication factor and block distribution of the HDFS files, and balancing the load across the cluster.

  - Monitoring the status and logs of the NameNode and DataNodes, and detecting and resolving any failures or errors.
  - Monitoring the network bandwidth and latency of the HDFS cluster, and optimizing the data transfer and communication.
  - Monitoring the HDFS security and access control, and enforcing the policies and permissions.
  - Performing regular backups and snapshots of the HDFS metadata and data, and restoring them in case of disasters.
  - Performing periodic upgrades and patches of the HDFS software and configuration, and ensuring the compatibility and stability of the cluster.

- Some of the tools and frameworks that can be used for HDFS monitoring & maintenance are:
  - Hadoop web UI: A web interface that provides information and statistics about the HDFS cluster, such as the number of nodes, files, blocks, and bytes.
  - Hadoop metrics: A framework that collects and publishes various metrics about the HDFS cluster, such as the disk space, throughput, latency, and errors.
  - Hadoop JMX: A framework that exposes the management and monitoring information of the HDFS components, such as the memory, threads, and garbage collection.
  - Hadoop command-line tools: A set of commands that can be used to perform various operations on the HDFS cluster, such as creating, deleting, copying, and listing files and directories.
  - Hadoop fsck: A command that can be used to check the consistency and validity of the HDFS files and blocks, and report any corrupted or missing blocks.
  - Hadoop balancer: A tool that can be used to balance the disk space usage and block distribution across the HDFS cluster, and improve the data locality and performance.
  - Hadoop distcp: A tool that can be used to copy large amounts of data between HDFS clusters, or between HDFS and other file systems.
  - Hadoop snapshot: A feature that can be used to create point-in-time copies of the HDFS files and directories, and preserve the data from accidental or malicious changes.
  - Hadoop backup and restore: A feature that can be used to backup and restore the HDFS metadata and data, and recover from failures or disasters.

Hadoop benchmarks in Hadoop Environment

- Hadoop benchmarks are tools or programs that measure the performance of Hadoop clusters and applications in various aspects, such as throughput, latency, scalability, reliability, etc.

- Hadoop benchmarks can be used for testing, tuning, debugging, and comparing different Hadoop configurations, versions, and implementations.
- Hadoop benchmarks can be classified into two categories: micro-benchmarks and macro-benchmarks.
  - Micro-benchmarks focus on specific components or features of Hadoop, such as HDFS, MapReduce, YARN, etc. They usually have simple and synthetic workloads that stress a particular aspect of the system. Examples of micro-benchmarks are TestDFSIO, TeraSort, PiEstimator, etc.
  - Macro-benchmarks simulate real-world applications or scenarios that run on Hadoop, such as data analytics, machine learning, graph processing, etc. They usually have complex and realistic workloads that involve multiple stages and data sources. Examples of macro-benchmarks are HiBench, BigBench, BigDataBench, etc.
- Hadoop benchmarks can be run in different modes, such as local mode, pseudo-distributed mode, and fully-distributed mode, depending on the number and type of nodes in the cluster.
  - Local mode runs the benchmark on a single node, without using HDFS or MapReduce. It is mainly used for development and debugging purposes.
  - Pseudo-distributed mode runs the benchmark on a single node, but with HDFS and MapReduce enabled. It simulates a multi-node cluster with different processes running on the same machine. It is useful for testing and prototyping purposes.
  - Fully-distributed mode runs the benchmark on a multi-node cluster, with HDFS and MapReduce distributed across different machines. It is the most realistic and reliable mode for measuring the performance of Hadoop in a production environment.
- Hadoop benchmarks can be executed using different tools or frameworks, such as Hadoop command-line interface, Hadoop Streaming, Hadoop Pipes, Apache Spark, Apache Flink, Apache Hive, Apache Pig, etc. Each tool or framework has its own advantages and disadvantages in terms of ease of use, expressiveness, efficiency, scalability, etc.

Hadoop in the cloud in Hadoop Environment

- Hadoop is a software framework that allows users to process large data sets in a distributed environment using a cluster of computers.
- Hadoop consists of four main modules: Hadoop Distributed File System (HDFS), MapReduce, YARN, and Hadoop Common.
- HDFS is a distributed file system that runs on standard or low-end hardware and provides high data throughput, fault tolerance, and scalability.
- MapReduce is a programming model that enables parallel processing of large data sets across the cluster.
- YARN is a resource management layer that allocates and schedules resources for the applications running on the cluster.

- Hadoop Common is a set of libraries and utilities that support the other modules.
- Hadoop can run on various platforms, including on-premises hardware, private cloud, public cloud, or hybrid cloud.
- Running Hadoop on the cloud has several advantages, such as:
  - Lower upfront cost and operational overhead, as the cloud provider manages the infrastructure and offers pay-as-you-go pricing.
  - Higher flexibility and scalability, as the cloud provider can provision and deprovision resources on demand and offer various storage and compute options.
  - Better availability and reliability, as the cloud provider can ensure high uptime and backup of the data and applications.
  - Easier integration and innovation, as the cloud provider can offer various tools and services that complement Hadoop, such as artificial intelligence, machine learning, analytics, and security.
- Some of the challenges of running Hadoop on the cloud are:
  - Data transfer and latency, as moving large data sets between the cloud and on-premises can be costly and time-consuming, and the network performance can affect the application performance.
  - Data security and privacy, as storing and processing sensitive data on the cloud can pose risks of unauthorized access, breach, or leakage, and require compliance with various regulations and standards.
  - Vendor lock-in and compatibility, as different cloud providers may have different offerings, features, and APIs for Hadoop, and migrating from one provider to another can be difficult and expensive.

Unit 3 - Hadoop Eco System and YARN , no SQL databases , MongoDB , Spark , Scala

- Hadoop Ecosystem is a platform or a suite which provides various services to solve the big data problems. It includes Apache projects and various commercial tools and solutions.
- There are four major elements of Hadoop i.e. HDFS, MapReduce, YARN, and Hadoop Common.
- HDFS is the distributed file system that stores the data in a cluster of nodes. MapReduce is the programming model that processes the data in parallel. Hadoop Common is the set of libraries and utilities that support the other components. YARN is the resource manager that allocates and manages the resources in the cluster.
- YARN stands for Yet Another Resource Negotiator. It is responsible for managing and monitoring workloads, scheduling tasks, and handling failures. YARN is also called as the operating system of Hadoop as it provides a common platform for running various applications on top of HDFS.
- NoSQL databases are non-relational databases that store and retrieve data in different ways than the traditional SQL databases. They are designed to handle large volumes of unstructured, semi-structured, or structured

data with high scalability, availability, and performance.

- MongoDB is one of the most popular NoSQL databases. It is a document-oriented database that stores data in JSON-like format. MongoDB supports dynamic schemas, meaning that the documents in a collection can have different fields and structures. MongoDB also provides various features such as indexing, aggregation, replication, sharding, and text search.
- Spark is an open-source framework for big data processing. It is based on the concept of resilient distributed datasets (RDDs), which are immutable collections of data that can be distributed across multiple nodes and processed in parallel. Spark supports various operations on RDDs such as transformations, actions, and queries.
- Spark can run on top of Hadoop, Mesos, Kubernetes, or standalone. It can also coexist with MapReduce and other Hadoop ecosystem components. Spark is also popular because it supports SQL, which helps overcome a shortcoming in core Hadoop technology. The Spark programming environment works interactively with Scala, Python, and R shells.
- Scala is a general-purpose programming language that combines the features of object-oriented and functional programming. It is designed to be concise, expressive, and scalable. Scala runs on the Java Virtual Machine (JVM) and interoperates with Java code. Scala is also the native language of Spark, meaning that it can access all the Spark APIs and libraries.

Hadoop Eco System and YARN

- Hadoop Eco System is a collection of open source software components and tools that work together to provide a distributed computing platform for big data processing and analysis.
- Some of the most well-known components of the Hadoop Eco System are HDFS, MapReduce, YARN, Hive, Pig, HBase, Spark, Oozie, Sqoop, Zookeeper, etc.
- YARN stands for Yet Another Resource Negotiator. It is a sub-project of Hadoop that provides a framework for resource management and job scheduling in a Hadoop cluster.
- YARN was introduced in Hadoop 2.0 to overcome the limitations of Hadoop 1.0, such as scalability, efficiency, and flexibility.

- YARN consists of two main components: a global ResourceManager (RM) that allocates resources across the cluster, and a per-application ApplicationMaster (AM) that coordinates the execution of tasks for each application.
- YARN supports multiple types of applications, not just MapReduce, and allows them to run concurrently on the same cluster.
- YARN also offers features such as security, high availability, fault tolerance, and dynamic resource allocation.

Hadoop ecosystem components

The Hadoop ecosystem is a collection of software components and tools that enable the processing and analysis of large-scale data sets on a distributed computing platform. The Hadoop ecosystem consists of four main layers: data storage, data processing, data access, and data management. Some of the most common components of the Hadoop ecosystem are:

- **HDFS**: Hadoop Distributed File System, a distributed and scalable file system that stores data across multiple nodes in a cluster.
- **MapReduce**: A programming model and framework for parallel processing of large data sets using key-value pairs.
- **YARN**: Yet Another Resource Negotiator, a resource management layer that allocates and schedules computing resources for different applications running on a Hadoop cluster.
- **Hive**: A data warehouse system that provides a SQL-like interface for querying and analyzing data stored in HDFS.
- **Pig**: A high-level scripting language and platform for data analysis and transformation using MapReduce.
- **Spark**: A fast and general-purpose engine for large-scale data processing, supporting batch, streaming, SQL, machine learning, and graph analytics.
- **HBase**: A distributed and column-oriented database that provides random access and consistent updates for large-scale structured and semi-structured data.
- **Oozie**: A workflow scheduler and coordinator that manages and executes Hadoop jobs and workflows.
- **Sqoop**: A tool that transfers data between Hadoop and relational databases.
- **Zookeeper**: A service that provides coordination, synchronization, configuration, and naming for distributed systems.

Schedulers in Hadoop Ecosystem

- Schedulers are algorithms that are used to schedule tasks in a Hadoop cluster when multiple clients submit requests for data processing.
- Schedulers help in ensuring optimal utilization of resources and access to unused capacity in the cluster.
- Schedulers also help in managing the priority, fairness, and SLA of different jobs and queues.
- There are mainly four types of schedulers in Hadoop ecosystem :
  - **FIFO (First In First Out) Scheduler**: This is the default and simplest scheduler in Hadoop. It schedules jobs in the order of their submission. It does not consider the size, complexity, or priority of the jobs. It is suitable for small clusters with homogeneous and short jobs.

  - **Capacity Scheduler**: This is a pluggable scheduler that allows multiple queues to be created with different capacities and properties. It allocates resources to each queue based on its capacity, and within each queue, it uses FIFO scheduling. It supports hierarchical queues, preemption, resource sharing, and ACLs. It is suitable for large clusters with heterogeneous and long-running jobs.
  - **Fair Scheduler**: This is another pluggable scheduler that aims to provide fair and equal share of resources to all jobs and queues. It dynamically adjusts the resource allocation based on the demand and the weight of the jobs and queues. It supports hierarchical queues, preemption, resource sharing, and ACLs. It is suitable for large clusters with heterogeneous and short-running jobs.
  - **YARN Scheduler**: This is the scheduler used by YARN, the resource management framework in Hadoop 2. It has two components: a global **Resource Manager** that manages the cluster resources, and a per-node **Node Manager** that manages the containers on each node. It supports two types of schedulers: **Capacity Scheduler** and **Fair Scheduler**. It also supports dynamic resource allocation, node labels, and reservation system. It is suitable for large clusters with heterogeneous and diverse workloads.

Fair and Capacity in Hadoop Ecosystem

- Hadoop is a batch processing ecosystem that can handle large-scale data analysis using distributed computing.
- Hadoop has a distributed storage layer called HDFS (Hadoop Distributed File System) that splits the incoming data into blocks and stores them across multiple nodes in a cluster.
- Hadoop also has a distributed processing layer called YARN (Yet Another Resource Negotiator) that manages the resources and tasks for the applications running on the cluster.
- Hadoop uses schedulers to allocate resources and schedule tasks for the applications based on different policies and priorities.
- There are mainly three types of schedulers in Hadoop: FIFO, Capacity, and Fair.
- FIFO (First In First Out) scheduler is the simplest and default scheduler that assigns resources to jobs in the order they are submitted. It does not consider the priority or size of the jobs and can cause starvation for smaller or later jobs.
- Capacity scheduler is a more advanced scheduler that allows multiple queues with different capacities and priorities to be created for different groups of users or applications. It ensures that each queue gets a minimum share of the cluster resources and can use the excess resources if available. It also supports preemption and limits for the queues to avoid starvation and resource wastage.
- Fair scheduler is another advanced scheduler that aims to provide a fair

share of resources to all the jobs over time. It dynamically balances the resources between the running jobs based on their demand and priority. It also supports multiple queues with weights and min/max shares to accommodate different needs and preferences. It also supports preemption and limits for the queues to avoid starvation and resource wastage.

Hadoop 2.0 New Features - NameNode high availability

- NameNode is the master node in HDFS that maintains the filesystem tree and the metadata of all the files and directories.
- In Hadoop 1.x, NameNode was a single point of failure (SPOF) in an HDFS cluster. If the NameNode failed or became unavailable, the entire cluster would be inaccessible until the NameNode was restored or replaced.
- Hadoop 2.0 overcomes this SPOF problem by providing support for multiple NameNodes. It introduces Hadoop 2.0 High Availability feature that brings in an extra NameNode (Passive Standby NameNode) to the Hadoop Architecture which is configured for automatic failover .
- The Active NameNode and the Standby NameNode use a shared storage directory called the EditLog to keep their states synchronized. The EditLog records every change that occurs to the file system metadata .
- The DataNodes send block reports and heartbeats to both the NameNodes. The Standby NameNode also performs checkpoints of the namespace by merging the EditLog with the FsImage (the file that stores the entire file system namespace) and saving it back to the shared storage .
- In case of a failure or a planned maintenance of the Active NameNode, the Standby NameNode takes over the role of the Active NameNode and starts serving the client requests. This process is called failover and can be triggered manually or automatically by a component called ZooKeeper Failover Controller (ZKFC) .
- The ZKFC is a daemon that runs on each of the NameNode machines and monitors the health and status of the NameNode. It also communicates with a ZooKeeper quorum (a set of servers that provide a consistent view of the cluster state) to elect an Active NameNode and ensure that there is only one Active NameNode at a time .
- The main benefit of the Hadoop 2.0 High Availability feature is that it enables the HDFS cluster to be available 24/7 for large data applications and eliminates the downtime and data loss caused by the NameNode failure .

HDFS Federation in Hadoop Ecosystem

- HDFS Federation is a feature introduced in Hadoop 2 that enhances the existing HDFS architecture by adding multiple NameNode/namespaces support to HDFS.
- A NameNode is a master node that manages the metadata of the files and directories stored in HDFS. A namespace is a logical grouping of files and directories under a common root directory.

- HDFS Federation overcomes the limitations of the previous HDFS architecture, such as:
  - Single point of failure: If the NameNode fails, the entire HDFS becomes inaccessible.
  - Scalability bottleneck: The NameNode has to store all the metadata in memory, which limits the number of files and blocks that can be managed by HDFS.
  - Performance bottleneck: The NameNode has to handle all the requests from the clients and the DataNodes, which can cause high network and CPU load.
- HDFS Federation allows the use of more than one NameNode/namespace, each with its own portion of the file system metadata. This provides the following benefits:
  - Isolation: Each namespace is independent and isolated from each other, which improves availability and security.
  - Scalability: The total capacity and throughput of HDFS can be increased by adding more NameNodes/namespaces.
  - Performance: The load on the NameNode is distributed among multiple NameNodes, which reduces the network and CPU congestion.
- HDFS Federation architecture consists of the following components:
  - Namespace Volume: A self-contained management unit that consists of a NameNode and a block pool. A block pool is a set of blocks that belong to a single namespace. Each namespace volume has a unique ID and can be mounted at any point in the global namespace tree.
  - DataNode: A slave node that stores the data blocks of the files in HDFS. A DataNode can belong to multiple block pools and report to multiple NameNodes. A DataNode identifies a block by its block pool ID and block ID.
  - Client: A user or an application that accesses the files and directories in HDFS. A client can interact with multiple NameNodes/namespaces by using a mount table that maps the paths in the global namespace to the corresponding namespace volumes.

MRv2 in Hadoop ecosystem

- MRv2 stands for MapReduce version 2, which is an application framework that runs within YARN (Yet Another Resource Negotiator) .
- YARN is a component of Hadoop 2 that separates the resource management and scheduling tasks from the data processing layer .
- MRv2 provides backward compatibility with the org.apache.hadoop.mapred APIs of Hadoop 1, which means that the compiled binaries can run without any modification on the new framework .
- MRv2 also supports new APIs such as org.apache.hadoop.mapreduce, which offer more features and flexibility than the old ones .
- MRv2 enables Hadoop to support other application engines besides MapReduce, such as Spark, Storm, and Tez, which can utilize YARN for

cluster resource management .
- MRv2 improves the performance of MapReduce by allowing dynamic allocation of resources, fine-grained control over tasks, and better fault tolerance .

YARN

- Yarn is a long continuous length of interlocked fibres, suitable for use in the production of textiles, sewing, crocheting, knitting, weaving, embroidery, or ropemaking .
- Yarn can be made of natural or synthetic materials, such as cotton, wool, rayon, metal, glass, or plastic .
- Thread is a type of yarn intended for sewing by hand or machine .
- Yarn can have different thicknesses, textures, colors, and patterns, depending on the type of fibre, the spinning method, the dyeing process, and the ply.
- Yarn can also be used as a noun to mean a long or implausible story, or as a verb to mean telling such a story .

Running MRv1 in YARN

- MRv1 is the original version of MapReduce that consists of interfaces and classes from the Java package org.apache.hadoop.mapred.
- YARN is the newer version of MapReduce that consists of interfaces and classes from the Java package org.apache.hadoop.mapreduce. YARN is also known as MRv2 or Yet Another Resource Negotiator.
- YARN separates the resource management and processing components of MapReduce, allowing multiple types of applications to run on the same cluster.
- MRv1 applications can run on YARN with minimal changes, as YARN provides backward compatibility for MRv1 applications .
- To run MRv1 applications on YARN, the following steps are required:
  - Set the mapreduce.framework.name property to yarn in the mapred-site.xml file.
  - Use the yarn command in the Hadoop-YARN bin folder rather than the hadoop command to submit the applications.
  - Use the ResourceManager web interface to monitor the applications running on the YARN cluster. The ResourceManager UI shows the basic cluster metrics, list of applications, and nodes associated with the cluster.
- The advantages of running MRv1 applications on YARN are :
  - Improved resource utilization and scalability, as YARN can dynamically allocate resources to applications based on demand and availability.
  - Enhanced fault tolerance and reliability, as YARN can recover from application and node failures without affecting the entire cluster.

  - Increased flexibility and extensibility, as YARN can support multiple types of applications and frameworks besides MapReduce, such as Spark, Hive, Pig, etc.

NoSQL Databases

- NoSQL databases are non-relational databases that store and retrieve data in different ways than traditional relational databases.
- NoSQL databases are designed to handle large volumes of unstructured, semi-structured, or dynamic data, such as social media posts, documents, or sensor data.
- NoSQL databases offer more flexibility, scalability, and performance than relational databases, especially for applications that require real-time processing, distributed computing, or cloud-based deployment.
- NoSQL databases can be classified into four main types: key-value, document, column, and graph databases.
- Key-value databases store data as pairs of keys and values, where the key is a unique identifier and the value can be any type of data. Examples of key-value databases are Redis, DynamoDB, and Riak.
- Document databases store data as documents, which are collections of fields and values that can be nested and have different schemas. Examples of document databases are MongoDB, CouchDB, and Elasticsearch.
- Column databases store data as columns, which are groups of values that share the same attribute. Each column can have a different number of rows and can be compressed and indexed for fast querying. Examples of column databases are Cassandra, HBase, and Bigtable.
- Graph databases store data as nodes and edges, which represent entities and relationships in a network. Graph databases allow complex queries that traverse the connections between data. Examples of graph databases are Neo4j, OrientDB, and Titan.

Introduction to NoSQL databases

- NoSQL databases are databases that do not use the SQL language or the relational model for data storage and retrieval.
- NoSQL databases are designed to handle large volumes of unstructured, semi-structured, or structured data that may change rapidly or frequently.

- NoSQL databases offer flexible schemas, high scalability, high performance, and easy distribution across multiple nodes.

- NoSQL databases can be classified into four main types based on their data model: document, key-value, wide-column, and graph.
- Document databases store data as JSON-like documents, where each document has a unique key and can contain nested fields and arrays. Document databases are suitable for applications that need to store and

query complex and heterogeneous data. Examples of document databases are MongoDB, CouchDB, and Elasticsearch.

- Key-value databases store data as pairs of keys and values, where each key is unique and can be used to retrieve the associated value. Key-value databases are suitable for applications that need to store and access simple and homogeneous data with high speed and low latency. Examples of key-value databases are Redis, DynamoDB, and Couchbase.

- Wide-column databases store data as tables of rows and columns, where each row has a unique key and each column can have different attributes and values. Wide-column databases are suitable for applications that need to store and analyze large amounts of sparse and dynamic data. Examples of wide-column databases are Cassandra, HBase, and Bigtable.
- Graph databases store data as nodes and edges, where each node represents an entity and each edge represents a relationship between entities. Graph databases are suitable for applications that need to store and query complex and interconnected data. Examples of graph databases are Neo4j, OrientDB, and Amazon Neptune.

MongoDB

- MongoDB is an open source, nonrelational database management system (DBMS) that uses flexible documents instead of tables and rows to process and store various forms of data .
- MongoDB is a distributed database at its core, so high availability, horizontal scaling, and geographic distribution are built in and easy to use.
- MongoDB is a document-oriented database, which means that data is stored as documents, and documents are grouped in collections. The document model is a lot more natural for developers to work with because documents are self-contained and can be treated as objects.
- MongoDB supports ad-hoc queries, secondary indexing, and real-time aggregations to provide powerful ways to access and analyze your data .
- MongoDB is developed by MongoDB Inc. and licensed under the Server Side Public License (SSPL) which is deemed non-free by several distributions.

Introduction to MongoDB

MongoDB is a document-based NoSQL database that provides efficient and flexible storage for a variety of different types of data sets. It is designed for modern application development and for the cloud. It has a scale-out architecture that allows you to meet the increasing demand for your system by adding more nodes to share the load. Some of the key aspects of MongoDB are:

- Documents: A record in MongoDB is a document, which is a data structure composed of field and value pairs. MongoDB documents are similar

to JSON objects. The values of fields may include other documents, arrays, and arrays of documents. Documents correspond to native data types in many programming languages.

- Collections: A collection is a group of documents stored in MongoDB, and can be thought of as roughly the equivalent of a table in a relational database. Collections are schemaless, which means that the documents within a collection can have different fields and structures.
- Databases: A database is a set of collections. A single MongoDB server can host multiple databases, each with its own collections and permissions.
- Queries: MongoDB provides a rich query language that allows you to perform CRUD (create, read, update, and delete) operations, as well as complex aggregations, text search, geospatial queries, and more. Queries can be executed using the MongoDB shell, drivers, or tools.
- Indexes: Indexes support the efficient execution of queries in MongoDB. Without indexes, MongoDB must scan every document in a collection to find the documents that match the query. With indexes, MongoDB can quickly narrow down the possible documents using the index and then only scan the relevant documents. Indexes are similar to indexes in relational databases, but MongoDB supports more types of indexes, such as multi-key, text, geospatial, and hashed indexes.
- Replication: Replication is the process of synchronizing data across multiple servers. MongoDB uses replication to provide high availability, fault tolerance, and scalability. MongoDB's replication feature is called replica sets, which consist of a primary node and one or more secondary nodes. The primary node receives all write operations and replicates them to the secondary nodes. The secondary nodes can serve read operations and can also become the primary node in case of a failure.
- Sharding: Sharding is the process of distributing data across multiple servers. MongoDB uses sharding to provide horizontal scalability, which means that you can add more servers to handle more data and traffic. MongoDB's sharding feature is called sharded clusters, which consist of shards, mongos, and config servers. Shards are the servers that store the data, mongos are the routers that direct the queries to the appropriate shards, and config servers are the servers that store the metadata and configuration of the cluster.

Data Types in MongoDB

- MongoDB is a document-oriented database that stores data in BSON format, which is a binary-encoded version of JSON.
- BSON supports various data types, some of which are similar to JSON and some of which are specific to MongoDB.
- The following are some of the common data types in MongoDB:
  - String: This is the most commonly used data type to store text data.

Strings in MongoDB must be UTF-8 valid.
  - Integer: This is a data type that is used to store numerical values, such as integers. MongoDB supports 32-bit or 64-bit integers, depending on the server.
  - Double: This is a data type that is used to store floating-point numbers, such as decimals. MongoDB stores all numbers that are not integers as doubles by default.
  - Boolean: This is a data type that is used to store logical values, such as true or false.
  - Object: This is a data type that is used to store embedded documents, which are key-value pairs. Objects in MongoDB are similar to JSON objects.
  - Array: This is a data type that is used to store ordered lists of values, which can be of any data type. Arrays in MongoDB are similar to JSON arrays.
  - Date: This is a data type that is used to store date and time values. MongoDB stores dates as 64-bit integers that represent the number of milliseconds since the Unix epoch (Jan 1, 1970).
  - ObjectId: This is a data type that is used to store unique identifiers for documents. MongoDB generates an ObjectId for each document automatically, which consists of 12 bytes that encode the timestamp, machine identifier, process identifier, and a counter.
  - Other data types: MongoDB also supports other data types, such as binary data, regular expressions, JavaScript code, decimal numbers, and geospatial data. For more details, refer to the official documentation.

Creating documents in MongoDB

- MongoDB is a document-oriented database that stores data in collections of JSON-like documents.
- A document is a set of key-value pairs, where the value can be any of the supported BSON data types, such as strings, numbers, arrays, objects, etc.
- To create documents in MongoDB, you can use one of the following methods: `insertOne()`, `insertMany()`, or `insert()`.
- The `insertOne()` method inserts a single document into a collection, using the following syntax:

  ```
  db.collection.insertOne(document, options)
  ```

  - The `document` parameter is the document to insert.
  - The `options` parameter is an optional object that specifies additional settings, such as write concern, bypass document validation, etc.
  - The method returns a `WriteResult` object that contains information

about the operation, such as the number of documents inserted, the `_id` field of the inserted document, etc.

- The `insertMany()` method inserts multiple documents into a collection, using the following syntax:

  ```
  db.collection.insertMany(documents, options)
  ```

  - The `documents` parameter is an array of documents to insert.
  - The `options` parameter is an optional object that specifies additional settings, such as write concern, bypass document validation, ordered or unordered insertion, etc.
  - The method returns a `WriteResult` object that contains information about the operation, such as the number of documents inserted, the `_id` fields of the inserted documents, etc.

- The `insert()` method inserts one or more documents into a collection, using the following syntax:

  ```
  db.collection.insert(document or array of documents, options)
  ```

  - The `document or array of documents` parameter is either a single document or an array of documents to insert.
  - The `options` parameter is an optional object that specifies additional settings, such as write concern, bypass document validation, etc.
  - The method returns a `WriteResult` object that contains information about the operation, such as the number of documents inserted, the `_id` fields of the inserted documents, etc.

- To create a collection in MongoDB, you can either explicitly use the `create()` command or implicitly create it when inserting the first document into a non-existing collection.

- The `create()` command creates a collection with the specified name and options, using the following syntax:

  ```
  db.createCollection(name, options)
  ```

  - The `name` parameter is the name of the collection to create.
  - The `options` parameter is an optional object that specifies additional settings, such as capped collection, validation rules, indexes, etc.
  - The command returns an object that contains information about the operation, such as the name of the created collection, the ok status, etc.

- To implicitly create a collection, you can use any of the insert methods on a non-existing collection, and MongoDB will create the collection automatically with the default options.

- For example, the following command will create a collection named `movie` and insert a document into it:

```
db.movie.insertOne({"name":"Avengers: Endgame"})
```

- To view the documents in a collection, you can use the `find()` method, which returns a cursor to the matching documents, using the following syntax:

  ```
  db.collection.find(query, projection)
  ```

  - The `query` parameter is an optional object that specifies the criteria for selecting documents.
  - The `projection` parameter is an optional object that specifies the fields to include or exclude in the returned documents.
  - The method returns a cursor object that allows you to iterate over the documents or apply additional methods, such as `sort()`, `limit()`, `skip()`, etc.

- For example, the following command will return all the documents in the `movie` collection:

  ```
  db.movie.find()
  ```

- To view the collections in a database, you can use the `show collections` command in the mongo shell, which lists the names of the collections in the current database.

- For example, the following command will show the collections in the `test` database:

  ```
  use test
  show collections
  ```

Updating documents in MongoDB

- MongoDB is a document-oriented database that stores data in JSON-like format.
- To update documents in MongoDB, you can use the `updateOne()`, `updateMany()`, or `replaceOne()` methods of the `db.collection` object.
- The `updateOne()` method updates a single document that matches a given filter condition. It takes two parameters: a filter object and an update object. The filter object specifies the criteria for selecting the document to update, and the update object specifies the changes to apply to the document. For example:

```
// Update the name field of the document with _id = 1
db.users.updateOne({_id: 1}, {$set: {name: "Alice"}})
```

- The `updateMany()` method updates all documents that match a given filter condition. It takes the same parameters as the `updateOne()` method, but applies the update to multiple documents. For example:

```
// Update the status field of all documents with age > 18
db.users.updateMany({age: {$gt: 18}}, {$set: {status: "active"}})
```

- The `replaceOne()` method replaces a single document that matches a given filter condition with a new document. It takes two parameters: a filter object and a replacement object. The filter object specifies the criteria for selecting the document to replace, and the replacement object specifies the new document to insert. For example:

```
// Replace the document with _id = 2 with a new document
db.users.replaceOne({_id: 2}, {name: "Bob", age: 25, status: "inactive"})
```

- To update documents in MongoDB, you can also use the `db.collection.findAndModify()` method, which combines the functionality of `find()`, `update()`, and `remove()` methods. It takes a query object, an update object, and an optional options object. The query object specifies the criteria for selecting the document to modify, the update object specifies the changes to apply to the document, and the options object specifies additional parameters such as whether to return the original or modified document, whether to sort the documents before updating, and whether to create a new document if none matches the query. For example:

```
// Find the document with the lowest age and increment it by 1, returning the modified docu
db.users.findAndModify({
  query: {},
  update: {$inc: {age: 1}},
  sort: {age: 1},
  new: true
})
```

Deleting documents in MongoDB

- MongoDB is a document-oriented database that stores data in collections of JSON-like documents.
- To delete documents from a collection, MongoDB provides the following methods:
  - `db.collection.deleteOne(filter, options)`: Deletes a single document that matches the filter condition. If multiple documents match the filter, only the first one is deleted. The options parameter can specify write concern and collation settings.
  - `db.collection.deleteMany(filter, options)`: Deletes all documents that match the filter condition. The options parameter can specify write concern and collation settings.
  - `db.collection.remove(query, justOne)`: Deprecated. Deletes documents from a collection that match the query condition. The justOne parameter can be set to true to delete only one document. This method is equivalent to `db.collection.deleteOne()` or `db.collection.deleteMany()` depending on the value of justOne.

- To delete an entire collection, MongoDB provides the following methods:
  - `db.collection.drop()`: Drops the collection from the database, along with its indexes. Returns true if the collection is dropped successfully, or false if the collection does not exist.
  - `db.dropCollection(name, options)`: Drops the collection with the specified name from the database, along with its indexes. The options parameter can specify write concern settings. Returns a document that contains the status of the operation.
- To delete a database, MongoDB provides the following methods:
  - `db.dropDatabase()`: Drops the current database, along with all its collections and indexes. Returns a document that contains the status of the operation.
  - `use <database>` and `db.dropDatabase()`: Switches to the specified database and drops it, along with all its collections and indexes. Returns a document that contains the status of the operation.

Querying Documents in MongoDB

- MongoDB is a document-oriented database that stores data in JSON-like format.
- To query data from MongoDB collection, you need to use the `find()` method, which returns a cursor that manages query results.
- The basic syntax of the `find()` method is as follows:

```
db.collection_name.find(query, projection)
```

- The `query` parameter is an optional document that specifies the criteria for selecting documents. If omitted, the `find()` method returns all documents in the collection.
- The `projection` parameter is an optional document that specifies the fields to include or exclude in the query result. If omitted, the `find()` method returns all fields in the documents.
- You can use various operators and expressions to build complex queries that match documents based on different criteria, such as equality, comparison, logical, array, element, evaluation, etc.
- You can also query embedded or nested documents using the dot notation or the `$elemMatch` operator.
- For example, to query documents that have an `address` field with a `city` subfield equal to `"New York"`, you can use the following query:

```
db.users.find({"address.city": "New York"})
```

- To query documents that have an `orders` array field with at least one element that matches the specified criteria, you can use the `$elemMatch` operator:

```
db.users.find({orders: {$elemMatch: {status: "delivered", amount: {$gt: 100}}}})
```

- To learn more about querying documents in MongoDB, you can refer to the official documentation or some online tutorials .

Indexing in MongoDB

- Indexing is a process that improves the performance of queries by creating data structures that store a small portion of the collection's data.
- Indexes support efficient execution of queries by reducing the number of documents that need to be scanned.
- MongoDB provides various types of indexes, such as single field, compound, multikey, text, geospatial, hashed, and wildcard indexes.
- Each index has a unique name and a specification that defines the fields to be indexed and the order of the index keys.
- By default, MongoDB creates a unique index on the `_id` field of each collection, which prevents the insertion of duplicate documents with the same `_id` value.
- To create an index, use the `db.collection.createIndex()` method, which takes an index key specification and an optional index options document as parameters.
- To list the indexes of a collection, use the `db.collection.getIndexes()` method, which returns an array of index documents.
- To drop an index, use the `db.collection.dropIndex()` method, which takes an index name or an index key specification as a parameter.
- To drop all indexes of a collection, use the `db.collection.dropIndexes()` method, which takes no parameters.
- To modify an existing index, such as changing its options or adding or removing fields, you need to drop the index and recreate it with the new specification.

Aggregation in MongoDB

- Aggregation is the process of selecting data from a collection in MongoDB and performing various operations on the selected data to return a computed result .
- Aggregation operations are expressions that can be used to produce reduced and summarized results in MongoDB.
- Aggregation operations can be performed using the aggregation pipeline, which is a framework that allows you to create a sequence of stages that process documents.
- Each stage in the aggregation pipeline performs a specific operation on the input documents, such as filtering, grouping, sorting, projecting, etc.
- The output documents of each stage are passed to the next stage as input, until the final result is produced.
- Aggregation pipelines can be used for various purposes, such as data analysis, data transformation, data enrichment, etc.
- Aggregation pipelines can be created using the `aggregate()` method,

which accepts one or more stage names as arguments.

- Some of the common aggregation stages are:
  - `$match`: filters the documents that match a specified condition.
  - `$group`: groups the documents by a specified expression and applies an accumulator function to each group.
  - `$sort`: sorts the documents by a specified order.
  - `$project`: modifies the documents by adding, removing, or renaming fields.
  - `$limit`: limits the number of documents to pass to the next stage.
  - `$skip`: skips a specified number of documents to pass to the next stage.
  - `$unwind`: deconstructs an array field from the input documents and outputs a document for each element.
  - `$lookup`: performs a left outer join with another collection and adds the joined documents to the input documents.
  - `$out`: writes the output documents to a specified collection.
- Aggregation pipelines can be nested, meaning that one pipeline can be used as the input for another pipeline.
- Aggregation pipelines can also be used to update documents in a collection, by using commands such as `updateMany()`, `updateOne()`, `replaceOne()`, etc.

Capped Collections in MongoDB

- Capped collections are fixed-size collections that support high-throughput operations that insert and retrieve documents based on insertion order .
- Capped collections work in a way similar to circular buffers: once a collection fills its allocated space, it makes room for new documents by overwriting the oldest documents in the collection .
- You must create capped collections explicitly using the `db.createCollection()` method, which is a mongosh helper for the create command .
- When creating a capped collection you must specify the maximum size of the collection in bytes, which MongoDB will pre-allocate for the collection .
- You can also specify the maximum number of documents that the capped collection can store, but this is optional.
- Capped collections have the following characteristics and limitations :
  - You cannot delete documents from a capped collection. To remove all documents from a capped collection, use the `db.collection.drop()` method to drop the collection and recreate the capped collection.
  - You cannot update documents in a way that increases their size. If you attempt to do so, MongoDB will return an error.
  - You can, however, update documents in a way that does not increase their size, such as changing the value of a field to a value of the same length.
  - You can use the `db.collection.remove()` method to remove the

last document inserted into a capped collection, but this is not recommended as it may impact the performance of the collection.
  - You cannot shard a capped collection.
  - You can create indexes on a capped collection, but you cannot create a unique index on a capped collection unless the index is on the `_id` field.
  - You can use the `db.collection.isCapped()` method to check if a collection is capped.
- Capped collections are typically used to store log information, high volume of data, and cache information . They are also useful for implementing a queue or a stream of data that only retains recent information.

Spark

- Spark is a word that has two distinct meanings, depending on whether it is used as a noun or a verb.
- As a noun, spark means a small fiery particle thrown off from a fire, alight in ashes, or produced by striking together two hard surfaces such as stone or metal .
- As a verb, spark means to emit sparks of fire or electricity, or to produce sparks at the point where an electric circuit is interrupted .
- Spark can also be used figuratively to mean a trace of a specified quality or intense feeling, a sense of liveliness and excitement, a lively young fellow, or to provide the stimulus for something, to ignite something, or to engage in courtship .
- Some examples of spark in a sentence are:
  - A log fire was sending sparks onto the rug.
  - Angry sparks were flashing in her eyes.
  - There was a spark of light.
  - A tiny spark of anger flared within her.
  - There was a spark between them at their first meeting.
  - The ignition sparks as soon as the gas is turned on.
  - The explosion sparked a fire.
  - The severity of the plan sparked off street protests.
  - He went a sparking among the rosy country girls.

Installing spark

Spark is an open-source distributed computing framework that can process large-scale data sets using in-memory caching and parallel processing. Spark can run on various platforms, such as Hadoop, Mesos, Kubernetes, standalone, or in the cloud. To install Spark, you need to follow these steps:

- Download the latest version of Spark from the official website: https://spark.apache.org/downloads.html. Choose the package type, the pre-built version, and the download type. You can also verify the integrity of the downloaded file using the provided checksums.
- Extract the downloaded file to a location of your choice. For example, you can use the following command on Linux or Mac OS to extract the file to /opt/spark:

```
tar xvf spark-3.2.0-bin-hadoop3.2.tgz -C /opt/spark
```

- Set the environment variables for Spark. You need to set the SPARK_HOME variable to point to the installation directory of Spark, and add the bin subdirectory to the PATH variable. For example, you can use the following commands on Linux or Mac OS to set the variables:

```
export SPARK_HOME=/opt/spark/spark-3.2.0-bin-hadoop3.2
export PATH=$PATH:$SPARK_HOME/bin
```

- Optionally, you can also set the PYSPARK_PYTHON variable to point to the Python executable that you want to use with Spark. For example, you can use the following command on Linux or Mac OS to set the variable:

```
export PYSPARK_PYTHON=/usr/bin/python3
```

- Test the installation by running the spark-shell or pyspark command. You should see a welcome message and a prompt to enter Spark commands. For example, you can use the following command on Linux or Mac OS to run the spark-shell:

```
spark-shell
```

- You can also run Spark applications using the spark-submit command. You need to provide the application name, the master URL, and any other options or arguments. For example, you can use the following command on Linux or Mac OS to run the wordcount example:

```
spark-submit --master local[4] examples/src/main/python/wordcount.py README.md
```

- To stop Spark, you can use the Ctrl+C keyboard shortcut or the exit() or quit() command in the shell. You can also use the stop() method on the SparkContext object in your application code. For example, you can use the following command on Linux or Mac OS to stop the spark-shell:

```
exit()
```

Spark Applications

- Spark applications are programs that use the Apache Spark framework to process large-scale data in parallel and distributed manner.

- Spark applications consist of a driver process and a set of executor processes that run on a cluster of nodes.
- The driver process runs the main function of the application, creates a Spark session, and coordinates the execution of tasks across the executors.
- The executors run the tasks assigned by the driver, store and cache data, and communicate with each other.
- Spark applications can be written in Scala, Java, Python, R, or C# and can use various libraries and APIs provided by Spark, such as Spark SQL, Spark Streaming, Spark MLlib, and Spark GraphX.
- Spark applications can run on different cluster managers, such as Apache Hadoop YARN, Apache Mesos, Kubernetes, or Spark's own standalone cluster manager.

- Spark applications can handle various types of data sources, such as files, databases, streaming data, or structured and semi-structured data.
- Spark applications can perform various types of data analysis, such as batch processing, interactive queries, machine learning, graph processing, and stream processing.

Jobs in Spark

Spark is a distributed computing framework that allows processing large-scale data in parallel using clusters of machines. Spark can run various types of applications, such as machine learning, streaming, graph analytics, and SQL queries. Spark jobs are the units of execution that perform the tasks defined by the user code.

Some of the main concepts related to Spark jobs are:

- **Application**: A Spark application is a user program that uses the Spark API to perform some data analysis or processing. An application can consist of one or more Spark jobs, depending on the number of actions invoked by the user code. An application runs on a Spark cluster, which can be local, standalone, YARN, Mesos, or Kubernetes.
- **Driver**: A driver is the process that runs the main() method of the Spark application and creates the SparkSession object. The driver coordinates the execution of Spark jobs and communicates with the cluster manager and the executors.
- **Executor**: An executor is a process that runs on a worker node in the cluster and executes the tasks assigned by the driver. An executor can run multiple tasks in parallel and can cache data in memory or disk for reuse. An executor is launched at the start of a Spark application and runs until the application terminates.
- **Job**: A Spark job is a parallel computation of tasks. Each action operation will create one Spark job. Each Spark job will be converted to a DAG (directed acyclic graph) which includes one or more stages.
- **Stage**: A stage is a set of tasks that can be executed in parallel. A stage is

created when the DAG of a Spark job encounters a shuffle operation, such as groupBy, reduceByKey, join, etc. A shuffle operation requires data to be redistributed across the cluster based on some partitioning scheme. A stage can have one or more tasks, depending on the number of partitions of the input data.
- **Task**: A task is the smallest unit of work in Spark. A task is a single execution of a user-defined function on a partition of the input data. A task can be a map task or a reduce task, depending on the type of operation it performs. A task runs on a single executor and can access the cached data on that executor.

Some of the sources of information for this response are:

- Submit Spark jobs in Azure Machine Learning (preview)
- What is the concept of application, job, stage and task in spark?
- Spark Basics - Application, Driver, Executor, Job, Stage and Task Walkthrough

Some of the examples of jobs in Spark are:

- Spark Engineer at Omega Solutions
- Program Lead at Spark Program
- Electrical Project Manager at Spark Power
- Inside Sales Representative at Spark Education
- Head of Biometrics at Spark Therapeutics

Stages and Tasks in Spark

- Spark is a distributed computing framework that executes parallel tasks on clusters of machines.
- Spark divides a job into smaller units of work called stages and tasks.
- A stage is a set of tasks that can be executed in parallel without data shuffling.
- A task is a unit of work that applies a transformation or an action to a partition of a dataset.
- The number of tasks in a stage equals the number of partitions in the input dataset.
- Spark creates a Directed Acyclic Graph (DAG) to represent the logical execution plan of a job.
- The DAG consists of nodes and edges, where nodes are RDDs (Resilient Distributed Datasets) and edges are transformations or actions.
- Spark splits the DAG into stages based on the shuffle boundaries, i.e. the operations that require data movement across the cluster.
- The first stage in a job is always a ShuffleMapStage, which prepares the data for subsequent stages by applying map and filter transformations.
- The last stage in a job is always a ResultStage, which performs the final action on the data, such as collect, count, save, etc.

- There can be intermediate stages between the first and the last stage, which are also ShuffleMapStages, but they perform shuffle operations to redistribute the data across the cluster.
- Spark assigns each stage a unique ID and a priority, and submits them to the DAGScheduler, which manages the execution of stages on the cluster.
- The DAGScheduler divides each stage into tasks and sends them to the TaskScheduler, which assigns them to the available executors on the worker nodes.
- The executors run the tasks and send the results back to the driver, which coordinates the whole process.
- Spark monitors the progress of the tasks and stages, and handles failures and retries if needed.
- Spark also optimizes the execution of the stages and tasks by applying techniques such as pipelining, caching, and lazy evaluation.

Resilient Distributed Databases in Spark

- Resilient Distributed Databases (RDDs) are the primary data structure in Spark .
- RDDs are immutable distributed collections of objects that can be operated on in parallel .
- RDDs can contain any type of Python, Java, or Scala objects, including user-defined classes .
- RDDs are reliable and memory-efficient when it comes to parallel processing .
- RDDs support two types of operations: transformations and actions .
- Transformations create a new RDD from an existing one, such as map, filter, join, etc .
- Actions return a value to the driver program or write data to an external storage system, such as count, collect, save, etc .
- RDDs are lazy evaluated, meaning that they are only computed when an action is performed on them .
- RDDs can be created from any storage source supported by Hadoop, such as local file system, HDFS, Cassandra, HBase, Amazon S3, etc.
- RDDs can also be created from an existing collection in the driver program using the parallelize method .
- RDDs can be cached or persisted in memory or disk for faster reuse .
- RDDs can be checkpointed to a reliable storage system to recover from failures .
- RDDs are resilient because they can be reconstructed from lineage information (the sequence of transformations that produced them) or from checkpoints .
- RDDs are distributed because they are partitioned across nodes in the cluster and can be computed in parallel .

Anatomy of a Spark job run

- A Spark job is a unit of execution that corresponds to an action on a Spark application, such as `collect()` or `saveAsTextFile()`.
- A Spark job consists of one or more stages, which are parallel computations that operate on a subset of the data.
- A stage is divided into tasks, which are the smallest unit of execution that run on a single executor (a process that runs on a worker node).
- A task applies a transformation or an action to a partition of an RDD (a distributed collection of data).
- A Spark job is executed by the Spark scheduler, which creates a directed acyclic graph (DAG) of stages and tasks based on the dependencies between RDDs.
- The Spark scheduler submits the stages and tasks to the cluster manager, which allocates resources (such as CPU cores and memory) to the executors.
- The executors run the tasks and send the results back to the driver (the process that runs the Spark application).
- The driver collects the results and returns them to the user or writes them to a storage system.

Spark on YARN

- Spark is a distributed computing framework that can run on various cluster managers, such as YARN, Mesos, Kubernetes, or standalone mode.
- YARN (Yet Another Resource Negotiator) is a cluster manager that is part of the Hadoop ecosystem and can manage the resources and scheduling of various applications running on a Hadoop cluster.
- Running Spark on YARN allows Spark to leverage the existing features and capabilities of YARN, such as security, resource isolation, scalability, and high availability.
- Running Spark on YARN requires a binary distribution of Spark which is built with YARN support. Binary distributions can be downloaded from the downloads page of the project website. To build Spark yourself, refer to Building Spark .
- There are two variants of Spark binary distributions you can download. One is pre-built with a certain version of Hadoop, which is recommended if you are using the corresponding version of Hadoop in your cluster. The other is pre-built with user-provided Hadoop, which allows you to use Spark with any Hadoop version by specifying the HADOOP_CONF_DIR environment variable.
- There are two deploy modes that can be used to launch Spark applications on YARN. In cluster mode, the Spark driver runs inside an application master process which is managed by YARN on the cluster, and the client can go away after initiating the application. In client mode, the Spark driver runs in the client process, and the application master is only used for requesting resources from YARN.
- To run Spark on YARN, you need to configure some Spark and YARN

properties, such as the number of executors, the amount of memory and cores per executor, the location of the Spark assembly jar, and the application name. You can use the spark-submit script to submit your Spark applications to YARN, with the –master option set to yarn and the –deploy-mode option set to either cluster or client .
- Running Spark on YARN also enables you to use dynamic allocation, which allows Spark to request or release executors based on the workload. This can improve the resource utilization and performance of your Spark applications. To enable dynamic allocation, you need to set spark.dynamicAllocation.enabled to true and configure the shuffle service on each node in the cluster .

SCALA

- Scala is a **general-purpose**, **high-level**, **multi-paradigm** programming language that supports both **object-oriented** and **functional** programming .
- Scala is **strongly** and **statically** typed, which means that the type of every variable and expression is checked at compile time and cannot be changed at run time .
- Scala is designed to be **concise** and **expressive**, avoiding boilerplate code and allowing programmers to write less code for the same functionality .
- Scala runs on the **Java Virtual Machine (JVM)** and can interoperate with Java code and libraries. It also has a **JavaScript** runtime that allows Scala code to run on web browsers and other JavaScript platforms .
- Scala has many features that make it a powerful and versatile language, such as:
  - **Case classes** and **pattern matching**, which enable concise and elegant data structures and algorithms .
  - **Traits** and **mixins**, which allow multiple inheritance and code reuse .
  - **Higher-order functions** and **closures**, which enable functional programming and abstraction .
  - **Implicits** and **type classes**, which enable ad-hoc polymorphism and extension methods .
  - **Lazy evaluation** and **streams**, which enable efficient and expressive computation of infinite and large data structures .
  - **Macros** and **metaprogramming**, which enable code generation and manipulation at compile time .
- Scala is an **evolving** language that has undergone several major versions since its inception in 2004. The latest version is **Scala 3**, which introduces many new features and improvements, such as:
  - **Union types** and **intersection types**, which enable more precise and flexible type definitions .
  - **Enums** and **algebraic data types**, which enable more concise and safe representation of data .

- **Opaque types** and **type lambdas**, which enable more powerful and modular abstraction .
- **Top-level definitions** and **export clauses**, which enable more flexible and modular code organization .
- **Given instances** and **using clauses**, which replace implicits and simplify the syntax and semantics of context parameters .
- **Extension methods** and **contextual abstractions**, which enable more natural and expressive extension of existing types and libraries .
- **Optional braces** and **significant indentation**, which enable more concise and readable syntax .

Introduction to Scala

- Scala is a general-purpose, multi-paradigm programming language that integrates features of object-oriented and functional programming.
- Scala runs on the Java Virtual Machine (JVM) and is compatible with existing Java code and libraries.
- Scala was designed by Martin Odersky and first released in 2004. The name Scala stands for "scalable language", as it is intended to grow with the demands of its users.
- Scala supports multiple programming styles, such as imperative, declarative, concurrent, and reactive. It also supports domain-specific languages (DSLs) and meta-programming.
- Scala has a concise and expressive syntax that allows programmers to write less code and avoid boilerplate. It also has a powerful type system that supports static typing, type inference, generics, and algebraic data types.
- Scala has many features that facilitate functional programming, such as higher-order functions, immutable data structures, pattern matching, lazy evaluation, and monads.
- Scala also supports object-oriented programming, with classes, traits, inheritance, and multiple dispatch. Scala allows multiple inheritance through the use of traits, which are similar to interfaces but can also contain concrete methods and fields.
- Scala has a rich collection library that provides many useful data structures and operations, such as lists, sets, maps, streams, futures, and options. Scala also has a standard library that provides modules for common tasks, such as testing, parsing, concurrency, and networking.
- Scala is a compiled language that produces Java bytecode that can run on any JVM. Scala also has an interactive interpreter (REPL) that allows programmers to execute Scala code interactively and experiment with different features.
- Scala is a popular choice for developing scalable and concurrent applications, such as web services, big data processing, and distributed systems. Scala is used by many companies and organizations, such as Twitter,

LinkedIn, Netflix, and Coursera.

Classes and Objects in Scala

- A class is a blueprint for creating objects. It defines the state and behavior of the objects of that class.
- An object is an instance of a class. It has a unique identity and can access the members (fields and methods) of its class.
- Scala supports both object-oriented and functional programming paradigms. It allows defining classes and objects as well as functions and values.
- To define a class in Scala, use the `class` keyword followed by the name of the class and an optional parameter list. For example:

```
class Person(name: String, age: Int) {
  // class body
}
```

- To create an object of a class, use the `new` keyword followed by the name of the class and an optional argument list. For example:

```
val p = new Person("Alice", 20) // p is an object of Person class
```

- Scala also supports the concept of singleton objects, which are objects that are declared and initialized at the same time. To define a singleton object, use the `object` keyword followed by the name of the object. For example:

```
object Math {
  // object body
}
```

- A singleton object can be accessed by its name directly, without using the `new` keyword. For example:

```
Math.PI // access the PI value defined in Math object
```

- A singleton object can also be used to define static members (fields and methods) that belong to the class with the same name. This is called a companion object. For example:

```
class Circle(radius: Double) {
  // class body
}

object Circle {
  // object body
  def area(r: Double): Double = Math.PI * r * r // static method to calculate area of a cir
}
```

- A companion object can access the private members of its companion class and vice versa. For example:

```scala
class Circle(radius: Double) {
  private val r = radius // private field
  def getRadius: Double = r // public method to access private field
}

object Circle {
  def printRadius(c: Circle): Unit = println(c.r) // object method can access private field
}
```

Basic Types and Operators in Scala

- Scala has a rich set of basic types, including numeric, character, string, and boolean types.
- Scala also supports operator overloading, which means that operators are treated as methods of their operands.
- Scala has a uniform syntax for operators and methods, which means that any method can be used as an infix operator, and any operator can be used as a method.

Numeric Types

- Scala has eight numeric types: Byte, Short, Int, Long, Float, Double, BigInt, and BigDecimal.
- Byte, Short, Int, and Long are signed integer types with different ranges of values.
- Float and Double are floating-point types with different precisions.
- BigInt and BigDecimal are arbitrary-precision types that can represent very large or very small numbers with exactness.
- Scala supports the usual arithmetic operators (+, -, *, /, %) on numeric types, as well as some bitwise operators (~, &, |, ^, <<, >>, >>>) on integer types.
- Scala also provides some useful methods on numeric types, such as abs, max, min, round, toInt, toDouble, etc.

Character and String Types

- Scala has two character types: Char and String.
- Char represents a single Unicode character, such as 'a' or '\u0041'.
- String represents a sequence of characters, such as "Hello" or "Scala".
- Scala supports the + operator on strings, which concatenates two strings together.
- Scala also provides some useful methods on strings, such as length, substring, trim, toUpperCase, toLowerCase, split, replace, etc.

Boolean Type

- Scala has a boolean type, Boolean, which can have two values: true or false.
- Scala supports the logical operators (&&, ||, !) on boolean values, as well as the relational operators (==, !=, <, >, <=, >=) on any types that support comparison.
- Scala also provides some useful methods on boolean values, such as toInt, toString, etc.

Built-in control structures in Scala

- Scala has only a few built-in control structures, such as `if`, `while`, `for`, `try`, `match`, and function calls .
- The reason Scala has so few is that it has included function literals since its inception . A function literal is a function that is not defined by a name, but by its parameters and body, such as `(a:Int, b:Int) => a + b`.
- Scala's control structures are closer to the functional style, which means they are expressions that return a value, rather than statements that perform side effects .
- Some of the unique features of Scala's control structures are:
  - `if` expressions can be used as ternary operators, such as `val max = if (a > b) a else b` .
  - `for` loops can be used with ranges, collections, or generators, such as `for (i <- 1 to 10) println(i)` or `for (x <- xs if x % 2 == 0) yield x * 2` .
  - `match` expressions can be used for pattern matching, such as `x match { case 0 => "zero"; case 1 => "one"; case _ => "other" }` .
  - `try` expressions can be used for exception handling, such as `try { doSomething() } catch { case e: Exception => handle(e) } finally { cleanUp() }` .
- Scala's control structures are designed to be concise, expressive, and consistent .

Functions and Closures in Scala

- A function is a piece of code that takes some input, performs some computation, and returns some output.
- A function can be defined using the `def` keyword, followed by the name of the function, a list of parameters, an optional return type, and a body enclosed in curly braces.
- A function can be invoked by using its name and passing the arguments that match the parameters.
- A function can be assigned to a variable or passed as an argument to another function. This is possible because functions are values in Scala, and they have a type called `FunctionN`, where `N` is the number of parameters.

- A function can be anonymous, meaning that it does not have a name. An anonymous function can be defined using the `=>` symbol, which separates the parameters from the body. An anonymous function can be assigned to a variable or passed as an argument to another function.
- A closure is a function that can access variables from its enclosing scope, even if they are not passed as parameters. A closure captures the values of those variables at the time of its creation, and can use them in its body.
- A closure can be useful for creating functions that are customized to a specific context, such as filtering a list based on some criteria, or performing some action on each element of a collection.

Inheritance in Scala

- Inheritance is a mechanism that allows a class to inherit the features and behavior of another class.
- The class that inherits is called the **subclass** or the **derived class**.
- The class that is inherited is called the **superclass** or the **base class**.
- In Scala, inheritance is achieved by using the `extends` keyword.
- A subclass can override the methods and fields of the superclass by using the `override` keyword.
- A subclass can also access the methods and fields of the superclass by using the `super` keyword.
- Scala supports **single inheritance**, which means that a class can only extend one superclass.
- Scala also supports **multiple inheritance** through **traits**, which are abstract types that can contain methods and fields.
- A class can mix in multiple traits by using the `with` keyword.
- Traits can also extend other traits or classes, forming a linear hierarchy called the **linearization** of the class.
- The order of the traits in the linearization determines the method resolution and the super calls.

Hadoop Eco System Frameworks , Pig , Hive and HBase

- Hadoop is an open-source framework that allows distributed processing of large-scale data sets across clusters of computers using simple programming models.
- Hadoop consists of two main components: Hadoop Distributed File System (HDFS) and MapReduce.
- HDFS is a distributed file system that provides high-throughput access to data and fault tolerance.
- MapReduce is a programming model that enables parallel processing of large data sets using key-value pairs.
- Hadoop also includes several additional modules that provide additional functionality, such as Pig, Hive and HBase .

- Pig is a high-level platform for creating MapReduce programs using a scripting language called Pig Latin.
- Pig helps to achieve ease of programming and optimization and hence is a major segment of the Hadoop Ecosystem.
- Hive is a data warehouse infrastructure that provides data summarization and ad-hoc querying using a SQL-like query language called HiveQL.
- Hive performs reading and writing of large data sets using HDFS and MapReduce .
- HBase is a distributed column-oriented database that supports structured data storage for large tables.
- HBase is a data model that is similar to Google's big table that designed to provide quick random access to huge amounts of structured data .
- HBase is built on top of HDFS and provides scalability and consistency guarantees.

Hadoop Eco System Frameworks

- Hadoop is an open-source software framework that allows for the distributed processing of large data sets across clusters of commodity hardware using simple programming models.
- Hadoop consists of four core components: Hadoop Distributed File System (HDFS), MapReduce, YARN, and Hadoop Common.
- HDFS is a distributed file system that provides high-throughput access to application data and can store large amounts of data across multiple nodes.
- MapReduce is a programming model and software framework for writing applications that process large amounts of data in parallel on clusters of nodes.
- YARN is a resource management layer that allocates computing resources to applications and schedules tasks on the cluster.
- Hadoop Common is a set of common utilities and libraries that support other Hadoop modules.
- Hadoop Ecosystem is a collection of tools, libraries, and frameworks that help you build applications on top of Hadoop. Some of the popular Hadoop Ecosystem components are :
  - Apache Pig: A high-level scripting language for data analysis and transformation.
  - Apache Hive: A data warehouse system for querying and analyzing structured and semi-structured data using SQL-like syntax.
  - Apache HBase: A column-oriented database that provides random access and strong consistency for large-scale data.
  - Apache Spark: A fast and general engine for large-scale data processing, supporting batch, streaming, and interactive queries.
  - Apache Kafka: A distributed messaging system that enables high-throughput and low-latency data ingestion and processing.
  - Apache Flume: A service for collecting, aggregating, and moving

large amounts of log data from various sources to HDFS or other destinations.
  - Apache Sqoop: A tool for transferring bulk data between Hadoop and relational databases.
  - Apache Oozie: A workflow scheduler that manages and coordinates Hadoop jobs.
  - Apache ZooKeeper: A service for maintaining configuration information, naming, and providing distributed synchronization and group services.
  - Apache Mahout: A library of scalable machine learning algorithms for data mining and analytics.

Applications of Big Data using Pig

- Apache Pig is a platform or tool that provides a high-level language called Pig Latin for processing large datasets on top of Hadoop and MapReduce.
- Pig Latin is a scripting language that allows users to write complex data transformations and analysis using simple commands.
- Pig can handle both structured and unstructured data, and can perform various operations such as filtering, grouping, joining, sorting, aggregating, and more.
- Some of the applications of big data using Pig are:
  - Exploring large datasets: Pig can be used to quickly and easily explore large datasets, such as web logs, social media data, sensor data, etc., by writing simple Pig scripts .
  - Ad-hoc queries: Pig can also support ad-hoc queries across large datasets, such as finding the top 10 most visited pages, the average duration of a session, the number of unique visitors, etc., by using built-in functions or user-defined functions .
  - Prototyping of algorithms: Pig can be used to prototype and test new algorithms for processing large datasets, such as machine learning, text analysis, graph processing, etc., by using Pig Latin or embedding other languages such as Python, Java, etc. in Pig scripts .
  - Time-sensitive data loads: Pig can process time-sensitive data loads, such as streaming data, real-time data, or batch data, by using different modes of execution, such as local mode, mapreduce mode, or tez mode.
  - De-identification of data: Pig can be used by telecom companies or other organizations to de-identify or anonymize sensitive customer data, such as call records, location data, personal information, etc., by using encryption, hashing, masking, or other techniques.
  - Analytical insights: Pig can be used to generate analytical insights from large datasets, such as customer behavior, market trends, product recommendations, etc., by using sampling, statistics, or other

methods.

Applications of Big Data using Hive

- Hive is a data warehouse system that allows users to analyze large volumes of structured and semi-structured data using SQL-like queries.
- Hive can run on top of Hadoop, a distributed framework that stores and processes big data across multiple nodes in a cluster.
- Hive can also integrate with other big data tools such as Spark, Pig, and HBase to perform various data analysis tasks.
- Some of the applications of big data using Hive are:
  - Data exploration and discovery: Hive can be used to explore and discover patterns, trends, and insights from large and complex data sets. For example, Hive can be used to analyze web logs, social media data, sensor data, etc.
  - Data warehousing and reporting: Hive can be used to create and manage data warehouses that store and organize data from different sources. Hive can also generate reports and dashboards that summarize and visualize the data. For example, Hive can be used to create data warehouses for e-commerce, banking, retail, etc.
  - Data mining and machine learning: Hive can be used to perform data mining and machine learning tasks such as classification, clustering, regression, recommendation, etc. Hive can leverage the distributed computing power of Hadoop and the libraries of Spark and Mahout to process large-scale data. For example, Hive can be used to build recommender systems, fraud detection systems, sentiment analysis systems, etc.
  - Data integration and transformation: Hive can be used to integrate and transform data from different sources and formats. Hive can handle structured, semi-structured, and unstructured data such as CSV, JSON, XML, etc. Hive can also perform data cleansing, filtering, aggregation, joining, etc. For example, Hive can be used to integrate and transform data from various web services, databases, files, etc.

Applications on Big Data using HBase

- HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS) .
- HBase provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases .
- HBase is well suited for real-time data processing or random read/write access to large volumes of data .
- HBase works well with Hive, a query engine for batch processing of big data, to enable fault-tolerant big data applications .

- HBase hosts very large tables on top of clusters of commodity hardware .
- HBase is modeled after Google's Bigtable, which acts upon Google File System .
- Some of the applications of HBase are:
  - In the healthcare sector, HBase is used for storing genome sequences and disease history of people or a particular area .
  - In the field of e-commerce, HBase is used for storing logs about customer search history and it also performs analytics and target advertisement for better business insights .
  - In sports, HBase is used to store match details and the history of each match .
  - In social media, HBase is used to store user profiles, messages, comments, likes, and other data .
  - In finance, HBase is used to store transaction records, stock market data, and fraud detection data .

Pig

- A pig is a mammal that belongs to the order Artiodactyla, which means it has an even number of toes on each foot.
- There are over one billion pigs on Earth, and they are found and raised all over the world .
- Pigs are also known as hogs or swine. Male pigs of any age are called boars; female pigs are called sows.
- Pigs are omnivorous, which means they eat both plants and animals. Pigs will eat almost anything, including feces.
- Pigs provide valuable products to humans, including pork, lard, leather, glue, fertilizer, and a variety of medicines.
- Pigs have the intelligence of a human toddler and are ranked as the fifth most intelligent animal in the world. They can learn their names, come when they are called, and play video games better than some primates.
- Pigs have a keen sense of smell and can detect odors up to seven miles away and 20 feet underground. They use their snouts for digging and finding food.
- Pigs do not have many sweat glands, so they do not sweat much. They cool themselves by wallowing in mud or water .
- Pigs have four toes on each foot, but only use two of them for walking. The other two are called trotters and are used for balance.
- Pigs are social animals and form strong bonds with each other. They communicate with a range of sounds, from grunts to squeals .

Pig - Introduction to PIG

- Pigs are mammals in the genus Sus, belonging to the family Suidae, which includes other even-toed ungulates such as cattle, sheep, and goats.
- Pigs are also known as hogs or swine, and are domesticated from wild

boars that inhabit Eurasia and Africa .
- Pigs are omnivorous animals that eat a variety of plant and animal matter, and have a well-developed sense of smell.
- Pigs are social and intelligent animals that can learn and perform various tasks, such as opening doors, playing video games, and recognizing themselves in mirrors.
- Pigs have a lifespan of about 8 years, and are among the most populous mammals on earth, with about one billion pigs alive at any given time.
- Pigs provide valuable products to humans, such as pork, lard, leather, glue, fertilizer, and medicines.

Execution Modes of Pig

Apache Pig is a high-level platform for analyzing large data sets using a scripting language called Pig Latin. Pig can run on a single machine or on a cluster of machines using Hadoop. Pig has different execution modes or types depending on the environment and the data source. These are:

- **Local mode**: In this mode, Pig runs on a single machine using the local file system. No Hadoop installation is required. This mode is useful for testing and debugging purposes. To run Pig in local mode, use the `-x local` option in the command line or the `pig -x local` command in the Grunt shell.
- **MapReduce mode**: In this mode, Pig runs on a Hadoop cluster using the Hadoop Distributed File System (HDFS). Hadoop installation and configuration are required. This mode is suitable for processing large-scale data sets in parallel. To run Pig in MapReduce mode, use the `-x mapreduce` option in the command line or the `pig -x mapreduce` command in the Grunt shell.
- **Tez mode**: In this mode, Pig runs on a Hadoop cluster using the Tez execution engine. Tez is a framework for building high-performance batch and interactive data processing applications on Hadoop. Tez optimizes the execution plan of Pig scripts by minimizing data movement and reducing the number of MapReduce jobs. To run Pig in Tez mode, use the `-x tez` option in the command line or the `pig -x tez` command in the Grunt shell.
- **Tez local mode**: In this mode, Pig runs on a single machine using the Tez execution engine and the local file system. This mode is similar to the local mode, but with the benefits of Tez optimization. To run Pig in Tez local mode, use the `-x tez_local` option in the command line or the `pig -x tez_local` command in the Grunt shell.
- **Spark mode**: In this mode, Pig runs on a Hadoop cluster using the Spark execution engine. Spark is a fast and general engine for large-scale data processing, supporting in-memory computation and streaming. Spark can improve the performance of Pig scripts by caching intermediate data in memory and reducing disk I/O. To run Pig in Spark mode, use the `-x`

`spark` option in the command line or the `pig -x spark` command in the Grunt shell.

- **Embedded mode**: In this mode, Pig can be embedded in a Java application or a user-defined function (UDF). This mode allows the user to extend the functionality of Pig by writing custom code in Java, Python, Ruby, or other languages. To run Pig in embedded mode, use the `PigServer` class in Java or the `piggybank` module in Python.

Comparison of Pig with Databases

- Pig is a high-level data-flow language and execution framework for parallel computation on large-scale data sets, while databases are systems that store, organize, and manipulate structured or semi-structured data using a query language, such as SQL.
- Pig can process unstructured or complex data formats, such as JSON, XML, or logs, while databases usually require a predefined schema or data model to operate on the data.
- Pig can run on top of Hadoop, a distributed file system and a platform for MapReduce programming, which enables parallel processing and scalability, while databases may not support these features or may require additional tools or configurations to do so.
- Pig can be used to construct dataflows, run scheduled jobs, crunch big volumes of data, aggregate or summarize it, and store it into relational database systems or other formats, while databases can be used for data storage, retrieval, analysis, and manipulation using SQL or other query languages.
- Pig can be extended using user-defined functions (UDFs) written in Java, Python, JavaScript, Ruby, or Groovy, while databases may have limited support for UDFs or may require specific languages or interfaces to create them.
- Pig has a simple and expressive syntax, called Pig Latin, which resembles SQL in some aspects, but also allows for more complex operations and transformations, while databases have different query languages, such as SQL, NoSQL, or graph query languages, which may vary in their syntax, features, and limitations.

Grunt in Pig

- A grunt is a vocalization made by a pig, usually to communicate with other pigs or humans.
- A grunt can convey different meanings depending on the context, tone, pitch, and duration of the sound.
- Some possible meanings of a grunt are:
  - A low-pitched, long grunt can indicate contentment, relaxation, or satisfaction.

  - A high-pitched, short grunt can indicate excitement, curiosity, or greeting.
  - A loud, harsh grunt can indicate aggression, warning, or distress.
  - A soft, rhythmic grunt can indicate hunger, nursing, or suckling.
  - A series of grunts can indicate communication, socialization, or coordination.
- A grunt can also be influenced by the personality, mood, and health of the pig, as well as the environment and situation they are in.
- A grunt can help humans understand the needs, emotions, and preferences of their pigs, and respond accordingly.

Pig Latin

- Pig Latin is a language game or argot in which words in English are altered, usually by adding a suffix or by moving the onset or initial consonant or consonant cluster of a word to the end of the word and adding a vocalic syllable to create such a suffix.
- The objective is to conceal the meaning of the words from others not familiar with the rules.
- The reference to Latin is a deliberate misnomer, as it is simply a form of jargon, used only for its English connotations as a strange and foreign-sounding language.
- Some examples of Pig Latin are:
  - "Pig" becomes "igpay"
  - "Latin" becomes "atinlay"
  - "Banana" becomes "ananabay"
  - "Happy" becomes "appyhay"
  - "Duck" becomes "uckday"
  - "Hello" becomes "ellohay"
  - "Smile" becomes "ilesmay"
- The rules for forming Pig Latin words are:
  - For words that begin with consonant sounds, all letters before the initial vowel are placed at the end of the word sequence. Then, "ay" is added, as in the following examples:
    - "Pig" becomes "igpay"
    - "Trash" becomes "ashtray"
    - "Glove" becomes "oveglay"
    - "Happy" becomes "appyhay"
  - For words that begin with vowel sounds or silent letter, one just adds "way" or "yay" to the end (or just "ay"). Examples are:

    * “Egg” becomes “eggway” or “eggyay”
    * “Inbox” becomes “inboxway” or “inboxyay”
    * “Eight” becomes “eightway” or “eightyay”
    * “Honor” becomes “onorway” or “onoryay”

- For words that end in “y”, such as “happy”, there are two variants: the y may be left unchanged, as in “appyhay”, or it may be changed to “i”, as in “appihay”.

- For words that contain a “qu” sound, the “u” is treated as a consonant, and the “qu” sound is moved to the end of the word before adding “ay”. Examples are:

    * “Quiet” becomes “ietquay”
    * “Squeal” becomes “ealsquay”
    * “Quail” becomes “ailquay”

- For words that contain a “th” sound, the “th” is treated as a single consonant, and the “th” sound is moved to the end of the word before adding “ay”. Examples are:

    * “Think” becomes “inkthay”
    * “Bath” becomes “athbay”
    * “The” becomes “ethay”

- For words that contain a “ch” sound, the “ch” is treated as a single consonant, and the “ch” sound is moved to the end of the word before adding “ay”. Examples are:

    * “Chair” becomes “airchay”
    * “Chew” becomes “ewchay”
    * “Church” becomes “urchchay”

- For words that contain a “sh” sound, the “sh” is treated as a single consonant, and the “sh” sound is moved to the end of the word before adding “ay”. Examples are:

    * “Shoe” becomes “oeshay”
    * “Shark” becomes “arkshay”
    * “Sheep” becomes “eepshay”

- For words that contain a “ph” sound, the “ph” is treated as a single consonant, and the “ph” sound is moved to the end of the word before adding “ay”. Examples are:

    * “Phone” becomes “onephay”
    * “Photo” becomes “otophay”
    * “Phantom” becomes “antomphay”

- For words that contain a “ck” sound, the “ck” is treated as a single consonant, and the “ck” sound is moved to the end of the word before adding “ay”. Examples are:

    - “Duck” becomes “uckday”
    - “Truck” becomes “ucktray”
    - “Black” becomes “ackblay”
  - For words that contain a “ng” sound, the “ng” is treated as a single consonant, and the “ng” sound is moved to the end of the word before adding “ay”. Examples are

User Defined Functions in Pig

- User defined functions (UDFs) are custom functions that can be written in Java, Python, or other languages and used in Pig scripts to perform specific tasks that are not supported by the built-in functions.
- UDFs can be used to manipulate data, perform complex calculations, call external services, or implement custom logic that is not possible with the existing Pig operators and functions.
- UDFs can be classified into four types based on their input and output: Eval functions, Filter functions, Load/Store functions, and Aggregate functions.
- Eval functions take one or more input values and return a single output value. For example, a UDF that converts a string to uppercase or a UDF that calculates the distance between two points.
- Filter functions take a single input value and return a boolean value indicating whether the input satisfies a certain condition. For example, a UDF that checks if a string contains a specific word or a UDF that filters out records based on some criteria.
- Load/Store functions are used to read data from or write data to external sources, such as files, databases, or web services. For example, a UDF that loads data from a JSON file or a UDF that stores data to a MongoDB collection.
- Aggregate functions take a bag of values as input and return a single output value that summarizes the input. For example, a UDF that computes the average, median, or standard deviation of a set of numbers or a UDF that concatenates a set of strings.
- To write a UDF in Java, one needs to extend the appropriate abstract class from the org.apache.pig.EvalFunc, org.apache.pig.FilterFunc, org.apache.pig.LoadFunc, or org.apache.pig.Algebraic interface, depending on the type of the UDF, and implement the required methods.
- To write a UDF in Python, one needs to use the @outputSchema decorator to specify the output schema of the UDF and define a function that takes the input arguments and returns the output value.

- To use a UDF in a Pig script, one needs to register the UDF jar file or Python script using the REGISTER statement and then invoke the UDF by its name and pass the required arguments. For example, REGISTER myudfs.jar; A = LOAD 'data.txt' AS (name:chararray, age:int); B = FOREACH A GENERATE myudfs.ToUpper(name), myudfs.AgeGroup(age);

Data Processing Operators in Pig

- Data processing operators are the main tools that Pig Latin provides to operate on the data.
- They allow you to transform the data by sorting, grouping, joining, projecting, and filtering.
- A Pig Latin statement is an operator that takes a relation as input and produces another relation as output .
- There are different types of data processing operators in Pig, such as:
  - Relational operators: These operators perform basic operations on relations, such as loading, storing, filtering, grouping, joining, etc. For example, LOAD, STORE, FILTER, GROUP, JOIN, etc.
  - Evaluation operators: These operators evaluate or manipulate the values in tuples or bags, such as arithmetic, string, or date operations. For example, +, -, *, /, CONCAT, SUBSTRING, ToDate, etc.
  - Diagnostic operators: These operators help in debugging or testing the Pig scripts, such as printing the schema or the data of a relation. For example, DESCRIBE, DUMP, EXPLAIN, ILLUSTRATE, etc.
  - Miscellaneous operators: These operators perform some special functions, such as defining aliases, macros, or user-defined functions. For example, DEFINE, IMPORT, REGISTER, etc.

Hive

- Hive is a data warehouse software that facilitates querying and managing large datasets residing in distributed storage.
- Hive provides a SQL-like interface called HiveQL to access structured and semi-structured data in various formats.
- Hive allows users to project structure on largely unstructured data and perform analytical operations on it.
- Hive is built on top of Apache Hadoop and supports storage on S3, ADLS, GS, etc. through HDFS.
- Hive can also integrate with other data processing tools such as Spark, Pig, and MapReduce.
- Hive has a central component called Hive Metastore (HMS) that stores the metadata of tables, partitions, columns, etc. in a relational database.
- Hive can also support user-defined functions (UDFs), user-defined aggregate functions (UDAFs), and user-defined table functions (UDTFs) to extend its functionality.

- Hive can run in different modes such as local, standalone, embedded, or remote.
- Hive can also support different execution engines such as Tez, Spark, or MapReduce.
- Hive can also support different file formats such as ORC, Parquet, Avro, JSON, etc. and different compression codecs such as Snappy, Zlib, etc..

Apache Hive architecture

Apache Hive is a data warehouse system that enables analytics at a massive scale on top of Hadoop. It provides a SQL-like query language called HiveQL that can process structured and semi-structured data. Hive also supports user-defined functions and custom data formats.

The main components of Apache Hive architecture are:

- **Hive clients**: These are the interfaces that allow users and applications to interact with Hive. They include the Hive command line (CLI), the Hive web interface (HWI), the Hive Beeline shell, and the Hive JDBC/ODBC drivers.
- **Hive services**: These are the components that process the queries and manage the metadata. They include the Hive server 2 (HS2), the Hive metastore (HMS), and the Hive compiler and execution engine.
- **Processing framework and resource management**: These are the components that handle the distributed processing and scheduling of the queries. They include the Hadoop MapReduce or Tez framework, and the YARN or Mesos resource manager.
- **Distributed storage**: This is the component that stores the data and metadata. It includes the Hadoop Distributed File System (HDFS) or other compatible file systems.

Installing Hive

- Hive is a data warehouse system that runs on top of Hadoop, a distributed file system that stores large amounts of data.
- Hive provides a SQL-like interface to query and analyze data stored in Hadoop.
- To install Hive, you need to have Hadoop installed and configured first.
- You can download the latest version of Hive from https://hive.apache.org/downloads.html
- You can extract the downloaded file to a desired location, such as /usr/local/hive
- You need to set some environment variables to use Hive, such as HIVE_HOME, HADOOP_HOME, and PATH
- You can edit the hive-site.xml file in the conf directory to customize the Hive configuration, such as the metastore location, the default database, and the Hive execution engine

- You can start the Hive shell by running the command hive in the bin directory
- You can use the Hive shell to create databases, tables, and partitions, and to run queries on the data stored in Hadoop
- You can also use other tools to interact with Hive, such as JDBC, ODBC, or HiveServer2

Hive shell

- Hive shell is a command-line interface that allows users to interact with Hive and run Hive queries .
- Hive shell can be launched by typing `$HIVE_HOME/bin/hive` in the terminal, where `$HIVE_HOME` is the environment variable that points to the Hive installation directory.
- Hive shell supports both interactive and batch modes. In interactive mode, users can type Hive queries and get the results on the screen. In batch mode, users can execute a script file that contains Hive queries by using the `-f` option.
- Hive shell also supports variables that can be used to pass parameters to Hive queries. Variables can be set by using the `set` command with the `hivevar` prefix, such as `set hivevar:tablename=mytable;`. Variables can be referenced by using the `${}` syntax, such as `select * from ${tablename};`.
- Hive shell provides some useful commands to manage the Hive session, such as `show databases;`, `use database;`, `show tables;`, `describe table;`, `quit;`, etc.
- Hive shell can also be used to access Hiveon OS workers remotely using the Hiveon OS network infrastructure. Hiveon OS is a Linux-based operating system for mining cryptocurrencies. Hive shell offers some unique features, such as access via an SSH client and console sharing.

Hive services

Hive services are the components that enable users to interact with Hive and perform various operations on data stored in Hive tables. Hive services include:

- **HiveServer2**: This is the main service that provides a JDBC/ODBC interface for clients to execute queries and access the metadata. HiveServer2 supports multiple concurrent users and sessions, and provides security features such as authentication and authorization.
- **Hive Metastore**: This is the service that stores the metadata of Hive tables, partitions, columns, and other objects. The metadata is stored in a relational database such as MySQL or PostgreSQL, and can be accessed by HiveServer2 and other Hive components. The Hive Metastore also provides a thrift interface for external tools and applications to interact with the metadata.

- **Hive Web Interface (HWI)**: This is a web-based graphical user interface that allows users to browse the Hive metadata, submit queries, and view the results. HWI is deprecated and replaced by Hive View in Ambari.
- **Hive CLI**: This is a command-line interface that allows users to execute Hive commands and queries. Hive CLI is mainly used for debugging and testing purposes, and is not recommended for production use. Hive CLI does not support concurrent users or sessions, and does not provide any security features.

Hive metastore

- Hive metastore is a central repository that stores metadata for Hive tables and partitions.
- Hive metastore provides a service that allows other applications to access the Hive metadata using a Thrift API or a JDBC connection.
- Hive metastore can be configured to use different back-end databases, such as Derby, MySQL, PostgreSQL, Oracle, etc.
- Hive metastore consists of two components: a metastore server and a metastore database.
- The metastore server is a Java process that runs on a separate machine from the Hive server and communicates with the metastore database using JDBC.
- The metastore database is a relational database that stores the schema and location of Hive tables and partitions, as well as other information such as statistics, privileges, etc.
- Hive metastore supports two modes of operation: embedded and remote.
- In embedded mode, the metastore server and the metastore database run on the same machine as the Hive server, and the metastore database uses Derby as the back-end.
- In remote mode, the metastore server and the metastore database run on different machines from the Hive server, and the metastore database can use any supported back-end.
- Remote mode is recommended for production environments, as it provides better scalability, reliability, and security than embedded mode.

Comparison of Hive with traditional databases

Hive is a data warehouse software system that provides data query and analysis on large datasets stored in Hadoop file systems. It supports a SQL-like interface called HiveQL, which can be used to perform various operations on the data, such as creating tables, loading data, querying, aggregating, and transforming. However, Hive is not a full-fledged database, and it has some differences from traditional relational databases (RDBMS) in terms of architecture, functionality, and performance. Some of the main differences are:

- Schema on read vs schema on write: Hive applies schema on read, which means that the data is not validated or enforced when it is stored, but

only when it is queried. This allows for flexibility and scalability, as the data can be stored in various formats and structures, and the schema can be changed or evolved over time. RDBMS applies schema on write, which means that the data is validated and enforced when it is inserted or updated, and the schema is fixed and predefined. This ensures data quality and consistency, but also limits the flexibility and scalability of the data storage and processing.

- Scalability and performance: Hive is designed to scale horizontally, which means that it can handle large volumes of data by adding more nodes to the cluster. Hive leverages the distributed processing power of Hadoop to run queries in parallel across multiple nodes, using MapReduce as the execution engine. However, this also means that Hive has a high latency, as it takes time to launch and run MapReduce jobs, and it does not support real-time or interactive queries. RDBMS is designed to scale vertically, which means that it can handle moderate volumes of data by adding more resources to a single node. RDBMS uses a centralized processing model, where queries are executed by a single server, using a query optimizer and an execution plan. This means that RDBMS has a low latency, as it can process queries quickly and efficiently, and it supports real-time or interactive queries.
- Data updates and transactions: Hive does not support record-level updates, insertions, or deletions, as it treats the data as immutable and append-only. Hive also does not support transactions, concurrency control, or ACID properties, as it does not have a locking mechanism or a recovery system. This simplifies the data management and reduces the overhead, but also limits the functionality and reliability of the data processing. RDBMS supports record-level updates, insertions, and deletions, as it treats the data as mutable and updatable. RDBMS also supports transactions, concurrency control, and ACID properties, as it has a locking mechanism and a recovery system. This enhances the functionality and reliability of the data processing, but also increases the overhead and complexity of the data management.

HiveQL

- HiveQL is a query language for Apache Hive, a data warehouse system that facilitates data analysis and processing using SQL-like syntax.
- HiveQL supports many features of standard SQL, such as select, where, group by, having, order by, join, union, etc.
- HiveQL also provides some extensions to SQL, such as map-reduce functions, user-defined functions, windowing and analytics functions, etc.
- HiveQL can operate on structured, semi-structured, and unstructured data stored in various formats, such as text, JSON, XML, ORC, Parquet, etc.
- HiveQL can execute queries on large-scale data sets distributed across multiple nodes using the Hadoop framework.

- HiveQL can also interact with other Hadoop components, such as HBase, Spark, Pig, etc.

Tables in Hive

- Tables in Hive are similar to tables in a relational database management system. They store data in columns and rows and belong to a database. Each table has a schema that defines the column names, data types, and constraints.
- Tables in Hive can be created using the `CREATE TABLE` statement. The syntax is as follows:

```
CREATE [TEMPORARY] [EXTERNAL] TABLE [IF NOT EXISTS] [db_name.]table_name
[(col_name data_type [COMMENT col_comment], ...)]
[COMMENT table_comment]
[PARTITIONED BY (col_name data_type [COMMENT col_comment], ...)]
[CLUSTERED BY (col_name, col_name, ...) [SORTED BY (col_name [ASC|DESC], ...)] INTO num_buck
[STORED AS file_format]
[LOCATION hdfs_path]
[TBLPROPERTIES (property_name=property_value, ...)];
```

- Tables in Hive can be classified into two types: internal and external. The main difference between them is how the data is managed and deleted.
  - Internal tables: Data is stored in the Hive data warehouse, which is located at `/hive/warehouse/` on the default storage for the cluster. When an internal table is dropped, the data and the metadata are deleted. Use internal tables when the data is exclusive to Hive and not accessed by other applications.
  - External tables: Data is stored outside the data warehouse, on any storage accessible by the cluster. When an external table is dropped, only the metadata is deleted, but the data remains intact. Use external tables when the data is shared by other applications or needs to be preserved after dropping the table.
- Tables in Hive can also be partitioned and bucketed to improve query performance and data organization. Partitioning divides a table into multiple sub-tables based on one or more columns, such as date or country. Bucketing splits the data within a partition into multiple files based on a hash function of one or more columns, such as customer_id or product_id.

Querying data in Hive

- Hive is a data warehouse system that allows users to query and analyze large datasets stored in Hadoop using a SQL-like language called Hive Query Language (HiveQL) .
- HiveQL is a declarative language that converts queries into MapReduce programs, which are executed on the Hadoop cluster .

- HiveQL supports various types of queries, such as:
  - Simple selects: to retrieve data from one or more tables or views, optionally with filters, aggregations, joins, and order by clauses .
  - Subqueries: to use the result of one query as the input of another query, either in the FROM, WHERE, or HAVING clauses .
  - Common table expressions (CTEs): to define temporary tables that can be referenced in the main query or other CTEs .
  - Window functions: to perform calculations over a set of rows that are related to the current row, such as rank, sum, or average .
  - Analytical functions: to perform complex statistical or mathematical operations on a group of rows, such as correlation, covariance, or linear regression .
- The basic syntax of a HiveQL query is:

```
SELECT [DISTINCT] column_list
FROM table_or_view [alias]
[WHERE condition]
[GROUP BY column_list [HAVING condition]]
[ORDER BY|SORT BY|CLUSTER BY|DISTRIBUTE BY column_list]
[LIMIT number];
```

- Some examples of HiveQL queries are:

```
-- Select all columns from the who table
SELECT * FROM who;

-- Select the name and age columns from the who table, where age is greater than 30
SELECT name, age FROM who WHERE age > 30;

-- Select the name and age columns from the who table, and sort them by age in descending o
SELECT name, age FROM who ORDER BY age DESC;

-- Select the name and average age of each country from the who table, and filter only the
SELECT country, AVG(age) AS avg_age
FROM who
GROUP BY country
HAVING avg_age > 40;

-- Select the name and rank of each person in the who table, based on their age within each
SELECT name, RANK() OVER (PARTITION BY country ORDER BY age DESC) AS rank
FROM who;
```

: Querying in Hive - GitHub Pages Hive Query | Make the Most of Big Data Analytics with Apache Hive
Intro to Hive Queries and How to Write Them Effectively - Pepperdata

User Defined Functions in Hive

- User defined functions (UDFs) are custom functions that can be developed in Java, integrated with Hive, and built on top of a Hadoop cluster to allow for efficient and complex computation that would not otherwise be possible with simple SQL.
- UDFs can be useful and very powerful, and yet online documentation is pretty weak.
- UDFs can be classified into three types: simple, generic, and table.
- Simple UDFs are the most common type of UDFs. They take one or more primitive types as input and return a single primitive type as output. For example, a simple UDF can convert a string to uppercase or calculate the square root of a number.
- Generic UDFs are more flexible than simple UDFs. They can take complex types such as arrays, maps, or structs as input and output. For example, a generic UDF can split a string into an array of words or concatenate two arrays into one.
- Table UDFs are also known as user defined table generating functions (UDTFs). They can take one or more primitive types as input and return a table (a set of rows) as output. For example, a table UDF can explode an array into multiple rows or parse a JSON string into a table.
- To create a UDF in Hive, the following steps are required :
  - Write the Java code for the UDF, implementing the appropriate interface (UDF, GenericUDF, or GenericUDTF) and overriding the necessary methods (evaluate, initialize, or close).
  - Compile the Java code and package it into a JAR file, along with any dependencies.
  - Register the JAR file in Hive using the ADD JAR command or the hive.aux.jars.path property.
  - Register the UDF class in Hive using the CREATE FUNCTION command, specifying the name, return type, and arguments of the UDF.
  - Use the UDF in Hive queries like a normal built-in function, invoking it by its name and passing the required arguments.
- To check which UDFs are loaded in the current Hive session, the SHOW FUNCTIONS command can be used.
- To drop a UDF from Hive, the DROP FUNCTION command can be used.

Sorting and Aggregating in Hive

- Sorting data in Hive can be achieved by using a standard `ORDER BY` clause, but it has a drawback. `ORDER BY` produces a result that is totally sorted, as expected, but to do so it sets the number of reducers to one, making it very inefficient for large datasets.
- A better alternative for sorting data in Hive is to use the `SORT BY` clause, which sorts the data within each reducer. This produces a partially sorted result that is faster and more scalable than `ORDER BY`.
- Another option for sorting data in Hive is to use the `DISTRIBUTE BY` clause,

which distributes the data among reducers based on a column or expression. This can be combined with `SORT BY` to sort the data within each reducer after distributing it.
- Aggregating data in Hive can be done by using built-in aggregate functions, such as `MAX`, `MIN`, `AVG`, `SUM`, `COUNT`, etc. These functions are usually used with the `GROUP BY` clause, which groups the data by one or more columns or expressions.
- If there is no `GROUP BY` clause specified, the aggregate functions aggregate over the whole table by default. Besides aggregate functions, all other columns that are selected must also be included in the `GROUP BY` clause.
- Hive also supports advanced aggregation by using `GROUPING SETS`, `ROLLUP`, `CUBE`, analytic functions, and windowing. These features allow for more complex and flexible aggregation queries, such as subtotals, grand totals, moving averages, rankings, etc.
- To order the aggregated results by a column or expression, the `ORDER BY` clause can be used after the `GROUP BY` clause. To order the results within each group, the `SORT BY` clause can be used instead.
- To concatenate strings in a group, the `collect_list` or `collect_set` functions can be used. These functions return an array of strings from the group, which can be further processed by using `array_sort`, `array_join`, or other array functions.

Map Reduce Scripts in Hive

- Map Reduce is a programming model for processing large-scale data sets in parallel and distributed manner.
- Hive is a data warehousing platform that supports SQL-like queries and analysis of large volumes of data stored in Hadoop.
- Hive can translate SQL queries into Map Reduce jobs and execute them on Hadoop cluster.
- Users can also plug in their own custom mappers and reducers in the data stream by using features natively supported in the Hive language.
- The TRANSFORM clause can be used to embed the custom mapper and reducer scripts in the Hive query.
- The syntax of the TRANSFORM clause is as follows:

```
SELECT TRANSFORM (input_columns)
USING 'mapper_script' [AS output_columns]
FROM input_table
[WHERE conditions]
[CLUSTER BY columns]
[MAPREDUCE 'reducer_script' [AS output_columns]];
```

- The input_columns are the columns from the input_table that are passed to the mapper_script as standard input (stdin).
- The mapper_script is any executable file that can read from stdin and

write to standard output (stdout).

- The output_columns are the columns that are produced by the mapper_script as stdout and returned by the TRANSFORM clause.
- The input_table is the source table that contains the data to be processed by the mapper_script.
- The WHERE clause can be used to filter the input data based on some conditions.
- The CLUSTER BY clause can be used to partition the output data based on some columns and send them to the reducer_script as input.
- The reducer_script is any executable file that can read from stdin and write to stdout.
- The output_columns are the columns that are produced by the reducer_script as stdout and returned by the MAPREDUCE clause.
- The mapper_script and the reducer_script can be written in any programming language, such as Python, Perl, Ruby, etc.
- The mapper_script and the reducer_script should follow the following rules:
  - They should read one line from stdin at a time and write one line to stdout at a time.
  - They should use the tab character ()^ as the delimiter for both input and output columns.
  - They should not write anything to standard error (stderr) unless there is an error.
  - They should handle any exceptions or errors gracefully and exit with a non-zero status code if there is a failure.
- The TRANSFORM and MAPREDUCE clauses can be used to perform complex data transformations and aggregations that are not supported by the built-in Hive functions or operators.
- They can also be used to leverage existing scripts or libraries that are written in other languages or frameworks.

Joins and subqueries in Hive

- Joins are used to combine data from two or more tables based on a common column or condition.
- Subqueries are queries that are nested inside another query, usually in the FROM or WHERE clause.
- Hive supports different types of joins, such as inner join, left outer join, right outer join, full outer join, and cross join.

- Hive also supports subqueries only in the FROM clause, where the subquery is treated as a table and has to be given a name or alias.
- The columns in the subquery select list are available in the outer query just like columns of a table.
- The subquery can also be a query expression with UNION, which combines the results of two or more queries.
- Hive supports arbitrary levels of subqueries, meaning that a subquery can contain another subquery inside it, and so on.
- Joins and subqueries are useful for performing complex analysis on data stored in Hive tables.

Some examples of joins and subqueries in Hive are:

- Inner join: This join returns only the records that have matching values in both tables.

```
SELECT a.col1, b.col2 FROM table1 a JOIN table2 b ON a.key = b.key;
```

- Left outer join: This join returns all the records from the left table, and the matched records from the right table. If there is no match, the right side will be null.

```
SELECT a.col1, b.col2 FROM table1 a LEFT JOIN table2 b ON a.key = b.key;
```

- Right outer join: This join returns all the records from the right table, and the matched records from the left table. If there is no match, the left side will be null.

```
SELECT a.col1, b.col2 FROM table1 a RIGHT JOIN table2 b ON a.key = b.key;
```

- Full outer join: This join returns all the records from both tables, and fills the null values with the corresponding values from the other table if there is a match.

```
SELECT a.col1, b.col2 FROM table1 a FULL JOIN table2 b ON a.key = b.key;
```

- Cross join: This join returns the Cartesian product of the two tables, meaning that every row from the first table is paired with every row from the second table.

```
SELECT a.col1, b.col2 FROM table1 a CROSS JOIN table2 b;
```

- Subquery in the FROM clause: This subquery acts as a table and can be joined with other tables or subqueries.

```
SELECT a.col1, b.col2 FROM (SELECT * FROM table1 WHERE col3 > 10) a JOIN table2 b ON a.key =
```

- Subquery with UNION: This subquery combines the results of two or more queries and can be joined with other tables or subqueries.

```
SELECT a.col1, b.col2 FROM (SELECT * FROM table1 UNION SELECT * FROM table3) a JOIN table2 b
```

- Nested subquery: This subquery contains another subquery inside it, and can be joined with other tables or subqueries.

```
SELECT a.col1, b.col2 FROM (SELECT * FROM table1 WHERE col3 IN (SELECT col4 FROM table4)) a
```

HBase

- HBase is a **column-oriented non-relational database management system** that runs on top of **Hadoop Distributed File System (HDFS)** .
- HBase provides a **fault-tolerant** way of storing **sparse data sets**, which are common in many big data use cases .
- HBase is well suited for **real-time data processing** or **random read/write access** to large volumes of data .
- HBase is modeled after **Google's Bigtable**, a distributed storage system for structured data .
- HBase does not have a **fixed database schema** in a non-relational database, which means developers can add new data without conforming to a schema model.
- HBase is ideal for **high-scale real-time applications**, such as a social media app or a streaming application.
- HBase is an **open-source** project developed as part of **Apache Software Foundation's Apache Hadoop project** .
- HBase can also run on top of **Alluxio**, a distributed in-memory file system.

HBase concepts

- HBase is a type of **NoSQL database** and is classified as a **key-value store** .
- HBase is a **column-oriented** database management system that runs on top of the **Hadoop Distributed File System (HDFS)** .
- HBase provides a **fault-tolerant** way of storing **sparse data sets**, which are common in many big data use cases.
- HBase is well suited for **real-time data processing** or **random read/write access** to large volumes of data .
- HBase is a data model that is similar to **Google's big table** designed to provide quick random access to huge amounts of structured data.
- HBase is a part of the **Hadoop ecosystem** that provides random real-time read/write access to data in the Hadoop File System.
- HBase is **horizontally scalable**, which means it can handle increasing data and load by adding more nodes to the cluster.
- HBase has a **master-slave** architecture, where a **HMaster** node manages the cluster and **HRegionServer** nodes store portions of the tables and perform the work on the data .
- HBase tables are divided into **regions**, which are contiguous ranges of rows that are stored together.
- Each region is assigned to a region server, which can serve multiple regions.
- Each region server is responsible for handling read and write requests, splitting regions, and reporting to the master.

- Each HBase table consists of one or more **column families**, which are logical groupings of columns that share common characteristics.
- Each column family contains one or more **columns**, which are identified by a **qualifier**.
- Each column can have multiple **versions**, which are timestamped values that are stored in descending order.
- Each row in an HBase table is identified by a unique **row key**, which is a byte array that can be any length.
- HBase supports **CRUD** operations (create, read, update, delete) on the data, as well as **scan** operations to retrieve a range of rows.
- HBase also supports **filters**, **coprocessors**, **bulk loading**, and **secondary indexing** to enhance the functionality and performance of the database.

HBase clients

- HBase clients are applications or libraries that can interact with HBase using its API or other protocols.
- HBase clients can perform various operations on HBase, such as creating, deleting, updating, and querying tables and data.
- HBase clients can be written in different programming languages, such as Java, Python, Ruby, Scala, and C++.
- HBase clients can use different methods to connect to HBase, such as the HBase shell, the Thrift server, the REST server, or the native Java API.
- HBase clients can use different features of HBase, such as filters, scanners, coprocessors, and security.
- HBase clients can benefit from the scalability, performance, and reliability of HBase, which is built on top of Hadoop and HDFS.
- HBase clients can also face some challenges, such as tuning the configuration, handling exceptions, and managing concurrency .

: https://www.oreilly.com/library/view/hbase-essentials/9781783987245/ch06.html
https://subscription.packtpub.com/book/big-data-and-business-intelligence/9781783987245/6
https://www.ibm.com/topics/hbase

HBase example

- HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS) .
- HBase provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases .
- HBase is well suited for real-time data processing or random read/write access to large volumes of data .
- An HBase table consists of rows and columns, where each column belongs to a column family .
- An HBase column represents an attribute of an object; for example, if the table is storing diagnostic logs from servers, each row might be a log

record, and a typical column could be the timestamp of when the log record was written, or the server name where the record originated .

- HBase supports CRUD (create, read, update, delete) operations using the HBase shell, Java API, or REST API .
- HBase also supports filters, scanners, and coprocessors for advanced data manipulation and processing .
- HBase can be integrated with other frameworks such as Spark, Hive, and Pig for data analysis and transformation .
- HBase is used in various domains such as healthcare, e-commerce, sports, social media, and more .
- For instance, in the healthcare sector, HBase is used for storing genome sequences and disease history of people or a particular area .
- In the field of e-commerce, HBase is used for storing logs about customer search history and it also performs analytics and target advertisement for better business insights .
- In sports, HBase is used to store match details and the history of each match .

HBase vs RDBMS

- HBase and RDBMS are both types of database management systems, but they differ in several ways.
- Some of the main differences are:
  - Data Model: RDBMS uses a relational data model, where data is stored in tables with predefined columns and rows. HBase, on the other hand, uses a column-family data model, where data is stored in column families, which contain columns and rows. HBase is often referred to as a NoSQL database because of its non-relational data model .
  - Scaling: RDBMS is designed to scale vertically, which means adding more resources to a single server. HBase is designed to scale horizontally, which means adding more servers to a cluster. HBase can handle large amounts of data by distributing it across multiple nodes in a Hadoop Distributed File System (HDFS) .
  - Consistency: RDBMS follows the ACID (Atomicity, Consistency, Isolation, Durability) properties, which ensure that transactions are reliable and consistent. HBase follows the BASE (Basically Available, Soft state, Eventual consistency) properties, which trade off strong consistency for high availability and performance .
  - Speed: RDBMS is optimized for fast and complex queries that involve joins, aggregations, and calculations. HBase is optimized for fast and simple queries that involve key-value lookups, scans, and filters. HBase can perform real-time analysis on large-scale data .
  - ACID Compliance: RDBMS is fully ACID compliant, which means it guarantees that transactions are atomic, consistent, isolated, and

durable. HBase is partially ACID compliant, which means it guarantees that transactions are atomic and durable, but not necessarily consistent and isolated. HBase supports single-row transactions, but not multi-row transactions .
  - JOINs: RDBMS supports JOINs, which are operations that combine data from multiple tables based on a common attribute. HBase does not support JOINs, which means that data has to be denormalized and stored in a single table or joined in the application layer .
  - Referential Integrity: RDBMS supports referential integrity, which is a constraint that ensures that the data in one table matches the data in another table. HBase does not support referential integrity, which means that the data in one table may not match the data in another table .

- HBase and RDBMS have different strengths and weaknesses, and they are suitable for different types of applications.
- RDBMS is more suitable for traditional, transactional applications that require strong consistency, complex queries, and referential integrity. Examples of such applications are online banking, e-commerce, and inventory management .
- HBase is more suitable for big data applications that require horizontal scaling, high-speed processing, and real-time analysis. Examples of such applications are social media, web analytics, and recommendation systems .

Advanced usage of HBase

HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS). It provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases. It is well suited for real-time data processing or random read/write access to large volumes of data.

Some of the advanced usage of HBase are:

- Storing and querying genome sequences and disease history in the healthcare sector .
- Storing and analyzing customer search history and performing targeted advertisement in the e-commerce sector .
- Storing and retrieving match details and history in the sports sector.
- Using row key and column key to convey meaning and exploit sorting order for solving common problems in designing storage solutions.
- Running MapReduce jobs on HBase tables for batch processing .
- Using filters, coprocessors, and custom comparators to enhance the performance and functionality of HBase.
- Using HBase shell, Java API, or REST API to interact with HBase .

- Using HBase replication, backup, and restore features to ensure data availability and durability .

Schema design in HBase

- HBase is a NoSQL database that stores data in a tabular format, where each row has a unique key and each column belongs to a column family.
- HBase does not support joins, but it allows denormalization and nested entities, which are columns that store complex values as serialized bytes .
- HBase schema design is driven by the access patterns and performance requirements of the application, rather than the structure and relationships of the data.
- Some general principles for HBase schema design are :
  - Choose a row key that is unique, short, and sortable, and that supports the most frequent queries.
  - Use column families to group related columns together, and keep the number of column families low (usually less than five).
  - Use column qualifiers to store dynamic or sparse attributes, and avoid using them as secondary indexes.
  - Use filters, scanners, and coprocessors to optimize read performance and reduce network traffic.
  - Use compression, bloom filters, and block cache to reduce disk and memory usage.
  - Use versioning, TTL, and compaction to manage data lifecycle and garbage collection.

Advanced Indexing in HBase

- HBase is a column-oriented NoSQL database management system that runs on top of the Hadoop Distributed File System (HDFS) .
- HBase has a single primary index that is lexicographically sorted on the row key. Access to records in any way other than through the row key requires scanning over potentially all the rows in the table to test them against a filter .
- Secondary indexing is a technique to improve the performance of queries that use non-row key columns as filters or predicates. Secondary indexes allow direct access to records based on the values of those columns, without scanning the whole table .
- HBase does not provide native support for secondary indexing. There are several approaches to implement secondary indexing on HBase, each with its own advantages and disadvantages . Some of the common approaches are:
  - Application-side indexing: The application maintains a separate table for each secondary index, and updates it manually whenever the main table is updated. This approach gives the application full control over the index schema and consistency, but also adds complexity

and overhead to the application logic .
  - Coprocessor-based indexing: The HBase coprocessor framework allows custom code to run on the server-side, and intercept events such as data mutations and scans. This approach can be used to automatically update the secondary index tables whenever the main table is updated, without involving the application. This approach reduces the application complexity and network traffic, but also introduces challenges such as index maintenance, fault tolerance, and load balancing .
  - External indexing: The external indexing approach uses a separate system, such as Solr or Elasticsearch, to index the data stored in HBase. This approach can leverage the advanced features and capabilities of the external system, such as full-text search, faceting, and aggregation. However, this approach also requires additional infrastructure and synchronization mechanisms to keep the index and the data consistent .

Zookeeper

A zookeeper is a person who works in a zoo and is responsible for the care and management of the animals. Some of the duties of a zookeeper are:

- Feeding, cleaning, and monitoring the health and behavior of the animals.
- Providing enrichment activities and training for the animals to stimulate their natural behaviors and enhance their well-being.
- Assisting with veterinary procedures, such as vaccinations, treatments, and surgeries.
- Maintaining the enclosures, habitats, and equipment of the animals.
- Educating the public and visitors about the animals and conservation issues.
- Participating in research and breeding programs for endangered species.

Some of the skills and qualifications of a zookeeper are:

- A high school diploma or equivalent, and preferably a bachelor's degree in zoology, biology, animal science, or a related field.
- Experience working with animals, either as a volunteer, intern, or employee in a zoo, wildlife center, farm, or veterinary clinic.
- Knowledge of animal behavior, nutrition, health, and welfare.
- Physical fitness, stamina, and ability to work in various weather conditions and handle heavy lifting.
- Communication, teamwork, and problem-solving skills.
- Passion, dedication, and respect for animals and their conservation.

Some of the benefits and challenges of being a zookeeper are:

- Working with a variety of animals and learning about their natural history and ecology.

- Contributing to the conservation and education of wildlife and their habitats.
- Developing a close bond and rapport with the animals and colleagues.
- Facing potential risks of injury, disease, and stress from working with wild animals.
- Dealing with ethical and emotional issues, such as euthanasia, animal rights, and public criticism.
- Working long, irregular, and sometimes night hours, including weekends and holidays.

Zookeeper concepts

- Zookeeper is a distributed application that provides coordination services for distributed systems.
- Zookeeper has a simple client-server model, where clients are nodes (machines) that use the services, and servers are nodes that provide the services.
- Zookeeper exposes common services such as naming, configuration management, synchronization, and group services in a simple interface.
- Zookeeper can be thought of as a distributed file system, where each node in the system has a path name and can store some data.
- Zookeeper maintains a hierarchical namespace of znodes, which are data nodes that can have children and data associated with them.
- Zookeeper guarantees that the data in the znodes is consistent, ordered, and atomic across the cluster.
- Zookeeper uses a leader-follower model, where one server acts as the leader and coordinates the updates from the followers.
- Zookeeper uses a quorum-based protocol, where a majority of servers must agree on a change before it is committed.
- Zookeeper supports ephemeral znodes, which are znodes that are automatically deleted when the client session that created them ends.
- Zookeeper supports watches, which are notifications that a client can set on a znode to be alerted when the znode changes.

How ZooKeeper helps in monitoring a cluster

- ZooKeeper is a centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services for distributed applications.
- ZooKeeper hosts are deployed in a cluster and, as long as a majority of hosts are up, the service will be available.
- ZooKeeper helps in monitoring a cluster by providing the following features:
  - **Status**: ZooKeeper exposes the status of each server in the cluster, such as the mode (leader, follower, observer, standalone), the state (serving, looking, following, leading), the connections, the latency,

the outstanding requests, the zxid (ZooKeeper transaction id), and the epoch .
  - **Metrics**: ZooKeeper collects and reports various metrics about the performance and health of the cluster, such as the number of requests, the number of watches, the number of nodes, the data size, the average latency, the min/max latency, the errors, the leader election time, and the quorum size.
  - **Consistency**: ZooKeeper ensures that the total node count inside the ZooKeeper tree is consistent across the cluster, and that any changes to the data are replicated to all the servers in a timely manner.
  - **Synchronization**: ZooKeeper provides distributed synchronization primitives, such as locks, barriers, queues, and leader election, that help coordinate the actions and state of the cluster nodes.
  - **Configuration**: ZooKeeper stores and manages the configuration information of the cluster, such as the server list, the client port, the tick time, the session timeout, the data directory, and the authentication scheme.
  - **Naming**: ZooKeeper provides a hierarchical namespace for naming and identifying the cluster nodes and resources, and allows clients to create, read, update, and delete nodes (called znodes) in the namespace.

How to build applications with Zookeeper

Zookeeper is a distributed system coordinator that provides services such as configuration management, synchronization, naming, and leader election. Zookeeper can help developers to build reliable and scalable distributed applications by simplifying the coordination logic and reducing the complexity of the system.

To build applications with Zookeeper, the following steps are required:

- Download and install Zookeeper from the official website or use a package manager such as apt or yum.
- Configure Zookeeper by creating a zoo.cfg file that specifies the server ID, data directory, client port, and other parameters. For a cluster of Zookeeper servers, also specify the server list and the quorum size.
- Start Zookeeper by running the zkServer.sh script with the start command. For a cluster, start each server on a different machine or port.
- Connect to Zookeeper using a client library such as the official Java API, the C API, or the Python API. The client library provides methods to create, read, update, and delete znodes, which are the basic units of data in Zookeeper. Znodes can store data, have children, and have watches that notify the client of changes.
- Use Zookeeper to implement distributed features such as configuration management, synchronization, naming, and leader election. For example,

to store configuration data, create a znode with the data and watch it for changes. To synchronize processes, use barriers or locks that are implemented by creating ephemeral znodes. To register and discover services, use a naming scheme that maps service names to znodes. To elect a leader, use a recipe such as the leader election algorithm that creates sequential znodes and compares them.

For more details and examples, refer to the official documentation and tutorials of Zookeeper.

IBM Big Data Strategy

- IBM, a US-based computer hardware and software manufacturer, had implemented a Big Data strategy, where the company offered solutions to store, manage, and analyze the huge amounts of data generated daily and equipped large and small companies to make informed business decisions .
- IBM's Big Data strategy was part of its Smarter Planet initiative, which sought to highlight how government and business leaders were capturing the potential of smarter systems to achieve economic and sustainable growth and societal progress.
- IBM's Big Data strategy consisted of six steps:
  - Understand your business objectives: Connect your data strategy with the business strategy and identify the key data-driven outcomes and use cases.
  - Assess your current state: Unpack pain points to reveal blockers and gaps in your data landscape and capabilities, and benchmark your data maturity against industry standards.
  - Map out data strategy framework: Define your data's target state, architecture, governance, quality, security, privacy, and ethics, and align them with your business objectives and use cases.
  - Prioritize data initiatives: Identify and prioritize the data initiatives that will deliver the most value and impact, and create a roadmap and business case for implementation.
  - Execute data initiatives: Implement the data initiatives using agile and iterative methods, and leverage best practices, tools, and accelerators to speed up delivery and reduce risk.
  - Monitor and optimize data performance: Measure and track the outcomes and benefits of the data initiatives, and continuously optimize and improve the data strategy based on feedback and changing needs.
- IBM's Big Data strategy also involved partnering with leading open source platforms and technologies, such as Cloudera, to provide enterprise-grade, secure, governed, and scalable data lakes and analytics solutions.
- IBM's Big Data strategy aimed to help its clients achieve the following benefits:
  - Increase data-driven innovation and insights: Unlock the value of

data and AI across the enterprise and enable faster and smarter decision making.
  - Enhance data quality and trust: Ensure data accuracy, completeness, consistency, and timeliness, and foster a culture of data stewardship and accountability.
  - Improve data efficiency and agility: Streamline and automate data processes and workflows, and enable data self-service and collaboration across teams and functions.
  - Reduce data complexity and cost: Simplify and rationalize data architectures and systems, and optimize data storage and consumption.
  - Mitigate data risk and compliance: Protect data assets and users from cyber threats and breaches, and adhere to data regulations and standards.

IBM Big Data Strategy

- IBM, a US-based computer hardware and software manufacturer, had implemented a Big Data strategy, where the company offered solutions to store, manage, and analyze the huge amounts of data generated daily and equipped large and small companies to make informed business decisions .
- IBM's Big Data strategy was part of its Smarter Planet initiative, which sought to highlight how government and business leaders were capturing the potential of smarter systems to achieve economic and sustainable growth and societal progress.
- IBM's Big Data strategy consisted of six steps:
  - Understand your business objectives: Connect your data strategy with the business strategy and identify the key data-driven outcomes and metrics.
  - Assess your current state: Unpack pain points to reveal blockers and gaps in your data capabilities, processes, and culture.
  - Map out data strategy framework: Define your data's target state, including the data architecture, governance, quality, security, and privacy requirements.
  - Prioritize data initiatives: Align your data initiatives with your business objectives and prioritize them based on value, feasibility, and risk.
  - Build a data roadmap: Break down your data initiatives into actionable steps and milestones and assign roles and responsibilities.
  - Execute and monitor: Implement your data initiatives and track their progress and impact using data-driven feedback loops.
- IBM's Big Data strategy also leveraged its portfolio of products and services, such as IBM Cloud Pak for Data, IBM Watson, IBM Db2, IBM Cognos, IBM SPSS, and IBM DataStage, to provide end-to-end data solutions for various industries and use cases.
- IBM's Big Data strategy aimed to help its clients achieve the following

benefits:
  - Give data assets and accelerators top priority: Develop a process and culture around data that enables true standardization, re-use, portability, speed to action and risk reduction across the end-to-end data lifecycle.
  - Enable data democratization and self-service: Empower data consumers and producers to access, share, and collaborate on data with minimal IT intervention and maximum governance and security.
  - Adopt a hybrid cloud and AI approach: Harness the power of cloud and AI to scale and optimize your data capabilities and unlock new insights and opportunities.
  - Foster data innovation and experimentation: Encourage a data-driven mindset and culture that supports testing, learning, and iterating on data initiatives and solutions.
  - Align data strategy with business strategy: Ensure that your data strategy is aligned with your business strategy and supports your strategic goals and objectives.

Introduction to Infosphere

- The term infosphere (information + sphere) refers to a metaphysical realm of information, data, knowledge, and communication, populated by informational entities called inforgs (or, informational organisms) .
- The infosphere is the whole system of services and documents, encoded in any semiotic and physical media, whose contents include any sort of data, information and knowledge, with no limitations either in size, typology, or logical structure .
- The infosphere can be seen as the environment in which inforgs interact and communicate, and as the context in which information ethics and information governance are applied .
- The infosphere can also refer to a specific data integration platform, such as IBM InfoSphere Information Server, that helps users to more easily understand, cleanse, monitor and transform data .
- The infosphere can also refer to a fictional setting, such as the Futurama Wiki, that documents the episodes, characters, and merchandise of the animated sitcom Futurama .

Introduction to BigInsights

- BigInsights is an IBM product that helps firms analyze the increasing volume, velocity and veracity of data of their interest .
- BigInsights is based on open source technologies, such as Apache Hadoop, and IBM-specific technologies, such as BigSheets and Big SQL .
- BigInsights does not replace a relational database management system (DBMS) or a traditional data warehouse, but rather complements them by providing a scalable and flexible platform for processing and analyzing

various types of data, including data that is loosely structured or largely unstructured, such as text data .
- BigInsights enables users to perform different types of analysis on data, such as batch processing, interactive querying, text analytics, machine learning, and graph analytics, using a variety of tools and languages, such as MapReduce, Pig, Hive, Jaql, R, and Python .
- BigInsights also provides a web-based console for managing and monitoring the cluster, as well as a rich set of features for security, governance, and integration with other systems .

Introduction to Big Sheets

- Big Sheets is a user interface program that allows non-technical business users to gather, analyze and visualize large amounts of data from different sources.
- Big Sheets can translate untapped data into actionable business insights by using a spreadsheet-like interface that supports familiar functions, formulas, charts and pivot tables .
- Big Sheets is powered by Google BigQuery, which is a big data analysis product from Google Cloud that can handle millions or billions of rows of data.
- Big Sheets can connect to data sources such as Google Drive, Google Cloud Storage, Google Sheets, BigQuery tables and external databases.
- Big Sheets can help users explore the risk factors, trends, patterns and correlations in their data, and share their findings with others.

Introduction to Big SQL

- Big SQL is a high performance massively parallel processing (MPP) SQL engine for Hadoop that makes querying enterprise data from across the organization an easy and secure experience .
- Big SQL can quickly access a variety of data sources including HDFS, RDBMS, NoSQL databases, object stores, and WebHDFS by using a single database connection or query .
- Big SQL supports ANSI-compliant SQL syntax and semantics, as well as advanced SQL features such as window functions, common table expressions, and user-defined functions.
- Big SQL leverages the power of the Hadoop ecosystem and integrates with tools such as Apache Spark, Apache Hive, Apache HBase, Apache Ranger, and Apache Atlas.
- Big SQL provides scalability, reliability, and security for enterprise data analytics and data lake solutions.